

DEEP LEARNING [23AML161]

EXERCISES

MODULE 1	3
CHAPTER: Training Deep Neural Networks	3
MODULE 2	5
CHAPTER: Convolutional Neural Networks.....	6
MODULE 3	9
CHAPTER: Sequence Processing	10
MODULE 4	12
CHAPTER: Text Processing	12
MODULE 5	13
CHAPTER: Representation Learning using Autoencoders.....	14
ANSWERS.....	14

NOTES:

- Hints to the answers refer to the 3rd edition of the first reference book [T1] mentioned in the syllabus. The soft copy of the book was already shared in Microsoft Teams.
- As the soft copy of the first textbook does not have page number imprinted on its pages, the page numbers mentioned in the exercises refer to the page number generated by the reader tool (such as Adobe Reader, web browser, etc.) that opens the document.
- Hints should be mapped only to the highlighted text in the book.

CIA - I

MODULE 1

CHAPTER: Training Deep Neural Networks

1. Identify what causes vanishing/exploding gradient problem during backpropagation and how weight initialization techniques can alleviate this problem.
Hint: From section “The Vanishing/Exploding Gradients Problems” through sub-section “Glorot and He Initialization” [T1, pg. 453 – 456].
2. Identify the problem that *ReLU (Rectified Linear Unit)* activation function suffers from, and how *Leaky ReLU*, *Exponential LU (ELU)* and *Scaled ELU (SELU)* can solve that problem. Provide required diagrams, expressions and code-snippet.
Hint: From sub-section “Better Activation Functions” through end of paragraph “ELU and SELU” [T1, pg. 456 – 461].
3. Identify the problem that activation function *Rectified Linear Unit (ReLU)* suffers from, and how *Gaussian ELU (GELU)* and *Swish* can solve that problem. Also mention how GELU can be approximated for faster computation and how Swish can approximate GELU (at a particular value for its hyperparameter). Provide required diagrams and expressions.
Hint: From sub-section “Better Activation Functions” through end of paragraph “GELU, Swish, and Mish”. Skip paragraphs for “Leaky ReLU” and “GELU and SELU” [T1, pg. 456, 461 – 463].
4. Identify the working principle of Batch Normalization (BN) to alleviate the vanishing/exploding gradient problem. Also apply fusing BN layer with its previous layer after training can be used to avoid its runtime penalty.

Hint: From sub-section “Batch Normalization” till before paragraph “Implementing batch normalization with Keras”. [T1, pg. 464 – 467].

5. Identify when to consider reusing pretrained layers. Mention all major considerations with examples and provide a figure visualizing the reuse of layers.

Hint: Annotated sub-section “Reusing Pretrained Layers”. [T1, pg. 472 – 474].

6. Explain *Momentum* and *Nesterov Accelerated Gradient (NAG)* optimizer and justify the reason the latter one being the fastest than the former one. Also provide comparison diagram and algorithm for Momentum.

Hint: Sub-section “Momentum” and “Nesterov Accelerated Gradient”. [T1, pg. 481 – 483].

7. Develop the intuition of how *AdaGrad* optimizer works and contrast it with *RMSProp* mentioning the improvement of the latter one over the former one. Provide algorithms for both optimizers.

Hint: Annotated sub-section “Adam” and “RMSProp”. [T1, pg. 486 – 486].

8. Explain the working principles of Adam optimizer and identify the modifications made into AdamW optimizer to address the overfitting issue of the former.

Hint: Annotated sub-section “Adam” and “AdamW”. [T1, pg. 486 – 489].

9. Apply reasons to prefer learning rate scheduling over the manual process of finding an appropriate learning rate. Also explain a few of the popular learning rate scheduling strategies.

Hint: Section “Learning Rate Scheduling”. [T1, pg. 491 – 494].

10. Explain the working principles of dropout regularization technique with a neat figure showing a snapshot of training step indicating drop out neurons. Also, develop code-snippet showing its application.

Hint: Sub-section “Dropout”. [T1, pg. 499 – 503].

11. Build intuition on how *Monte Carlo* dropout can boost the performance of any trained dropout model without retraining and can handle prediction uncertainty. Also explain how *max-norm regularization* works.

Hint: Sub-section “Monte Carlo (MC) Dropout” and “Max-Norm Regularization”. [T1, pg. 503 – 507].

MODULE 2

CHAPTER: Convolutional Neural Networks

12. Explain the general architecture of a *convolutional layer* with respect to image recognition tasks and identify the need for *padding* and *stride*. Provide appropriate diagrams.

Hint: Section Convolutional Layers with figure 14-2, 14-4 before sub-section “Filters” [T1, pg. 615 – 618]

13. How do multiple feature maps work in a convolutional layer with respect to provide better representation of its inputs. Provide a diagram depicting multiple filters each processing a color image and outputting feature maps, and equation for computing the output of a neuron in a convolutional layer.

Hint: Sub-section “Stacking Multiple Feature Maps” with figure 14-6 and equation 14-1 with description [T1, pg. 619 – 622]

14. Solve the following by considering a *convolutional layer* with 200 filters with 5 x 5 kernel, stride of 1 and “same” padding. The input is 150 x 100 three channel images.
- What is the total number of parameters in the convolutional layer?
 - What is the total number of floating-point multiplications to be computed in that layer for each input?
 - Calculate the memory requirement [in GB] of that layer for its output for a batch of 100 input images if feature maps are represented using 32-bit floats?

Hint: Refer answer section

15. Solve the following by considering a *convolutional Neural Network* composed of two convolutional layers, each with 3 × 3 kernels, a stride of 2, and “same” padding. The first layer outputs 100 and the last one outputs 200 feature maps. The inputs are 200 × 300 pixels three channel color images.
- What is the total number of parameters in the CNN?
 - Calculate the minimum memory requirement [in megabytes] of this network to make a prediction for a single instance if feature maps are represented using 32-bit floats?
 - Calculate the minimum memory requirement [in megabytes] of this network when training on a mini batch of 50 images?

Hint: Refer answer section

16. Identify the need for pooling layers and explain its working over all of its popular variants (i.e. average, max and global average pool). Provide an appropriate figure depicting the main pooling concept.

Hint:

- Section “Pooling layers” including annotated sub-section “Translation Invariance” till phrase “...it is possible to get some level of translation invariance at a larger scale.” with figure 14-10 [T1, pg. 628 – 630]
- Annotated sub-section “Max Pooling” and “Average Pooling vs. Max Pooling” [T1, pg. 631]
- Annotated sub-section “Global Average Pooling” [T1, pg. 633]

17. With appropriate code-snippets, explain the general construction of a convolutional neural network. Also, identify the general guidelines for using convolution kernels.

Highlighted content in section “CNN Architectures” before sub-section “LeNet -5” with figure 14-12 [T1, pg. 634 – 636]

Hint: Annotated Tip sub-section “Guidelines of Using Convolution Kernels” for the second part of the question [T1, pg. 634]

18. Solve by calculating the spatial dimension of the feature maps for the pooling and convolutional layers given stride of 1 and padding as “valid” in the below mentioned LeNet-5 architecture. Also, calculate the number of (trainable) parameters in the respective layer.

Layer	Type	Maps	Size	Kernel Size	Stride	Parameters
Out	Fully connected	—	10	—	—	?
F6	Fully connected	—	84	—	—	?
C5	Convolutional	120	?	5 x 5	1	?
S4	Avg. pool	16	?	2 x 2	2	—
C3	Convolutional	16	?	5 x 5	1	?
S2	Avg. pool	6	?	2 x 2	2	—
C1	Convolutional	6	?	5 x 5	1	?
In	Input	1	32 x 32	—	—	—

Hint: Refer answer section

19. Discuss how data augmentation works. Also, identify what problem a local response normalization solves and how.

Hint: Highlighted text in sub-section “Data Augmentation” [T1, pg. 640 – 641] and annotated sub-section “Local Response Normalization” [T1, pg. 641 – 642]

20. Identify how inception modules contribute to GoogLeNet to perform better than LeNet-5 and AlexNet with much less parameters than these models. Draw a neat diagram of an inception module.

Hint: Highlighted text in sub-section “GoogLeNet” with figure 14-14 till phrase “...Now let’s look at the architecture of the GoogLeNet CNN (see Figure 14- 15)” [T1, pg. 642 – 644]

For CIA - II

21. Explain how residual learning works to train a very deep neural network overcoming the vanishing gradient problem. Provide all relevant figures.

Hint: Highlighted text in sub-section “ResNet” with figure 14-16, 14-17 and 14-19 [T1, pg. 647 – 650]

22. With an appropriate diagram, explain the concept behind *depthwise separable convolution layer* used in Xception architecture. Also, identify why this architecture is considered as a GoogLeNet variant.

Hint: Highlighted text in sub-section “Xception” with fig. 14-20 [T1, pg. 651 – 652]

23. a) Explain with an appropriate diagram how metric *Intersection over Union* works to measure the bounding box prediction performance.

b) Identify the working principle of object localization in images using sliding-CNN approach.

Hint: Highlighted text from annotated sub-section from “Better metric for bounding box prediction” to sub-section “Object Detection” with figure 14-24 and 14-25 [T1, pg. 670 – 674]

24. Develop design considerations to replace a dense layer at the top with a convolutional layer in a CNN in object detection task and mention its benefits. With an appropriate figure, explain how a Fully Convolutional Network processes a varying length image.

Hint: Highlighted text in sub-section “Fully Convolutional Networks”, with figure 14-26 [pg. 674 – 677]

25. Identify how semantic segmentation works covering both traditional transposed convolutional layer and skip connections approach in a fully convolutional network. Provide appropriate diagrams.

Hint: Highlighted text in sub-section Semantic Segmentation with fig. 14-28 and 14-29 [pg. 682 - 686]

MODULE 3

CHAPTER: Sequence Processing

26. Illustrate the working of a recurrent neural network and identify how it is different from feed-forward neural network. Provide appropriate diagrams and expressions to calculate output of a recurrent layer.

Hint: Section “Recurrent Neurons and Layers”, [T1, pg. 693 – 696 before sub-section “Memory Cells”] with figure 15-1, 15-2 and expression 15-1, 15-2.

27. Explain the architecture and the working of *sequence-to-sequence* network, *sequence-to-vector* network, *vector-to-sequence* network and *encoder-decoder* network. Also, develop intuition for strategy “*backpropagation through time*” to train a recurrent network. Provide appropriate diagrams.

Hint: Sub-section “Input and Output Sequences” and “Training RNNs” with figure 15-4 and 15-5 [T1, 697 – 699]

28. Explain the following with respect to time-series forecasting. Provide relevant examples.

- a) Univariate & Multivariate time-series,
- b) Seasonality
- c) Lagged time-series & Differencing
- d) Autocorrelated time-series
- e) Stationary time-series

29. Make use of simple recurrent neural network (RNN) layers to develop a deep RNN giving code-snippet and explain its working. Assume appropriate considerations and provide the network architecture diagram along with justification for return sequences in recurrent layers.

Hint: Sub-section “Forecasting Using a Deep RNN” with code-snippet and figure 15-10 [pg. 717 – 719].

30. Choose appropriate measure to alleviate unstable gradient problem in handling long sequences with a recurrent neural network (RNN) and discuss the solutions.

Hint: Highlighted text in sub-section “Fighting the Unstable Gradients Problem” [T1, pg. 727 – 729]

31. Experiment with an LSTM (*long short-term memory*) cell to tackle short-term memory problem. Provide network architecture diagram, relevant code-snippet and equations for LSTM computations.

Hint: Sub-section “LSTM cells” with code-snippet, figure 15-12 and equation 15-4 [T1, pg. 731 – 734]

32. Identify the simplifications made into gated recurrent unit (GRU) over long short-term memory (LSTM). Show architecture of a GRU cell and provide equations for GRU computations.

Hint: Sub-section “GRU” with figure 15-13 and equation 15-5 for the first part [T1, pg. 735 – 736].

33. Identify how learning long-term patterns in sequence can be extended by using 1-dimensional convolutional layer along with GRU layer in a sequential model. Provide appropriate code-snippet.

Highlighted sub-section “Using 1D convolutional layers to process sequences” [T1, pg. 736 – 737] with code-snippet.

MODULE 4

CHAPTER: Text Processing

34. Build a language model over a “stateless” character recurrent neural network (Char-RNN) to predict the next character in a sentence considering an appropriate dataset. Provide all required code-snippets and explanations.

Hint: Section “Generating Shakespearean Text Using a Character RNN” with figure 16-1 [T1, pg. 743 – 747]

35. Identify the design considerations for choosing a stateful RNN over a stateless RNN for text generation task. Also, build a language model over a “stateful” character RNN to predict the next character in a sentence considering an appropriate dataset and providing all required explanations.

Hint: Sub-section “Stateful RNN” with figure 16-2 [T1, pg. 749 - 750]

36. Develop code-snippets to build a recurrent neural network for sentiment classification considering an appropriate dataset and explain all the modeling steps. How can model performance be improved by applying masking on padding tokens?

Hint: Section “Sentiment Analysis” with sub-section “Masking” till phrase “... but it will not propagate it any further, since `return_sequences` is not set to `True`.” [T1, 751 – 754]

37. Build a simple machine translation model using an encoder-decoder recurrent neural network considering appropriate assumptions. Provide all design considerations, model architecture, and code-snippets.

Hint: Section “An Encoder–Decoder Network for Neural Machine Translation” till before phrase “Let’s build and train this model! ...” with figure 16-3 and 16-4 [T1, pg. 757 – 759]

38. Make use of *Bidirectional RNNs* and *Beam Search* to improve natural language translation performance by identifying all their design considerations. Provide required figures and code-snippets.

Hint: Sub-section “Bidirectional RNNs” and “Beam Search” with figure 16-5 and 16-6 [T1: 763 – 766]

39. Identify how *Bahdanau attention mechanism* alleviates the short-term memory limitation of RNNs. Provide a figure and equation for this attention mechanism.

Hint: Section “Attention Mechanisms” till before phrase “Another common attention mechanism, known as Luong attention...” with figure 16-7 [T1, pg. 766 – 769]

40. Construct a neat diagram of an original transformer architecture [2017] and identify all its design aspects and working principles in context of neural machine translation.

Hint: Sub-section “Attention Is All You Need: The Original Transformer Architecture” till before “Positional encodings” with fig. 16-8 [T1, pg. 769 – 772]

41. Identify how positional encodings inject word order information into Transformer models. Provide its equation and illustrative visual representation to support the answer.

Hint: Annotated sub-section “Positional encodings” till before “Multi-head attention” with eq. 16-2 and fig. 16-9 [T1, pg. 772 – 773]

42. Make use of *scaled dot-product attention layer* (provide equation) and develop intuition how *multi-head attention* works also showing its architecture diagram.

Hint: Annotated sub-section “Multi-head attention” with eq. 16-3 and fig. 16-10 [T1, pg. 774 – 777]

43. Develop code-snippet to show the working of the following using Hugging Face’s Transformers library over an appropriate task.

- a. Default inference pipeline with a specific model
- b. Inference using Auto-classes for both tokenization and model
- c. Model fine-tuning

Hint: Annotated text in sub-section “Hugging Face’s Transformers Library” with all highlighted code-snippets [T1, pg. 785 – 788]

MODULE 5

CHAPTER: Representation Learning using Autoencoders

44. Explain the general architecture of an autoencoder outlining the design and working principles. Also build a simple undercomplete linear autoencoder over appropriate assumptions.
Hint: From annotated sub-section “Parts of a simple autoencoder” till the end of section “Performing PCA with an Undercomplete Linear Autoencoder” with figure 17-1 [T1, pg. 795 – 798]
45. With a neat block diagram, explain the general architecture of a stacked autoencoder outlining the design considerations and working principles. Also, develop code-snippet to implement it with appropriate assumptions and explanations.
Hint: From annotated sub-section “Stacked Autoencoders” till before sub-section “Visualizing the Reconstructions” with figure 17-3 [T1, pg. 799 – 801]
46. Make use of a *stacked autoencoder* to train the model in an unsupervised way over a large dataset that is mostly unlabeled. Also, discuss autoencoder training techniques “weight tying” and “greedy layer-wise training”. Provide appropriate visual representations wherever required.
Hint: Annotated sub-section “Unsupervised Pretraining Using Stacked Autoencoders”, “Tying Weights” and “Training One Autoencoder at a Time” with fig. 17-6, 17-7 [T1, pg. 804 – 808]
47. Explain the working of *convolutional autoencoder* and *denoising autoencoders*. Develop code-snippets to implement each of these.
Hint: Highlighted text in section “Convolutional Autoencoders” and “Denoising Autoencoders” with figure 17-8 [T1, pg. 809 – 811]
48. Develop code-snippets to build a *sparse autoencoder* using both standard ℓ_1/ℓ_2 regularization and *Kullback-Leibler divergence* and explain the working of each of these approaches. Provide sparsity loss plot for ℓ_1 , ℓ_2 and Kullback-Leibler divergence.
Hint: Section “Sparse Autoencoders” figure 17-10, equation 17-1, 17-2 [T1, pg. 812 – 816]
49. Identify the characteristics and working principles of *variational autoencoders*. Construct a neat diagram showing its architecture and develop relevant code-snippets for its implementation.
Hint: Section “Variational Autoencoders” and “Generating Fashion MNIST Images” with figure 17-11 and equation 17-3, 17-4 [T1, pg. 816 - 822]

ANSWERS

CHAPTER: Training Deep Neural Networks

1. Identify what causes vanishing/exploding gradient problem during backpropagation and how weight initialization techniques can alleviate this problem.

Answer:

The Vanishing/Exploding Gradients Problems:

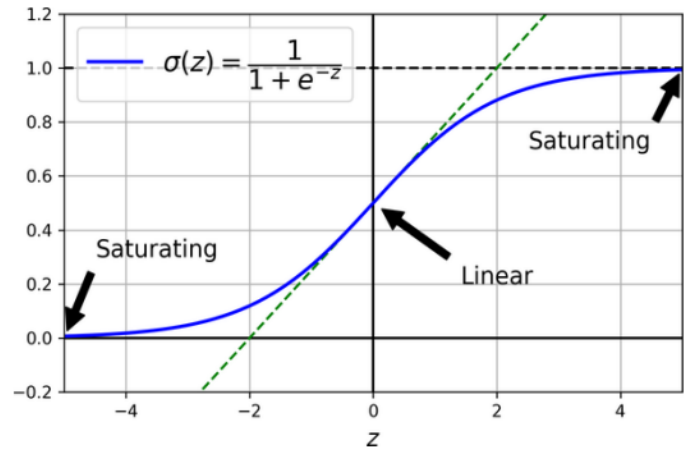
The backpropagation algorithm's second phase works by going from the output layer to the input layer, propagating the error gradient along the way. Once the algorithm has computed the gradient of the cost function with regard to each parameter in the network, it uses these gradients to update each parameter with a gradient descent step.

Unfortunately, gradients often get smaller and smaller as the algorithm progresses down to the lower layers. As a result, the gradient descent update leaves the lower layers' connection weights virtually unchanged, and training never converges to a good solution. This is called the vanishing gradients problem. In some cases, the opposite can happen: the gradients can grow bigger and bigger until layers get insanely large weight updates and the algorithm diverges. This is the exploding gradients problem, which surfaces most often in recurrent neural networks. More generally, deep neural networks suffer from unstable gradients; different layers may learn at widely different speeds.

Instability caused by inappropriate activation function and initialization technique:

It wasn't clear what caused the gradients to be so unstable when training a DNN. Later it was found that with sigmoid (logistic) activation function and weight initialization technique with a normal distribution with a mean of 0 and a standard deviation of 1, the variance of the outputs of each layer is much greater than the variance of its inputs. Going forward in the network, the variance keeps increasing after each layer until the activation function saturates at the top layers. This saturation is actually made worse by the fact that the sigmoid function has a mean of 0.5, not 0 (the hyperbolic tangent function has a mean of 0 and behaves slightly better than the sigmoid function in deep networks).

In sigmoid activation function (see figure), when inputs become large (negative or positive), the function saturates at 0 or 1, with a derivative extremely close to 0 (i.e., the curve is flat at both extremes). Thus, when backpropagation kicks in it has virtually no gradient to propagate back through the network, and what little gradient exists keeps getting diluted as backpropagation progresses down through the top layers, so there is really nothing left for the lower layers.



Glorot and He Initialization:

It was shown that the unstable gradients problem can be alleviated in a way that needs the signal to flow properly in both directions: in the forward direction when making predictions, and in the reverse direction when backpropagating gradients. The signal should not be died out, nor explode and saturate.

For the signal to flow properly, it was argued that the variance of the outputs of each layer is needed to be equal to the variance of its inputs, and the gradients to have equal variance before and after flowing through a layer in the reverse direction. It is actually not possible to guarantee both unless the layer has an equal number of inputs and outputs (these numbers are called the fan-in and fan-out of the layer), but a good compromise that has proven to work very well in practice: the connection weights of each layer must be initialized randomly as described in equation below, where $fan_{avg} = (fan_{in} + fan_{out}) / 2$. This initialization strategy is called *Xavier initialization* or *Glorot initialization*.

$$\text{Normal distribution with mean 0 and variance } \sigma^2 = \frac{1}{fan_{avg}}$$

$$\text{Or a uniform distribution between } -r \text{ and } +r, \text{ with } r = \sqrt{\frac{3}{fan_{avg}}}$$

LeCun Initialization:

If you replace fan_{avg} with fan_{in} in the above equation, we get an initialization strategy that Yann LeCun proposed. He called it LeCun initialization. LeCun initialization is equivalent to Glorot initialization when $fan_{in} = fan_{out}$.

Using Glorot initialization can speed up training considerably, and it is one of the practices that led to the success of deep learning. The initialization strategy proposed for the ReLU activation function and its variants is called *He initialization*. For SELU, Yann LeCun's initialization method should be used, preferably with a normal distribution.

2. Identify the problem that *ReLU (Rectified Linear Unit)* activation function suffers from, and how *Leaky ReLU*, *Exponential LU (ELU)* and *Scaled ELU (SELU)* can solve that problem. Provide required diagrams, expressions and code-snippet.

Answer:

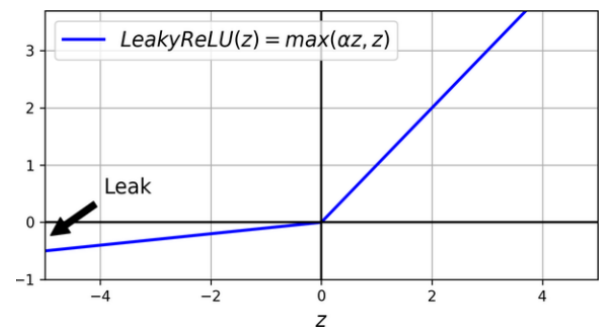
The problems with unstable gradients were in part due to a poor choice of activation function. It turns out that other activation functions behave much better in deep neural networks—in particular, the ReLU activation function, mostly because it does not saturate for positive values, and also because it is very fast to compute.

ReLU Activation Function & It's Concerns

Unfortunately, the ReLU activation function is not perfect. It suffers from a problem known as the dying ReLUs: during training, some neurons effectively “die”, meaning they stop outputting anything other than 0. In some cases, half of network’s neurons could be found dead, especially while using a large learning rate. A neuron dies when its weights get tweaked in such a way that the input of the ReLU function (i.e., the weighted sum of the neuron’s inputs plus its bias term) is negative for all instances in the training set. When this happens, it just keeps outputting zeros, and gradient descent does not affect it anymore because the gradient of the ReLU function is zero when its input is negative. To solve this problem, a variant of the ReLU function could be used, such as the leaky ReLU.

Leaky ReLU

The leaky ReLU activation function is defined as $\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$ (see figure). The hyperparameter α defines how much the function “leaks”: it is the slope of the function for $z < 0$. Having a slope for $z < 0$ ensures that leaky ReLUs never die; Leaky variants always outperformed the strict ReLU activation function.



Randomized Leaky ReLU (RRReLU)

In RReLU, α is picked randomly in a given range during training and is fixed to an average value during testing. RReLU also performed fairly well and seemed to act as a regularizer, reducing the risk of overfitting the training set.

Parametric Leaky ReLU (PReLU)

In PReLU, α is authorized to be learned during training: instead of being a hyperparameter, it becomes a parameter that can be modified by backpropagation like any other parameter. PReLU was reported

to strongly outperform ReLU on large image datasets, but on smaller datasets it runs the risk of overfitting the training set.

Keras includes the classes LeakyReLU and PReLU in the `tf.keras.layers` package.

```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2) # defaults to alpha=0.3
dense = tf.keras.layers.Dense(50, activation=leaky_relu, kernel_initializer="he_normal")
```

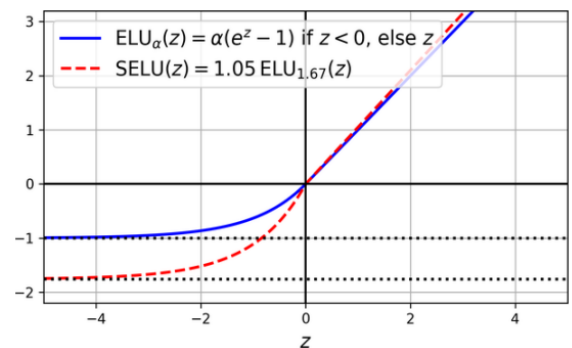
ELU and SELU

Exponential linear unit (ELU) outperformed all the ReLU variants in the authors' experiments: training time was reduced, and the neural network performed better on the test set. Equation below shows this activation function's definition.

$$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

The ELU activation function looks a lot like the ReLU function (see figure), with a few major differences:

- It takes on negative values when $z < 0$, which allows the unit to have an average output closer to 0 and helps alleviate the vanishing gradients problem. The hyperparameter α defines the opposite of the value that the ELU function approaches when z is a large negative number. It is usually set to 1, but it can be tweaked like any other hyperparameter.
- It has a nonzero gradient for $z < 0$, which avoids the dead neurons problem.
- If α is equal to 1 then the function is smooth everywhere, including around $z = 0$, which helps speed up gradient descent since it does not bounce as much to the left and right of $z = 0$.



Using ELU with Keras is as easy as setting `activation="elu"`, and like with other ReLU variants, He initialization should be used. The main drawback of the ELU activation function is that it is slower to compute than the ReLU function and its variants (due to the use of the exponential function). Its faster convergence rate during training may compensate for that slow computation, but still, at test time an ELU network will be a bit slower than a ReLU network.

Scaled ELU (SELU) activation function, as its name suggests, is a scaled variant of the ELU activation function (about 1.05 times ELU, using $\alpha \approx 1.67$). The authors showed that if you build a neural network composed exclusively of a stack of dense layers (i.e., an MLP), and if all hidden layers use the SELU

activation function, then the network will self-normalize: the output of each layer will tend to preserve a mean of 0 and a standard deviation of 1 during training, which solves the vanishing/exploding gradients problem. As a result, the SELU activation function may outperform other activation functions for MLPs, especially deep ones. To use it with Keras, just set `activation="selu"`. There are, however, a few conditions for self-normalization to happen.

- The input features must be standardized: mean 0 and standard deviation 1.
 - Every hidden layer's weights must be initialized using LeCun normal initialization.
 - The self-normalizing property is only guaranteed with plain MLPs.
 - Regularization techniques can not be used.
3. Identify the problem that activation function *Rectified Linear Unit (ReLU)* suffers from, and how *Gaussian ELU (GELU)* and *Swish* can solve that problem. Also mention how GELU can be approximated for faster computation and how Swish can approximate GELU (at a particular value for its hyperparameter). Provide required diagrams and expressions.

Answer:

The problems with unstable gradients were in part due to a poor choice of activation function. It turns out that other activation functions behave much better in deep neural networks—in particular, the ReLU activation function, mostly because it does not saturate for positive values, and also because it is very fast to compute.

ReLU Activation Function & It's Concerns

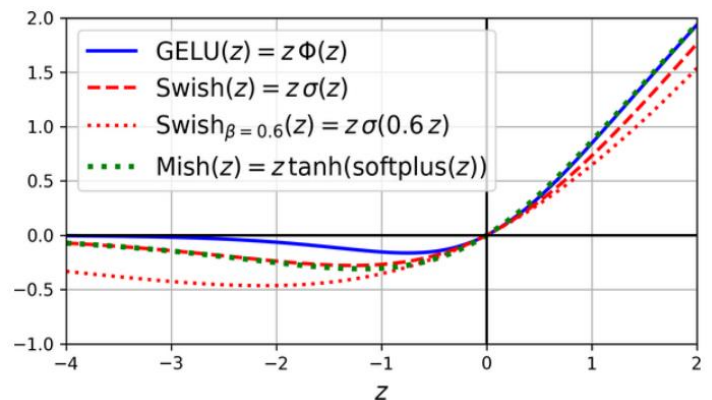
Unfortunately, the ReLU activation function is not perfect. It suffers from a problem known as the dying ReLUs: during training, some neurons effectively “die”, meaning they stop outputting anything other than 0. In some cases, half of network's neurons could be found dead, especially while using a large learning rate. A neuron dies when its weights get tweaked in such a way that the input of the ReLU function (i.e., the weighted sum of the neuron's inputs plus its bias term) is negative for all instances in the training set. When this happens, it just keeps outputting zeros, and gradient descent does not affect it anymore because the gradient of the ReLU function is zero when its input is negative. To solve this problem, a variant of the ReLU function could be used, such as the leaky ReLU.

GELU Activation Function

It is a smooth variant of the ReLU activation function. Its definition is given in equation below where Φ is the standard Gaussian cumulative distribution function (CDF): $\Phi(z)$ corresponds to the probability that a value sampled randomly from a normal distribution of mean 0 and variance 1 is lower than z .

$$\text{GELU}(z) = z \Phi(z)$$

As can be seen in figure, GELU resembles ReLU: it approaches 0 when its input z is very negative, and it approaches z when z is very positive. However, the GELU activation function is neither convex nor monotonic: from left to right, it starts by going straight, then it wiggles down, reaches a low point around -0.17 (near $z \approx -0.75$), and finally bounces up and ends up going straight toward the top right.



This fairly complex shape and the fact that it has a curvature at every point may explain why it works so well, especially for complex tasks: gradient descent may find it easier to fit complex patterns. However, it is a bit more computationally intensive, and the performance boost it provides is not always sufficient to justify the extra cost. That said, it is possible to show that it is approximately equal to $z\sigma(1.702 z)$, where σ is the sigmoid function: using this approximation also works very well, and it has the advantage of being much faster to compute.

Swish Activation Function

Swish outperformed every other function, including GELU. Swish is generalized by adding an extra hyperparameter β to scale the sigmoid function's input. The generalized Swish function is $\text{Swish}_\beta(z) = z\sigma(\beta z)$, so GELU is approximately equal to the generalized Swish function using $\beta = 1.702$. β can be tuned like any other hyperparameter. Alternatively, it's also possible to make β trainable and let gradient descent optimize it: much like PReLU, this can make the model more powerful, but it also runs the risk of overfitting the data.

4. Identify the working principle of *Batch Normalization* (BN) to alleviate the *vanishing/exploding gradient problem*. Also apply fusing BN layer with its previous layer after training can be used to avoid its runtime penalty.

Answer:

Batch Normalization (BN) to alleviate the vanishing/exploding gradient problem:

Although using He initialization along with ReLU (or any of its variants) can significantly reduce the danger of the vanishing/exploding gradients problems at the beginning of training, it doesn't guarantee that they won't come back during training. A technique called *batch normalization* (BN) can address these problems. The technique consists of adding an operation in the model just before (or after) the activation function of each hidden layer. This operation simply zero-centers and normalizes each input, then scales and shifts the result using two new parameter vectors per layer: one for scaling, the other for shifting. In other words, the operation lets the model learn the optimal scale and mean of each of the layer's inputs. In many cases, if you add a BN layer as the very first layer of your neural network, you do not need to standardize your training set. That is, there's no need for StandardScaler or Normalization; the BN layer will do it for you (well, approximately, since it only looks at one batch at a time, and it can also rescale and shift each input feature).

In order to zero-center and normalize the inputs, the algorithm needs to estimate each input's mean and standard deviation. It does so by evaluating the mean and standard deviation of the input over the current mini-batch (hence the name "batch normalization"). The whole operation is summarized step by step in equation below.

$$\begin{aligned}
 1. \quad \boldsymbol{\mu}_B &= \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)} \\
 2. \quad \boldsymbol{\sigma}_B^2 &= \frac{1}{m_B} \sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_B \right)^2 \\
 3. \quad \hat{\mathbf{x}}^{(i)} &= \frac{\mathbf{x}^{(i)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \varepsilon}} \\
 4. \quad \mathbf{z}^{(i)} &= \boldsymbol{\gamma} \otimes \hat{\mathbf{x}}^{(i)} + \boldsymbol{\beta}
 \end{aligned}$$

gamma and beta are hyperparameters vectors per BN layer for scaling and shifting, respectively

In this algorithm:

- $\boldsymbol{\mu}_B$ is the vector of input means, evaluated over the whole mini-batch B (it contains one mean per input).
- m_B is the number of instances in the mini-batch.
- $\boldsymbol{\sigma}_B$ is the vector of input standard deviations, also evaluated over the whole mini-batch (it contains one standard deviation per input).
- $\hat{\mathbf{x}}^{(i)}$ is the vector of zero-centered and normalized inputs for instance i .
- ε is a tiny number that avoids division by zero and ensures the gradients don't grow too large (typically 10^{-5}). This is called a *smoothing term*.
- $\boldsymbol{\gamma}$ is the output scale parameter vector for the layer (it contains one scale parameter per input).

- $\otimes$ represents element-wise multiplication (each input is multiplied by its corresponding output scale parameter).
- β is the output shift (offset) parameter vector for the layer (it contains one offset parameter per input). Each input is offset by its corresponding shift parameter.
- $Z^{(i)}$ is the output of the BN operation. It is a rescaled and shifted version of the inputs.

Most implementations of batch normalization estimate these final statistics (final input means and standard deviation vector per BN layer) during training by using a moving average of the layer's input means and standard deviations. This is what Keras does automatically when BatchNormalization layer is used. To sum up, four parameter vectors are learned in each batch-normalized layer: γ (the output scale vector) and β (the output offset vector) are learned through regular backpropagation, and μ (the final input mean vector) and σ (the final input standard deviation vector) are estimated using an exponential moving average. Note that μ and σ are estimated during training, but they are used only after training.

Batch normalization considerably improved all the deep neural networks. The vanishing gradients problem was strongly reduced, to the point that they could use saturating activation functions such as the tanh and even the sigmoid activation function. The networks were also much less sensitive to the weight initialization. Much larger learning rates could also be used, significantly speeding up the learning process. Batch normalization acts like a regularizer, reducing the need for other regularization techniques (such as dropout).

Fusing BN Layers:

There is a runtime penalty of having BN layers: the neural network makes slower predictions due to the extra computations required at each layer. Fortunately, it's often possible to fuse the BN layer with the previous layer after training, thereby avoiding the runtime penalty. This is done by updating the previous layer's weights and biases so that it directly produces outputs of the appropriate scale and offset. For example, if the previous layer computes $\mathbf{XW} + \mathbf{b}$, then the BN layer will compute $\gamma \otimes (\mathbf{XW} + \mathbf{b} - \mu) / \sigma + \beta$ (ignoring the smoothing term ϵ in the denominator). If we define $\mathbf{W}' = \gamma \otimes \mathbf{W} / \sigma$ and $\mathbf{b}' = \gamma \otimes (\mathbf{b} - \mu) / \sigma + \beta$, the equation simplifies to $\mathbf{XW}' + \mathbf{b}'$. So, if the previous layer's weights and biases ($\mathbf{W}$ and $\mathbf{b}$) are with the updated weights and biases ($\mathbf{W}'$ and $\mathbf{b}'$), the BN layer can be get rid of.

Implementing Batch Normalization with Keras:

This model applies BN after every hidden layer and as the first layer in the model (after flattening the input images).

```

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
])

```

5. Identify when to consider reusing pretrained layers. Mention all major considerations with examples and provide a figure visualizing the reuse of layers.

Answer:

Transfer Learning for Faster Training and Requirement for Less Data: It is generally not a good idea to train a very large DNN from scratch without first trying to find an existing neural network that accomplishes a similar task to be tackled. If such a neural network can be found, then generally most of its layers can be reused, except for the top ones. This technique is called *transfer learning*. It will not only speed up training considerably but also requires significantly less training data.

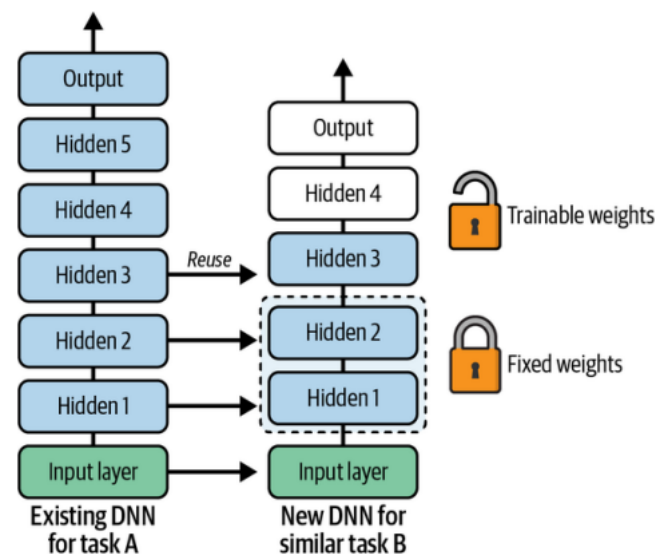
Example: Suppose there is a DNN that was trained to classify pictures into 100 different categories, including animals, plants, vehicles, and everyday objects, and the requirement is to train a DNN to classify specific types of vehicles. These tasks are very similar, even partly overlapping, so reusing parts of the first network can be tried out (see figure).

Input Size Requirement: If the input pictures for the new task don't have the same size as the ones used in the original task, usually a preprocessing step is required to be added to resize them to the size expected by the original model. More

generally, transfer learning will work best when the inputs have similar low-level features.

Consideration for Reusing Output Layer: The output layer of the original model should usually be replaced because it is most likely not useful at all for the new task, and probably will not have the right number of outputs.

Consideration for Reusing Upper Hidden Layers: Similarly, the upper hidden layers of the original model are less likely to be as useful as the lower layers, since the high-level features that are most



useful for the new task may differ significantly from the ones that were most useful for the original task. So, the right number of layers to reuse needs to be found out.

Training Aspects:

- All the reused layers to be freezed first (i.e., making their weights non-trainable so that gradient descent won't modify them, and they will remain fixed).
 - Then the model is to be trained and checked how it performs.
 - Then one or two of the top hidden layers can be unfreezed to let backpropagation tweak them and again its performance to be checked for improvement. The more training data available, the more layers can be unfreezed. It is also useful to reduce the learning rate while unfreezing the reused layers to avoid wrecking their fine-tuned weights.
 - If good performance is still not achieved and having access to little training data, then the top hidden layer(s) can be dropped and all the remaining hidden layers can be freezed again. This step can be iterated until the right number of layers to reuse is found.
 - But having access to plenty of training data, the top hidden layers can be replaced instead of dropping them. Additional hidden layers can also be added.
6. Explain *Momentum* and *Nesterov Accelerated Gradient (NAG)* optimizer and justify the reason the latter one being the fastest than the former one. Also provide comparison diagram and algorithm for Momentum.

Answer:

Momentum:

The regular gradient descent will take small steps when the slope is gentle and big steps when the slope is steep, but it will never pick up speed. As a result, regular gradient descent is generally much slower to reach the minimum than momentum optimization. It updates the weights θ by directly subtracting the gradient of the cost function $J(\theta)$ with regard to the weights ($\nabla_{\theta} J(\theta)$) multiplied by the learning rate η . The equation is $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$. It does not care about what the earlier gradients were. If the local gradient is tiny, it goes very slowly.

Momentum optimization cares a great deal about what previous gradients were: at each iteration, it subtracts the local gradient from the momentum vector $\mathbf{m}$ (multiplied by the learning rate η), and it updates the weights by adding this momentum vector (see equation below). In other words, the

gradient is used as an acceleration, not as a speed. To simulate some sort of friction mechanism and prevent the momentum from growing too large, the algorithm introduces a new hyperparameter β , called the momentum, which must be set between 0 (high friction) and 1 (no friction). A typical momentum value is 0.9.

Momentum algorithm

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta + \mathbf{m}$

If the gradient remains constant, the terminal velocity (i.e., the maximum size of the weight updates) is equal to that gradient multiplied by the learning rate η multiplied by $1 / (1 - \beta)$ (ignoring the sign). For example, if $\beta = 0.9$, then the terminal velocity is equal to 10 times the gradient times the learning rate, so momentum optimization ends up going 10 times faster than gradient descent. This allows momentum optimization to escape from plateaus much faster than gradient descent.

Gradient descent goes down the steep slope quite fast, but then it takes a very long time to go down the valley. In contrast, momentum optimization will roll down the valley faster and faster until it reaches the bottom (the optimum). In deep neural networks that don't use batch normalization, the upper layers will often end up having inputs with very different scales, so using momentum optimization helps a lot. It can also help roll past local optima.

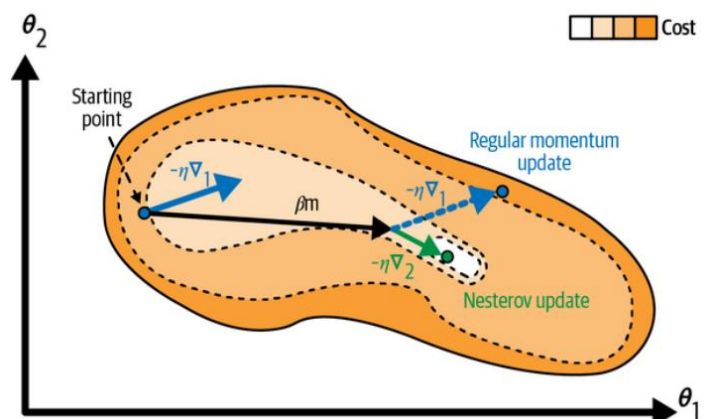
Nesterov Accelerated Gradient (a.k.a. Nesterov momentum optimization):

Nesterov accelerated gradient (NAG) - a small variant to momentum optimization is almost always faster than regular momentum optimization. This method also known as *Nesterov momentum optimization*, measures the gradient of the cost function not at the local position θ but slightly ahead in the direction of the momentum, at $\theta + \beta \mathbf{m}$ (see equation below).

Nesterov accelerated gradient algorithm

1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$
2. $\theta \leftarrow \theta + \mathbf{m}$

This small tweak works because in general the momentum vector will be pointing in the right direction (i.e., toward the optimum), so it will be slightly more accurate to use the gradient measured a bit farther in that direction rather than the gradient at the original position (see in figure where ∇_1 represents the gradient of the cost function measured at the starting point θ , and ∇_2 represents the gradient at the point



located at $\theta + \beta m$). As can be seen, the Nesterov update ends up closer to the optimum. After a while, these small improvements add up and NAG ends up being significantly faster than regular momentum optimization.

7. Develop the intuition of how *AdaGrad* optimizer works and contrast it with *RMSPprop* mentioning the improvement of the latter one over the former one. Provide algorithms for both optimizers.

Answer:

AdaGrad

Gradient descent starts by quickly going down the steepest slope, which does not point straight toward the global optimum, then it very slowly goes down to the bottom of the valley. It would be nice if the algorithm could correct its direction earlier to point a bit more toward the global optimum. The AdaGrad algorithm achieves this correction by scaling down the gradient vector along the steepest dimensions (see equation).

$$\begin{aligned} 1. \quad & \mathbf{s} \leftarrow \mathbf{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta) \\ 2. \quad & \theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \epsilon} \end{aligned}$$

Step 1: The first step accumulates the square (element-wise multiplication) of the gradients into the vector $\mathbf{s}$. If the cost function is steep along the i dimension, then $\mathbf{s}$ will get larger and larger at each iteration.

Step 2: The second step is almost identical to gradient descent, but the gradient vector is scaled down (element-wise division) by a factor of $\sqrt{\mathbf{s} + \epsilon}$, (ϵ is a smoothing term to avoid division by zero, typically set to 10^{-10})

Adaptive Learning Rate: It decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes. This is called an adaptive learning rate. It helps point the resulting updates more directly toward the global optimum.

Limitation of AdaGrad: AdaGrad frequently performs well for simple quadratic problems, but it often stops too early when training neural networks: the learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum. So, this optimizer should not be used to train deep neural networks though it may be efficient for simpler tasks such as linear regression.

RMSPprop

Exponential Decay: The RMSProp algorithm fixes the above-mentioned limitation of AdaGrad by accumulating only the gradients from the most recent iterations, as opposed to all the gradients since the beginning of training. It does so by using exponential decay in the first step (see equation).

$$1. \quad \mathbf{s} \leftarrow \rho \mathbf{s} + (1 - \rho) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$$

$$2. \quad \theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \epsilon}$$

Decay Rate (hyperparameter): The decay rate ρ (rho) is typically set to 0.9 - the default value that often works well.

8. Explain the working principles of Adam optimizer and identify the modifications made into AdamW optimizer to address the overfitting issue of the former.

Answer:

Adam

Similarity with Momentum and RMSProp Optimizer: Adam, which stands for adaptive moment estimation, combines the ideas of momentum optimization and RMSProp. Just like momentum optimization, it keeps track of an exponentially decaying average of past gradients, and just like RMSProp, it keeps track of an exponentially decaying average of past squared gradients (see equation). These are estimations of the mean and (uncentered) variance of the gradients.

$$\begin{aligned}
 1. \quad \mathbf{m} &\leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_{\theta} J(\theta) && \text{\# tracks an exponentially decaying average of past gradients} \\
 2. \quad \mathbf{s} &\leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta) && \text{\# tracks an exponentially decaying average of past squared gradients} \\
 3. \quad \widehat{\mathbf{m}} &\leftarrow \frac{\mathbf{m}}{1 - \beta_1^t} \\
 4. \quad \widehat{\mathbf{s}} &\leftarrow \frac{\mathbf{s}}{1 - \beta_2^t} \\
 5. \quad \theta &\leftarrow \theta + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \epsilon}
 \end{aligned}$$

In this equation, t represents the iteration number (starting at 1). It can be noticed from step 1, 2 and 5 that Adam's close similarity to both momentum optimization and RMSProp: β_1 corresponds to β in momentum optimization, and β_2 corresponds to ρ (rho) in RMSProp. The only difference is that step 1 computes an exponentially decaying average rather than an exponentially decaying sum. Steps 3 and 4 help to boost $\mathbf{m}$ and $\mathbf{s}$ at the beginning of the training as they will be biased toward 0 due to their initialization at 0.

Hyperparameters: The momentum decay hyperparameter β_1 is typically initialized to 0.9, while the scaling decay hyperparameter β_2 is often initialized to 0.999. The smoothing term ϵ is usually initialized to a tiny number such as 10^{-7} .

AdamW

AdamW is a variant of Adam that integrates a regularization technique called weight decay. Weight decay reduces the size of the model's weights at each training iteration by multiplying them by a decay factor (hyperparameter) such as 0.99.

9. Apply reasons to prefer *learning rate scheduling* over the manual process of finding an appropriate learning rate. Also explain a few of the popular learning rate scheduling strategies.

Answer:

Constant Learning Rate: A good learning rate can be found by training the model for a few hundred iterations, exponentially increasing the learning rate from a very small value to a very large value, and then looking at the learning curve and picking a learning rate slightly lower than the one at which the learning curve starts shooting back up. The model can then be initialized and trained with that learning rate.



Dynamic Learning Rate: Instead of a constant learning rate, training can be started with a large learning rate and then can be reduced once training stops making fast progress, resulting in reaching a good solution faster compared to the optimal constant learning rate. There are many different strategies to reduce the learning rate during training. These strategies are called *learning schedules*.

Power Scheduling

Set the learning rate to a function of the iteration number t : $\eta(t) = \eta_0 / (1 + t/s)^c$. The initial learning rate η , the power c (typically set to 1), and the steps s are hyperparameters. The learning rate drops at each step. After s steps, the learning rate is down to $\eta_0 / 2$. After s more steps it is down to $\eta_0 / 3$, then it goes down to $\eta_0 / 4$, then $\eta_0 / 5$, and so on. As can be seen, this schedule first drops quickly, then more and more slowly. Of course, power scheduling requires tuning η_0 and s (and possibly c).

Exponential Scheduling

Set the learning rate to $\eta(t) = \eta_0 0.1^{t/s}$. The learning rate will gradually drop by a factor of 10 every s steps. While power scheduling reduces the learning rate more and more slowly, exponential scheduling keeps slashing it by a factor of 10 every s steps.

Piecewise Constant Scheduling

Use a constant learning rate for several epochs (e.g., $\eta = 0.1$ for 5 epochs), then a smaller learning rate for another number of epochs (e.g., $\eta = 0.001$ for 50 epochs), and so on. Although this solution can work very well, it requires fiddling around to figure out the right sequence of learning rates and how long to use each of them.

Performance Scheduling

Measure the validation error every N steps (just like for early stopping) and reduce the learning rate by a factor of λ when the error stops dropping.

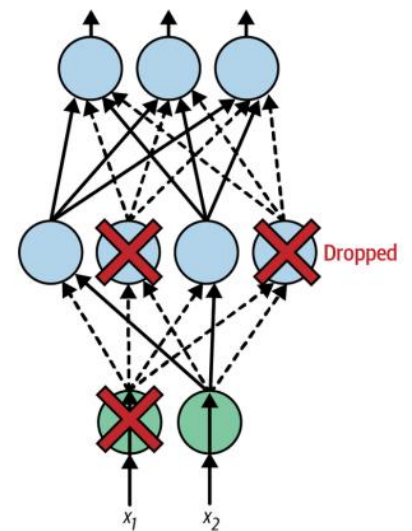
1cycle Scheduling

Contrary to the other approaches, it starts by increasing the initial learning rate η_0 , growing linearly up to η_1 halfway through training. Then it decreases the learning rate linearly down to η_0 again during the second half of training, finishing the last few epochs by dropping the rate down by several orders of magnitude (still linearly). When using a momentum, the training gets started with a high momentum first (e.g., 0.95), then dropped down to a lower momentum during the first half of training (e.g., down to 0.85, linearly), and then brought it back up to the maximum value (e.g., 0.95) during the second half of training, finishing the last few epochs with that maximum value. Over many experiments, it was shown that this approach was often able to speed up training considerably and reach better performance.

10. Explain the working principles of *dropout regularization* technique with a neat figure showing a snapshot of training step indicating drop out neurons. Also, develop code-snippet showing its application.

Answer:

Dropout is one of the most popular regularization techniques for deep neural networks. It is a fairly simple algorithm: at every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily “dropped out”, meaning it will be entirely ignored during this training step, but it may be active during the next step (see figure). The hyperparameter p is called the *dropout rate*, and it is typically set between 10% and 50%: closer to 20%–30% in recurrent neural nets, and closer to 40%–50% in convolutional neural networks. After training, neurons don’t get dropped anymore.



Neurons trained with dropout cannot co-adapt with their neighboring neurons; they have to be as useful as possible on their own. They also cannot rely excessively on just a few input neurons; they must pay attention to each of their input neurons. They end up being less sensitive to slight changes in the inputs. In the end, it results in a more robust network that generalizes better.

Another way to understand the power of dropout is to realize that a unique neural network is generated at each training step. Since each neuron can be either present or absent, there are a total of 2^N possible networks (where 2^N is the total number of droppable neurons). This is such a huge number that it is virtually impossible for the same neural network to be sampled twice. Once you have run 10,000 training steps, you have essentially trained 10,000 different neural networks, each with just one training instance. These neural networks are obviously not independent because they share many of their weights, but they are nevertheless all different. The resulting neural network can be seen as an averaging ensemble of all these smaller neural networks.

Keep Probability:

There is one small but important technical detail. Suppose $p = 75\%$: on average only 25% of all neurons are active at each step during training. This means that after training, a neuron would be connected to four times as many input neurons as it would be during training. To compensate for this fact, each neuron’s input connection weights should be multiplied by four during training. If we don’t, the neural network will not perform well as it will see different data during and after training. More generally, we need to divide the connection weights by the *keep probability* ($1 - p$) during training.

Implementation:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he normal"),
```

```
tf.keras.layers.Dropout(rate=0.2),
tf.keras.layers.Dense(100, activation="relu", kernel_initializer="he_normal"),
tf.keras.layers.Dropout(rate=0.2),
tf.keras.layers.Dense(10, activation="softmax")
])

[...] # compile and train the model
```

If the model is observed getting overfitted, the dropout rate should be increased and conversely, it should be decreased if the model underfits the training set. It can also help to increase the dropout rate for large layers, and reduce it for small ones. Moreover, many state-of-the-art architectures only use dropout after the last hidden layer. Dropout does tend to significantly slow down convergence, but it often results in a better model when tuned properly. So, it is generally well worth the extra time and effort, especially for large models.

11. Build intuition on how *Monte Carlo* dropout can boost the performance of any trained dropout model without retraining and can handle prediction uncertainty. Also explain how *max-norm regularization* works.

Answer:

Monte Carlo (MC) Dropout:

Monte Carlo (MC) dropout can boost the performance of any trained dropout model without having to retrain it or even modify it at all. It also provides a much better measure of the model's uncertainty, and it can be implemented in just a few lines of code.

It is the full implementation of MC dropout, boosting the already trained dropout model without retraining it.

```
import numpy as np

y_probas = np.stack([model(X_test, training=True) for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```

Note that `model(X)` is similar to `model.predict(X)` except it returns a tensor rather than a NumPy array, and it supports the `training` argument. In this code example, setting `training=True` ensures that the Dropout layer remains active, so all predictions will be a bit different. Just 100 predictions were made over the test set, and their average was computed. More specifically, each call to the model returns a matrix with one row per instance and one column per class. Because there are 10,000 instances in the test set and 10 classes, this is a matrix of shape [10000, 10]. 100 such matrices were stacked, so `y_probas` is a 3D array of shape [100, 10000, 10]. Once we average over the first dimension (`axis=0`), we get `y_proba`, an array of shape [10000, 10], like we would get with a

single prediction. That's all! Averaging over multiple predictions with dropout turned on gives us a Monte Carlo estimate that is generally more reliable than the result of a single prediction with dropout turned off. MC dropout is a great technique that boosts dropout models and provides better uncertainty estimates. And of course, since it is just regular dropout during training, it also acts like a regularizer.

Max-Norm Regularization:

Another popular regularization technique for neural networks is called *max-norm regularization*: for each neuron, it constrains the weights $\mathbf{w}$ of the incoming connections such that $\|\mathbf{w}\| \leq r$, where r is the max-norm hyperparameter and $\|\cdot\|_2$ is the ℓ_2 norm.

Max-norm regularization does not add a regularization loss term to the overall loss function. Instead, it is typically implemented by computing $\|\mathbf{w}\|_2$ after each training step and rescaling $\mathbf{w}$ if needed ($\mathbf{w} \leftarrow \mathbf{w} r / \|\mathbf{w}\|_2$). Reducing r increases the amount of regularization and helps reduce overfitting. Max-norm regularization can also help alleviate the unstable gradients problems when not using batch normalization.

After each training iteration, the model's `fit()` method will call the object returned by `max_norm()`, passing it the layer's weights and getting rescaled weights in return, which then replace the layer's weights.

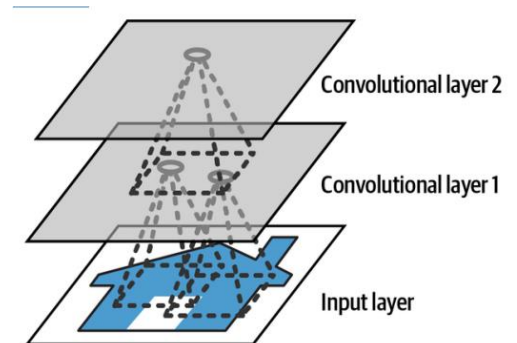
CHAPTER: Convolutional Neural Networks

12. Explain the general architecture of a convolutional layer with respect to image recognition tasks and identify the need for padding and stride. Provide appropriate diagrams.

Answer:

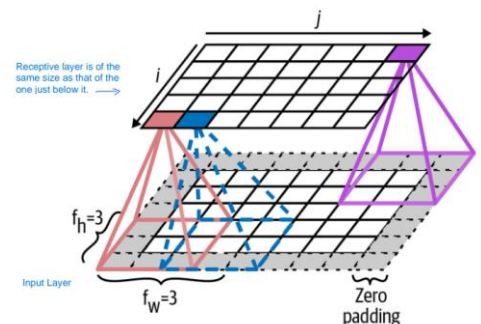
Convolutional Layer:

The most important building block of a convolutional neural network (CNN) is the convolutional layer: neurons in the first convolutional layer are not connected to every single pixel in the input image, but only to pixels in their receptive fields (see figure). In turn, each neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer. This architecture allows the network to concentrate on small low-level features in the first hidden layer, then assemble them into larger higher-level features in the next hidden layer, and so on. This hierarchical structure is common in real-world images, which is one of the reasons why CNNs work so well for image recognition.



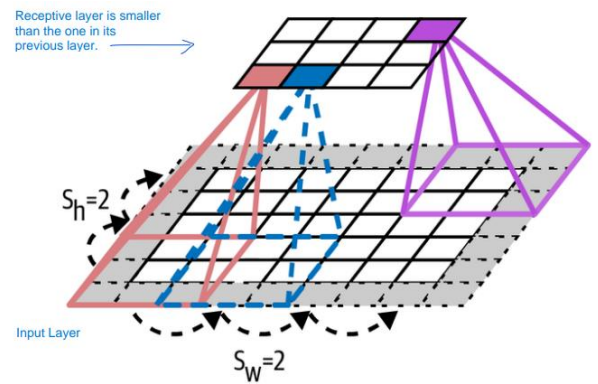
Padding:

A neuron located in row i , column j of a given layer is connected to the outputs of the neurons in the previous layer located in rows i to $i + f_h - 1$, columns j to $j + f_w - 1$, where f_h and f_w are the height and width of the receptive field (see figure). In order for a layer to have the same height and width as the previous layer, it is common to add zeros around the inputs, as shown in the diagram. This is called zero padding.



Stride:

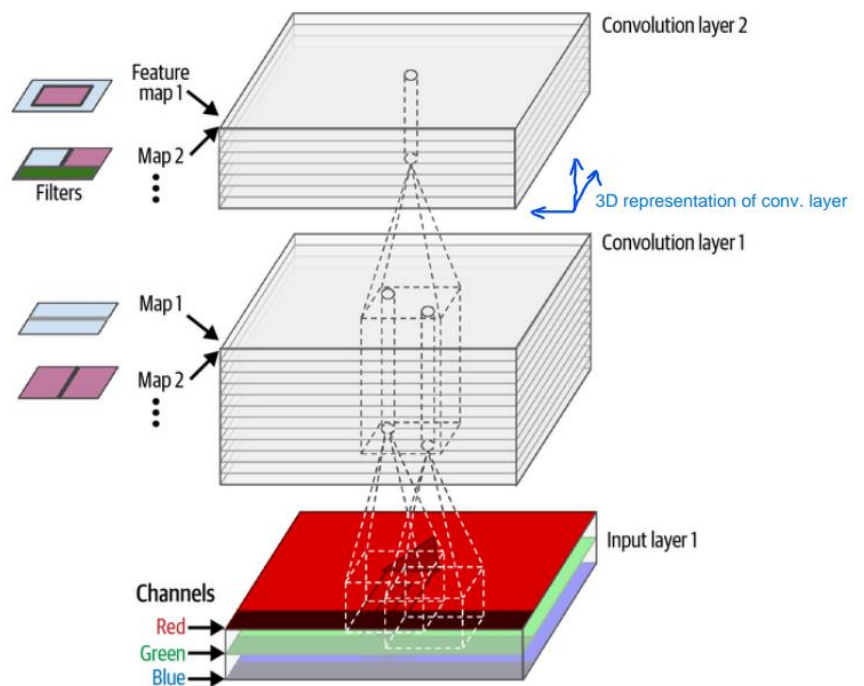
It is also possible to connect a large input layer to a much smaller layer by spacing out the receptive fields, as shown in figure. This dramatically reduces the model's computational complexity. The horizontal or vertical step size from one receptive field to the next is called the *stride*. In the diagram, a 5×7 input layer (plus zero padding) is connected to a 3×4 layer, using 3×3 receptive fields and a stride of 2 (in this example the stride is the same in both directions, but it does not have to be so). A neuron located in row i , column j in the upper layer is connected to the outputs of the neurons in the previous layer located in rows $i \times s_h$ to $i \times s_h + f_h - 1$, columns $j \times s_w$ to $j \times s_w + f_w - 1$, where s_h and s_w are the vertical and horizontal strides.



13. How do multiple feature maps work in a convolutional layer with respect to provide better representation of its inputs. Provide a diagram depicting multiple filters each processing a color image and outputting feature maps, and equation for computing the output of a neuron in a convolutional layer.

Answer:

A convolutional layer has multiple filters and outputs one feature map per filter, so it is more accurately represented in 3D (see figure). It has one neuron per pixel in each feature map, and all neurons within a given feature map share the same parameters (i.e., the same kernel and bias term). Neurons in different feature maps use different parameters. A neuron's receptive field is the same as described earlier, but it extends across all the feature maps of the previous layer. In short, a convolutional layer simultaneously applies multiple trainable filters to its inputs, making it capable of detecting multiple features anywhere in its inputs.



Multi-dimensional Input: Input images are also composed of multiple sublayers: one per color channel (typically in RGB). Grayscale images have just one channel, but some images may have many more—for example, satellite images that capture extra light frequencies (such as infrared).

Specifically, a neuron located in row i , column j of the feature map k in a given convolutional layer l is connected to the outputs of the neurons in the previous layer $l - 1$, located in rows $i \times s_h$ to $i \times s_h + f_h - 1$ and columns $j \times s_w$ to $j \times s_w + f_w - 1$, across all feature maps (in layer $l - 1$). Within a layer, all neurons located in the same row i and column j but in different feature maps are connected to the outputs of the exact same neurons in the previous layer.

Computing the output of a neuron in a convolutional layer:

The equation below summarizes the preceding explanations in one mathematical equation: it shows how to compute the output of a given neuron in a convolutional layer. It calculate the weighted sum of all the inputs, plus the bias term.

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_{n'}-1} x_{i',j',k'} \times w_{u,v,k',k} \quad \text{with} \quad \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

In this equation:

- $z_{i,j,k}$ is the output of the neuron located in row i , column j in feature map k of the convolutional layer (layer l).
- s_h and s_w are the vertical and horizontal strides, f_h and f_w are the height and width of the receptive field, and $f_{n'}$ is the number of feature maps in the previous layer (layer $l - 1$).
- $x_{i',j',k'}$ is the output of the neuron located in layer $l - 1$, row i' , column j' , feature map k' (or channel k' if the previous layer is the input layer).
- b_k is the bias term for feature map k (in layer l). This can be thought of as a knob that tweaks the overall brightness of the feature map k .
- $w_{u,v,k',k}$ is the connection weight between any neuron in feature map k of the layer l and its input located at row u , column v (relative to the neuron's receptive field), and feature map k' .

14. Solve the following by considering a *convolutional layer* with 200 filters with 5 x 5 kernel, stride of 1 and “same” padding. The input is 150 x 100 three channel images.

- What is the total number of parameters in the convolutional layer?
- What is the total number of floating-point multiplications to be computed in that layer for each input?
- Calculate the memory requirement [in GB] of that layer for its output for a batch of 100 input images if feature maps are represented using 32-bit floats?

Answer:

a) Total number parameters in a convolutional layer

$$\begin{aligned} &= (\text{filter width} * \text{filter height} * \text{input channels} + \text{bias}) * \text{feature maps} \\ &= (5 * 5 * 3 + 1) * 200 \\ &= \mathbf{15200} \end{aligned}$$

b) Floating-point multiplications in the layer for each input

$$\begin{aligned} &= (\text{filter width} * \text{filter height} * \text{input channels}) * (\text{input width} * \text{input height}) * \text{feature maps} \\ &= (5 * 5 * 3) * (150 * 100) * 200 \\ &= \mathbf{225000000} \text{ (225 million)} \end{aligned}$$

c) Memory requirement [in bits] for the layer for a batch of 100 input images

$$\begin{aligned} &= \text{spatial dimension of feature maps} * \text{bits represent each neuron} * \text{feature maps} * \text{batch size} \\ &= (150 * 100) * 32 * 200 * 100 \\ &= 9600000000 \text{ bits} \\ &\approx \mathbf{1.12 \text{ GB}} \text{ [dividing by 8 to get bytes, then 1024 to get KB, then 1024 to get MB and at last by 1024} \\ &\text{once again to get GB]} \end{aligned}$$

15. Solve the following by considering a *convolutional Neural Network* composed of two convolutional layers, each with 3×3 kernels, a stride of 2, and "same" padding. The first layer outputs 100 and the last one outputs 200 feature maps. The inputs are 200×300 pixels three channel color images.

- What is the total number of parameters in the CNN?
- Calculate the minimum memory requirement [in megabytes] of this network to make a prediction for a single instance if feature maps are represented using 32-bit floats?
- Calculate the minimum memory requirement [in megabytes] of this network when training on a mini batch of 50 images?

Answer:

a) Total number parameters in the CNN:

= Total number of parameters in the first convolutional layer

+

Total number parameters in the second convolutional layer

= (filter width * filter height * input channels + bias) * feature maps in first conv. layer

+

(filter width * filter height * first conv. layer feature maps + bias) * feature maps in last conv. layer

$$\begin{aligned}
&= [(3 * 3 * 3 + 1) * 100] + [(3 * 3 * 100 + 1) * 200] \\
&= 2800 + 180200 \\
&= \mathbf{183000} \text{ [Answer]}
\end{aligned}$$

b) Calculating total minimum memory requirement at a time to predict for a single instance

Considering stride of 2 and “same” padding, spatial dimension of feature maps in **first** conv. layer is

= input width / 2, input height / 2

= 100, 150 (by 200 / 2, 300 / 2), and

spatial dimension of feature maps in **last** conv. layer is

= input width / 2, input height / 2

= 50, 75 (by 100 / 2, 150 / 2)

Memory requirement [in bits] for the **first** conv. layer for an input image

= spatial dimension of feature maps * bits represent each neuron * feature maps

= 100 * 150 * 32 * 100

= 48000000 bits

= 5.72 MB [dividing by 8 to get bytes, then 1024 to get KB, then 1024 to get MB]

Memory requirement [in bits] for the **second** conv. layer for an input image

= spatial dimension of feature maps * bits represent each neuron * feature maps

= 50 * 75 * 32 * 200

= 24000000 bits

= 2.86 MB [dividing by 8 to get bytes, then 1024 to get KB, then 1024 to get MB]

As during inference, minimum required memory at a time would be the memory occupied only by the current conv. layer and its previous layer, let's calculate the combined requirement for first and second layer and then the combined requirement for the second and the third layer.

Combined memory requirement for first and second conv. layer

= 5.72 MB + 2.86 MB = 8.58 MB

Now, memory required to hold all parameters for all conv. layers

= Total number of parameters * bits representing each neuron

= 183000 [calculated earlier] * 32 bits

= 5856000 bits

= 0.7 MB [dividing by 8 to get bytes, then 1024 to get KB, then 1024 to get MB]

So, total minimum memory requirement at a time to predict for a single instance is

= Combined memory requirement for first and second conv. layer + memory required to hold all parameters for all conv. layers

= 8.58 MB + 0.7 MB

= **9.28 MB [Answer]**

c) Minimum memory required when training the CNN on a mini-batch of 50 images

= Memory required by input layer for a single instance * 50 +

Memory required to hold all parameters for all conv. layers +

Combined memory requirement for first and second conv. layer * 50

= $200 * 300 * 3 * 32 * 50$ bits [input width * input height * input channels * bits representing each neuron * mini-batch size] +

0.7 MB [calculated earlier] +

8.58 MB [calculated earlier] * 50

= 34.33 MB [dividing by 8 to get bytes, then 1024 to get KB, then 1024 to get MB] + 0.7 MB + 429 MB

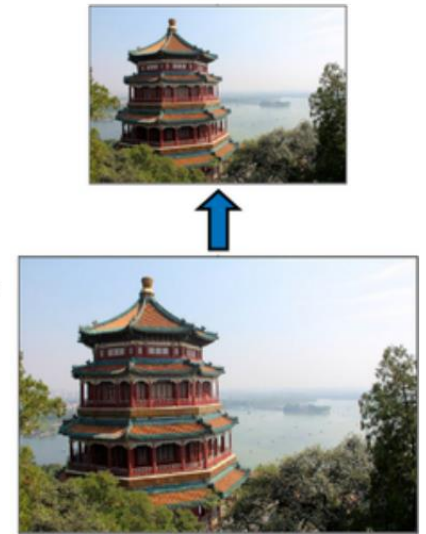
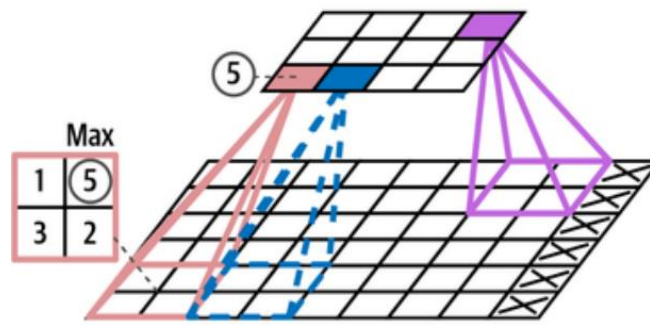
= **464.03 MB [Answer]**

16. Identify the need for pooling layers and explain its working over all of its popular variants (i.e. average, max and global average pool). Provide an appropriate figure depicting the main pooling concept.

Answer:

The Need for Pooling Layers

The goal is a pooling layer is to subsample (i.e., shrink) the input image (or feature maps) to reduce the computational load, the memory usage, and the number of parameters (thereby limiting the risk of



overfitting). Just like in convolutional layers, each neuron in a pooling layer is connected to the outputs of a limited number of neurons in the previous layer, located within a small rectangular receptive field. Its size, the stride, and the padding type must be defined. However, a pooling neuron has no weights; all it does is aggregate the inputs using an aggregation function such as the max or mean. Pooling layer also introduces some level of invariance to small translations, rotations and scale.

Max Pooling:

The figure shows a *max pooling layer*, which is the most common type of pooling layer. In this example, a 2×2 pooling kernel is used, with a stride of 2 and no padding. Only the max input value in each receptive field makes it to the next layer, while the other inputs are dropped. For example, in the lower-left receptive field in figure, the input values are 1, 5, 3, 2, so only the max value, 5, is propagated to the next layer. Because of the stride of 2, the output image has half the height and half the width of the input image (rounded down since no use of padding).

```
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Average Pooling:

To create an *average pooling layer*, `AveragePooling2D` (alias `AvgPool2D`) instead of `MaxPool2D` is used. It works exactly like a max pooling layer, except it computes the mean rather than the max. Average pooling layers used to be very popular, but people mostly use max pooling layers now, as they generally perform better. This may seem surprising, since computing the mean generally loses less information than computing the max. But on the other hand, max pooling preserves only the strongest features, getting rid of all the meaningless ones, so the next layers get a

cleaner signal to work with. Moreover, max pooling offers stronger translation invariance than average pooling, and it requires slightly less compute.

Global Average Pooling:

Another pooling layer that is often seen in modern architectures is the *global average pooling layer*. It works very differently: all it does is compute the mean of each entire feature map (it's like an average pooling layer using a pooling kernel with the same spatial dimensions as the inputs). This means that it just outputs a single number per feature map and per instance. Although this is of course extremely destructive (most of the information in the feature map is lost), it can be useful just before the output layer. To create such a layer, simply `GlobalAveragePooling2D` class is used (alias `GlobalAvgPool2D`):

```
global_avg_pool = tf.keras.layers.GlobalAvgPool2D()
```

Pooling along the Depth Dimension:

Max pooling and average pooling can be performed along the depth dimension instead of the spatial dimensions, although it's not as common. This can allow the CNN to learn to be invariant to various features. For example, it could learn multiple filters, each detecting a different rotation of the same pattern (such as handwritten digits), and the depth-wise max pooling layer would ensure that the output is the same regardless of the rotation. The CNN could similarly learn to be invariant to anything: thickness, brightness, skew, color, and so on.

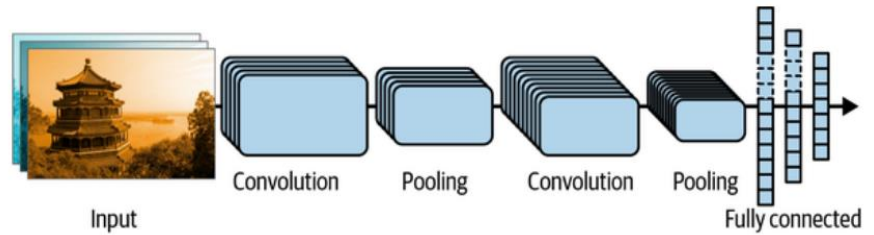
17. With appropriate code-snippets, explain the general construction of a convolutional neural network. Also, identify the general guidelines for using convolution kernels.

Answer:

General Construction of a CNN (the Architecture):

Typical CNN architectures stack a few convolutional layers (each one generally followed by a ReLU layer), then a pooling layer, then another few convolutional layers (+ReLU), then another pooling layer, and so on. The image gets smaller and smaller as it progresses through the network, but it also typically gets deeper and deeper (i.e., with more feature maps), thanks to the convolutional layers

(see figure). At the top of the stack, a regular feedforward neural network is added, composed of a few fully connected layers (+ReLUs), and the final layer outputs the prediction (e.g., a softmax layer that outputs estimated class probabilities).



Code-snippets (Standard CNN Implementation in Keras):

```
from functools import partial

DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, padding="same",
                        activation="relu", kernel_initializer="he_normal")

model = tf.keras.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    tf.keras.layers.MaxPool2D(), DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=128, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=64, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=10, activation="softmax")
])
```

Code Explanation:

- It creates a Sequential model. Its first layer is a DefaultConv2D with 64 fairly large filters (7 × 7). It uses the default stride of 1 because the input images are not very large. It also sets input_shape=[28, 28, 1], because the images are 28 × 28 pixels, with a single color channel (i.e., grayscale).
- Then it adds a max pooling layer that uses the default pool size of 2, so it divides each spatial dimension by a factor of 2.
- The structure is repeated twice: two convolutional layers followed by a max pooling layer. For larger images, we could repeat this structure several times.
- The number of filters doubles as it climbs up the CNN toward the output layer (it is initially 64, then 128, then 256): it makes sense for it to grow, since the number of low-level features is often fairly low (e.g., small circles, horizontal lines), but there are many different ways to combine them

into higher-level features. It is a common practice to double the number of filters after each pooling layer: since a pooling layer divides each spatial dimension by a factor of 2, the number of feature maps can be doubled in the next layer without fear of exploding the number of parameters, memory usage, or computational load.

- Next is the fully connected network, composed of two hidden dense layers and a dense output layer. Since it's a classification task with 10 classes, the output layer has 10 units, and it uses the softmax activation function. Input is flattened just before the first dense layer, since it expects a 1D array of features for each instance. Two dropout layers are also added, with a dropout rate of 50% each, to reduce overfitting.

Guidelines of Using Convolution Kernels:

A common mistake is to use convolution kernels that are too large. For example, instead of using a convolutional layer with a 5×5 kernel, two layers with 3×3 kernels can be stacked: it will use fewer parameters and require fewer computations, and it will usually perform better. One exception is for the first convolutional layer: it can typically have a large kernel (e.g., 5×5), usually with a stride of 2 or more. This will reduce the spatial dimension of the image without losing too much information, and since the input image only has three channels in general, it will not be too costly.

18. Solve by calculating the spatial dimension of the feature maps for the pooling and convolutional layers given stride of 1 and padding as “valid” in the below mentioned LeNet-5 architecture. Also, calculate the number of (trainable) parameters in the respective layer.

Layer	Type	Maps	Size	Kernel Size	Stride	Parameters
Out	Fully connected	—	10	—	—	?
F6	Fully connected	—	84	—	—	?
C5	Convolutional	120	?	5×5	1	?
S4	Avg. pool	16	?	2×2	2	—
C3	Convolutional	16	?	5×5	1	?
S2	Avg. pool	6	?	2×2	2	—
C1	Convolutional	6	?	5×5	1	?
In	Input	1	32×32	—	—	—

Answer:

Formula:

Standard expression to find feature map width for feature maps in convolutional layer with padding “valid”

$$= 1 + (\text{input width} - \text{kernel width}) / \text{stride},$$

And, the same for finding parameters

$$= (\text{kernel size} * \text{depth dimension of input} * \text{filters (or feature maps)}) \quad \# \text{ for weights}$$

$$+ \text{filters} \quad \# \text{ for biases}$$

And, output image size of avg. pooling layer with stride 2 with “valid” padding

= Half the height and half the width of the input image (rounded down due to no padding)

So,

Applying the formula, we get

Layer	Size	Parameters
Out	10	$(84 * 10) + 10 = 850$
F6	84	$(1 * 1 * 120 * 84) + 84 = 10164$
C5	1×1 $fw_{c5} = 1 + (5 - 5) / 1 = 1,$ $f_{h_{c5}} = 1$	$(5 * 5 * 16 * 120) + 120 = 48120$
S4	$(10/2) \times (10/2)$ 5×5	—
C3	10×10 $fw_{c3} = 1 + (14 - 5) / 1 = 10,$ $f_{h_{c3}} = 10$	$(5 * 5 * 6 * 16) + 16 = 2416$
S2	$(28/2) \times (28/2)$ $= 14 \times 14$	—
C1	28×28 $fw_{c1} = 1 + (32 - 5) / 1 = 28,$ $f_{h_{c1}} = 28$	$(5 * 5 * 6) + 6$ $= 156$

19. Discuss how data augmentation works. Also, identify what problem a local response normalization solves and how.

Answer:

Data Augmentation:

Data augmentation artificially increases the size of the training set by generating many realistic variants of each training instance. This reduces overfitting, making this a regularization technique. The generated instances should be as realistic as possible: ideally, given an image from the augmented training set, a human should not be able to tell whether it was augmented or not. Simply adding white noise will not help; the modifications should be learnable (white noise is not). For example, every picture in the training set can be slightly shifted, rotated, and resized by various amounts, and the resulting pictures can be added to the training set. To do this, Keras data augmentation layers can be used. This forces the model to be more tolerant to variations in the position, orientation, and size of the objects in the pictures. To produce a model that's more tolerant of different lighting conditions, many images can be generated with various contrasts. In general, the pictures can also be flipped horizontally (except for text, and other asymmetrical objects). By combining these transformations, training set size can be increased greatly.

Local Response Normalization:

A competitive normalization step, called local response normalization (LRN), can be used immediately after the ReLU step of convolutional layers for the most strongly activated neurons to inhibit other neurons located at the same position in neighboring feature maps. Such competitive activation has been observed in biological neurons. This encourages different feature maps to specialize, pushing them apart and forcing them to explore a wider range of features, ultimately improving generalization. The equation below shows how to apply LRN.

$$b_i = a_i \left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{high}}} a_j^2 \right)^{-\beta} \quad \text{with} \quad \begin{cases} j_{\text{high}} = \min \left(i + \frac{r}{2}, f_n - 1 \right) \\ j_{\text{low}} = \max \left(0, i - \frac{r}{2} \right) \end{cases}$$

In this equation:

- b_i is the normalized output of the neuron located in feature map i , at some row u and column v (u and v are not shown considering only the neurons located at this row and column).
- a_i is the activation of that neuron after the ReLU step, but before normalization.
- k , α , β , and r are the hyperparameters. k is called the *bias*, and r is called the *depth radius*.

- f_n is the number of feature maps.

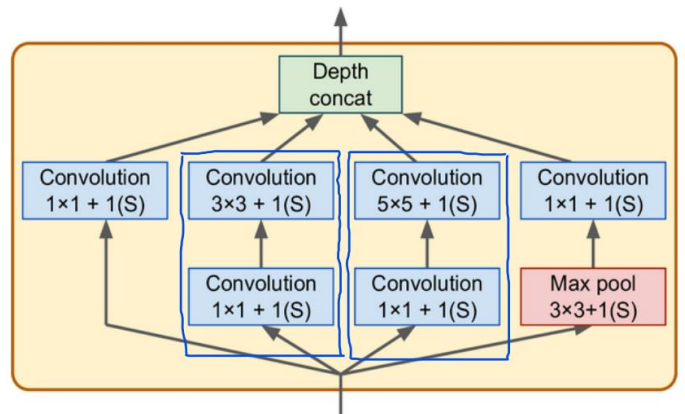
For example, if $r = 2$ and a neuron has a strong activation, it will inhibit the activation of the neurons located in the feature maps immediately above and below its own. This step can be implemented by using the `tf.nn.local_response_normalization()` function that can be wrapped in a Lambda layer to be used it in a Keras model).

20. Identify how inception modules contribute to GoogLeNet to perform better than LeNet-5 and AlexNet with much less parameters than these models. Draw a neat diagram of an inception module.

Answer:

This great performance came in large part from the fact that the network was much deeper than previous CNNs. This was made possible by subnetworks called *inception modules*, which allow GoogLeNet to use parameters much more efficiently than previous architectures: GoogLeNet actually has 10 times fewer parameters than AlexNet. The figure shows the architecture of an inception module. The notation " $3 \times 3 + 1(S)$ " means that the layer uses a 3×3 kernel, stride 1, and "same" padding.

The input signal is first fed to four different layers in parallel. All convolutional layers use the ReLU activation function. Note that the top convolutional layers use different kernel sizes (1×1 , 3×3 , and 5×5), allowing them to capture patterns at different scales. Also note that every single layer uses a stride of 1 and "same" padding (even the max pooling layer), so their outputs all have the same height and width as their inputs.



This makes it possible to concatenate all the outputs along the depth dimension in the final depth concatenation layer (i.e., to stack the feature maps from all four top convolutional layers). It can be implemented using Keras `Concatenate` layer, using the default `axis=-1`.

Capturing Patterns Across Depth Dimension:

Convolutional layers with 1×1 kernels in the inception modules cannot capture any features for sure because they look at only one pixel at a time. In fact, these layers serve three purposes:

- Although they cannot capture spatial patterns, they can capture patterns along the depth dimension (i.e., across channels).

- They are configured to output fewer feature maps than their inputs, so they serve as bottleneck layers, meaning they reduce dimensionality. This cuts the computational cost and the number of parameters, speeding up training and improving generalization.
- Each pair of convolutional layers ($[1 \times 1, 3 \times 3]$ and $[1 \times 1, 5 \times 5]$) acts like a single powerful convolutional layer, capable of capturing more complex patterns. A convolutional layer is equivalent to sweeping a dense layer across the image (at each location, it only looks at a small receptive field), and these pairs of convolutional layers are equivalent to sweeping two-layer neural networks across the image.

In short, the whole inception module can be thought of as a convolutional layer on steroids, able to output feature maps that capture complex patterns at various scales.

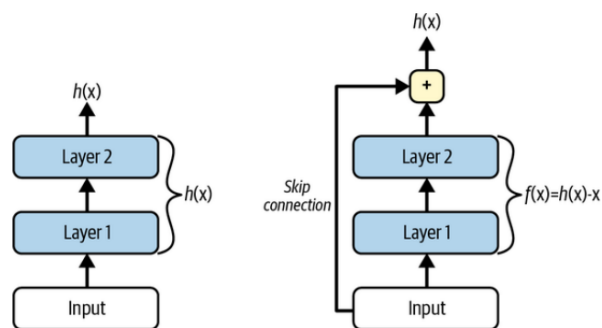
21. Explain how *residual learning* works to train a very deep neural network overcoming the vanishing gradient problem. Provide all relevant figures.

Answer:

Skip Connections & Residual Learning:

Skip connections (also called shortcut connections) are the signals fed into a layer that are also added to the output of a layer located higher up the stack.

When training a neural network, the goal is to make it model a target function $h(\mathbf{x})$. If input $\mathbf{x}$ is added to the output of the network (i.e., adding a skip connection), then the network will be forced to model $f(\mathbf{x}) = h(\mathbf{x}) - \mathbf{x}$ rather than $h(\mathbf{x})$. This is called *residual learning* (see figure).



Faster Training with Skip Connection:

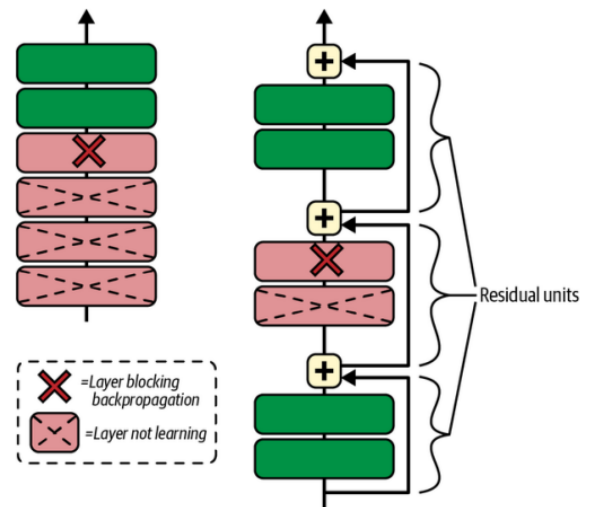
When a regular neural network is initialized, its weights are close to zero, so the network just outputs values close to zero. If a skip connection is added, the resulting network just outputs a copy of its inputs; in other words, it initially models the identity function. If the target function is fairly close to the identity function (which is often the case), this will speed up training considerably.

Residual Units:

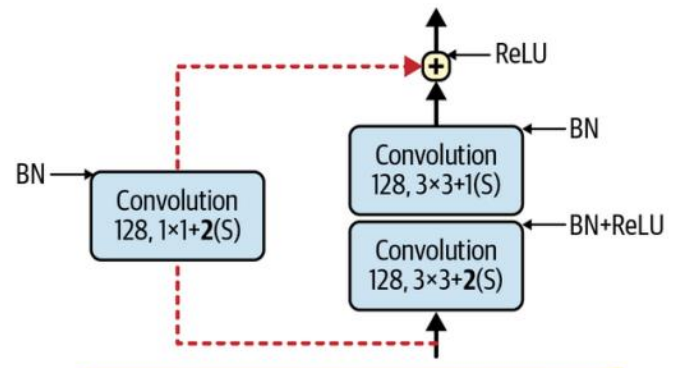
Moreover, if many skip connections are added, the network can start making progress even if several layers have not started learning yet (see figure). Thanks to skip connections, the signal can easily make its way across the whole network. The deep residual network can be seen as a stack of residual units (RUs), where each residual unit is a small neural network with a skip connection.

Residual Units in ResNet:

ResNet's architecture starts and ends exactly like GoogLeNet, and in between is just a very deep stack of residual units. Each residual unit is composed of two convolutional layers (and no pooling layer), with batch normalization (BN) and ReLU activation, using 3×3 kernels and preserving spatial dimensions (stride 1, "same" padding).



The number of feature maps is doubled every few residual units, at the same time as their height and width are halved (using a convolutional layer with stride 2). When this happens, the inputs cannot be added directly to the outputs of the residual unit because they don't have the same shape. To solve this problem, the inputs are passed through a 1×1 convolutional layer with stride 2 and the right number of output feature maps (see figure).



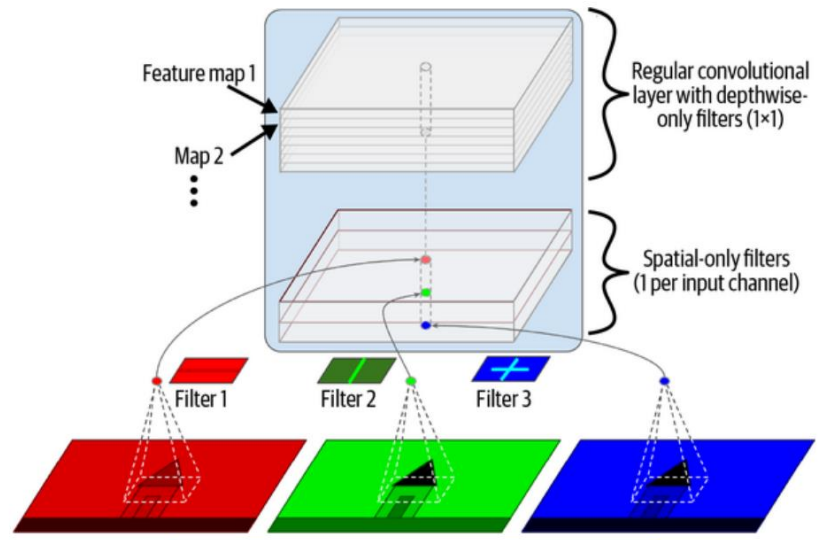
22. With an appropriate diagram, explain the concept behind *depth-wise separable convolution layer* used in Xception architecture. Also, identify why this architecture is considered as a GoogLeNet variant.

Answer:

Depthwise Separable Convolution Layer:

Xception (or *Extreme Inception*)

merges the ideas of GoogLeNet and ResNet, but it replaces the inception modules with a special type of layer called a *depthwise separable convolution layer* (or separable convolution layer for short). While a regular convolutional layer uses filters that try to simultaneously capture spatial patterns (e.g., an oval) and cross-channel patterns (e.g., mouth + nose + eyes = face), a separable convolutional layer makes the strong assumption that spatial patterns and cross-channel patterns can be modeled separately (see figure). Thus, it is composed of two parts:



- the first part applies a single spatial filter to each input feature map, then
- the second part looks exclusively for cross-channel patterns—it is just a regular convolutional layer with 1×1 filters.

Since separable convolutional layers only have one spatial filter per input channel, their usage should be avoided after layers that have too few channels, such as the input layer (the figure is the exception as it is just for illustration purposes). For this reason, the Xception architecture starts with 2 regular convolutional layers, but then the rest of the architecture uses only separable convolutions (34 in all), plus a few max pooling layers and the usual final layers (a global average pooling layer and a dense output layer).

Xception as a GoogLeNet Variant:

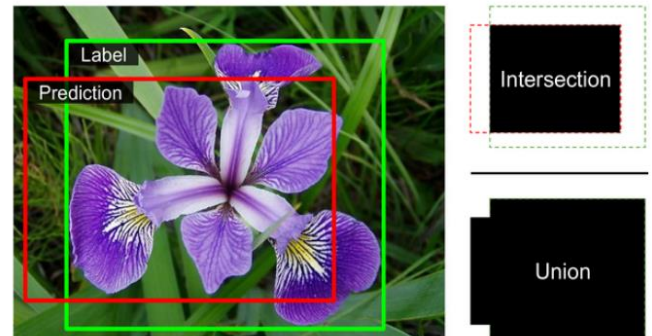
Convolutional layers with 1×1 filters in an inception module look exclusively for cross-channel patterns. However, the convolutional layers that sit on top of them are regular convolutional layers that look both for spatial and cross-channel patterns. So, an inception module can be thought of as an intermediate between a regular convolutional layer (which considers spatial patterns and cross-channel patterns jointly) and a separable convolutional layer (which considers them separately). In practice, it seems that separable convolutional layers often perform better. So, even though Xception contains no inception modules at all, it is considered a variant of GoogLeNet.

23. a) Explain with an appropriate diagram how metric *Intersection over Union* works to measure the bounding box prediction performance.

b) Identify the working principle of object localization in images using sliding-CNN approach.

a) Better Metric for Bounding Box Prediction:

The MSE often works fairly well as a cost function to train the image classification model, but it is not a great metric to evaluate how well the model can predict bounding boxes. The most common metric for this is the *intersection over union* (IoU): the area of overlap between the predicted bounding box and the target bounding box, divided by the area of their union (see figure). In Keras, it is implemented by the `tf.keras.metrics.MeanIoU` class.



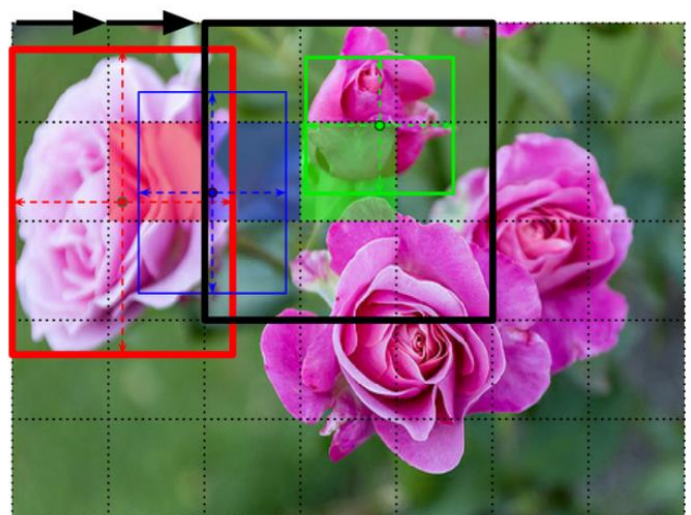
b) Working Principle of Object Localization using Sliding-CNN Approach:

Classification + Localization + Objectness Score: The task of classifying and localizing multiple objects in an image is called *object detection*. A common approach is to take a CNN that is trained to classify and locate a single object roughly centered in the image, then slide this CNN across the image and make predictions at each step. The CNN is generally trained to predict not only class probabilities and a bounding box, but also an *objectness score*: this is the estimated probability that the image does indeed contain an object centered near the middle. This is a binary classification output; it can be produced by a dense output layer with a single unit, using the sigmoid activation function and trained using the binary cross-entropy loss.

Sliding-CNN Approach: The sliding-CNN approach is illustrated in figure. In this example, the image was chopped into a 5×7 grid, and we see—the thick black rectangle—sliding across all 3×3 regions and making predictions at each step.

In this figure, the CNN has already made predictions for three of these 3×3 regions:

- When looking at the top-left 3×3 region (centered on the red-shaded grid cell located in the second row and second column), it detected the leftmost rose. Notice that the predicted bounding box exceeds the



boundary of this 3×3 region. That's absolutely fine: even though the CNN could not see the bottom part of the rose, it was able to make a reasonable guess as to where it might be. It also predicted class probabilities, giving a high probability to the "rose" class. Lastly, it predicted a fairly high objectness score, since the center of the bounding box lies within the central grid cell (in this figure, the objectness score is represented by the thickness of the bounding box).

- When looking at the next 3×3 region, one grid cell to the right (centered on the shaded blue square), it did not detect any flower centered in that region, so it predicted a very low objectness score; therefore, the predicted bounding box and class probabilities can safely be ignored.
- Finally, when looking at the next 3×3 region, again one grid cell to the right (centered on the shaded green cell), it detected the rose at the top, although not perfectly: this rose is not well centered within this region, so the predicted objectness score was not very high.

Postprocessing to Remove Duplicate Detection using Non-max Suppression: Sliding the CNN across the whole image would give a total of 15 predicted bounding boxes, organized in a 3×5 grid, with each bounding box accompanied by its estimated class probabilities and objectness score. It will often detect the same object multiple times, at slightly different positions. Some postprocessing is needed to get rid of all the unnecessary bounding boxes. A common approach for this is called *non-max suppression*. Here's how it works:

- 1) First, get rid of all the bounding boxes for which the objectness score is below some threshold: since the CNN believes there's no object at that location, the bounding box is useless.
- 2) Find the remaining bounding box with the highest objectness score, and get rid of all the other remaining bounding boxes that overlap a lot with it (e.g., with an IoU greater than 60%).
- 3) Repeat step 2 until there are no more bounding boxes to get rid of.

Downside of the Sliding-CNN Approach: This simple approach to object detection works pretty well, but it requires running the CNN many times (15 times in this example), so it is quite slow.

24. Develop design considerations to replace a dense layer at the top with a convolutional layer in a CNN in object detection task. With an appropriate figure, explain how a Fully Convolutional Network processes a varying length image.

Answer:

Let's understand replacing a dense layer (generally placed on top of the CNN) with a convolutional layer. Suppose a dense layer with 200 neurons sits on top of a convolutional layer that outputs 100 feature maps, each of size 7×7 (this is the feature map size, not the kernel size). Each neuron will compute a weighted sum of all $100 \times 7 \times 7$ activations from the convolutional layer (plus a bias term). Now let's see what happens if the dense layer is replaced with a convolutional layer using 200 filters, each of size 7×7 , and with "valid" padding. This layer will output 200 feature maps, each 1×1 (since the kernel is exactly the size of the input feature maps and we are using "valid" padding). In other words, it will output 200 numbers, just like the dense layer did; and these numbers will be precisely the same as those the dense layer produced. The only difference is that the dense layer's output was a tensor of shape [batch size, 200], while the convolutional layer will output a tensor of shape [batch size, 1, 1, 200].

Conditions to Replace a Dense Layer with a Convolutional Layer:

To convert a dense layer to a convolutional layer, the number of filters in the convolutional layer must be equal to the number of units in the dense layer, the filter size must be equal to the size of the input feature maps, and you must use "valid" padding. The stride may be set to 1 or more.

Benefit of Using a Convolution Layer as a Top Layer:

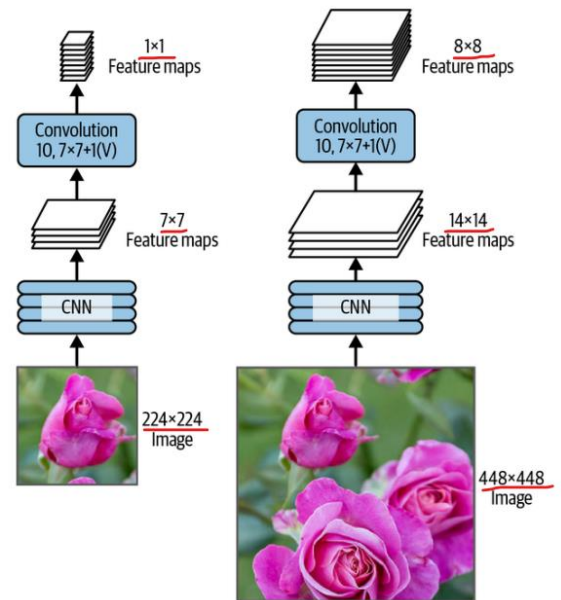
While a dense layer expects a specific input size (since it has one weight per input feature), a convolutional layer will happily process images of any size (however, it does expect its inputs to have a specific number of channels, since each kernel contains a different set of weights for each input channel). Since a Fully Convolutional Networks (FCN) contains only convolutional layers (and pooling layers, which have the same property), it can be trained and executed on images of any size.

A Fully Convolutional Network Processing a Varying Length Image:

For example, suppose a CNN is already trained for flower classification and localization on 224×224 images, and it outputs 10 numbers:

- Outputs 0 to 4 are sent through the softmax activation function, and this gives the class probabilities (one per class).
- Output 5 is sent through the sigmoid activation function, and this gives the objectness score.
- Outputs 6 and 7 represent the bounding box's center coordinates; they also go through a sigmoid activation function to ensure they range from 0 to 1.
- Lastly, outputs 8 and 9 represent the bounding box's height and width; they do not go through any activation function to allow the bounding boxes to extend beyond the borders of the image.

Now, the CNN's dense layers can be converted to convolutional layers. This can just be done by copying the weights from the dense layers to the convolutional layers without retraining the model. Alternatively, the CNN could have been converted into an FCN before training. Now suppose the last convolutional layer before the output layer (also called the bottleneck layer) outputs 7×7 feature maps when the network is fed a 224×224 image (see the left side of the figure). If the FCN is fed a 448×448 image (see the right side of the figure), the bottleneck layer will now output 14×14 feature maps. Since the dense output layer was replaced by a convolutional layer using 10 filters of size 7×7 , with "valid" padding and stride 1, the output will be composed of 10 feature maps, each of size 8×8 (since $14 - 7 + 1 = 8$). In other words, the FCN will process the whole image only once, and it will output an 8×8 grid where each cell contains 10 numbers (5 class probabilities, 1 objectness score, and 4 bounding box coordinates). It's exactly like taking the original CNN and sliding it across the image using 8 steps per row and 8 steps per column. To visualize this, imagine chopping the original image into a 14×14 grid, then sliding a 7×7 window across this grid; there will be $8 \times 8 = 64$ possible locations for the window, hence 8×8 predictions. However, the FCN approach is much more efficient, since the network only looks at the image once.



25. Identify how semantic segmentation works covering both traditional transposed convolutional layer and skip connections approach in a fully convolutional network. Provide appropriate diagrams.

Answer:

In semantic segmentation, each pixel is classified according to the class of the object it belongs to. But different objects of the same class are not distinguished. The main difficulty in this task is that when images go through a regular convolutional neural network (CNN), they gradually lose their spatial resolution (due to the layers with strides greater than 1); so, a regular CNN may end up knowing an object's tentative location in the image, but it will not be much more precise than that. Just like object detection, there are many different approaches to tackle this problem.

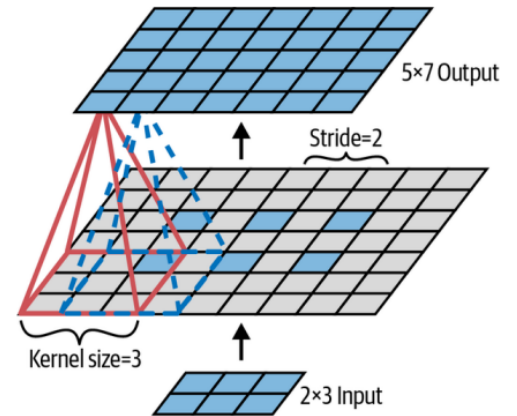
Upsampling layer in a Fully Convolutional Network (FCN):

A pretrained CNN can be turned into an FCN. But last layer output feature maps will be too coarse because these will be several times smaller than the input image due to the overall strides applied

(adding up all the strides greater than 1) to the input image. So, a single upsampling layer can be added so that the resolution can be increased by a factor it was reduced due to strides.

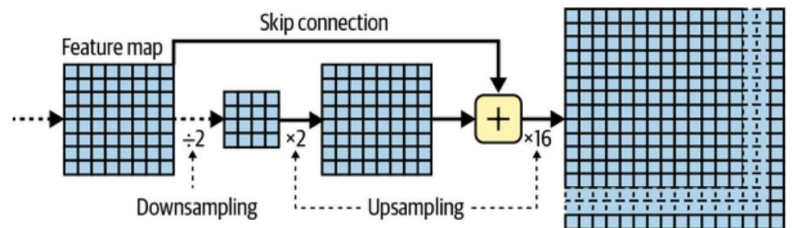
Upsampling using Transposed Convolutional Layer:

This is equivalent to first stretching the image by inserting empty rows and columns (full of zeros), then performing a regular convolution (see figure). The transposed convolutional layer can be initialized to perform something close to linear interpolation, but since it is a trainable layer, it will learn to do better during training. In Keras, 2D transposed convolutional layer can be created using the `Conv2DTranspose` layer. In a transposed convolutional layer, the stride defines how much the input will be stretched, not the size of the filter steps, so the larger the stride, the larger the output (unlike for convolutional layers or pooling layers).



Upsampling with Skip Connections:

Transposed convolutional layers for upsampling is imprecise. To do better, skip connections can be added from lower layers: for example, the output image can first be upsampled by a factor of 2, and the output of a lower layer that has this double resolution can be added. The result can then be upsampled by a factor of 16, leading to a total upsampling factor of 32 (see figure). This recovers some of the spatial resolution that could be lost in earlier pooling layers. In short, the output of the original CNN goes through the following extra steps: upsample $\times 2$, add the output of a lower layer (of the appropriate scale), upsample $\times 2$, add the output of an even lower layer, and finally upsample $\times 8$. It is even possible to scale up beyond the size of the original image: this can be used to increase the resolution of an image, which is a technique called super-resolution.



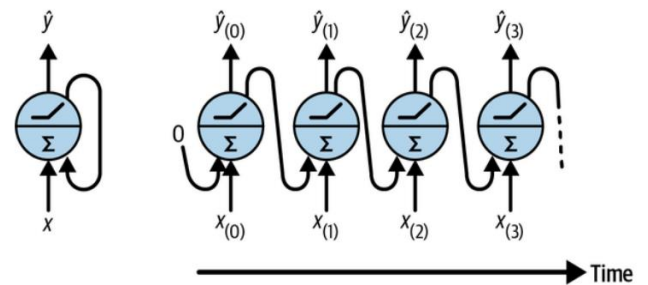
CHAPTER: Sequential Processing

26. Illustrate the working of a *recurrent neural network* and identify how it is different from *feed-forward neural network*. Provide appropriate diagrams and expressions to calculate output of a recurrent layer.

Answer:

Activation Flow Direction & Unrolling Through Time:

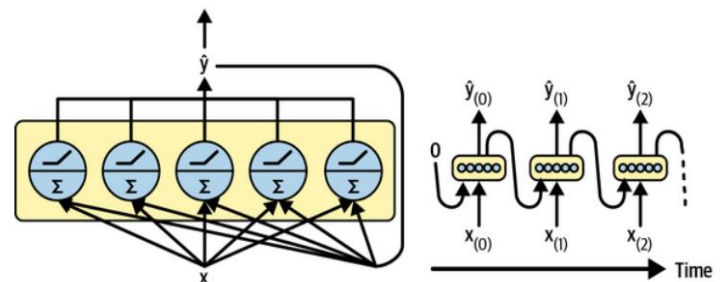
In feedforward neural networks, the activations flow only in one direction, from the input layer to the output layer, for a recurrent neural network, it also has connections pointing backward. Let's look at the simplest possible RNN, composed of one neuron receiving inputs, producing an output, and sending that output back to itself, as shown in figure



(left). At each *time step* t , this recurrent neuron receives the inputs $\mathbf{x}_{(t)}$ as well as its own output from the previous time step, $\hat{\mathbf{y}}_{(t-1)}$. Since there is no previous output at the first time step, it is generally set to 0. This tiny network can be represented against the time axis, as shown in figure (right). This is called *unrolling the network through time* (it's the same recurrent neuron represented once per time step).

Layer of Recurrent Units:

A layer of recurrent neurons can also be created easily. At each time step t , every neuron receives both the input vector $\mathbf{x}_{(t)}$ and the output vector from the previous time step $\hat{\mathbf{y}}_{(t-1)}$, as shown in figure. Both the inputs and outputs are now vectors.



Weights of a Recurrent Layer:

Each recurrent neuron has two sets of weights: one for the inputs $\mathbf{x}_{(t)}$ and the other for the outputs of the previous time step, $\hat{\mathbf{y}}_{(t-1)}$. Let's call these weight vectors $\mathbf{w}_x$ and $\mathbf{w}_{\hat{y}}$. If the whole recurrent layer is considered instead of just one recurrent neuron, all the weight vectors can be placed in two weight matrices: $\mathbf{W}_x$ and $\mathbf{W}_{\hat{y}}$.

Calculating the Output of a Recurrent Layer:

The output vector of the whole recurrent layer can then be computed as shown in equation below, where $\mathbf{b}$ is the bias vector and $\phi(\cdot)$ is the activation function (e.g., ReLU).

$$\hat{\mathbf{y}}_{(t)} = \phi(\mathbf{W}_x^T \mathbf{x}_{(t)} + \mathbf{W}_y^T \hat{\mathbf{y}}_{(t-1)} + \mathbf{b})$$

Just as with feedforward neural networks, a recurrent layer's output can be computed in one shot for an entire mini-batch by placing all the inputs at time step t into an input matrix $\mathbf{X}_{(t)}$ as shown in the equation below.

$$\begin{aligned} \hat{\mathbf{Y}}_{(t)} &= \phi(\mathbf{X}_{(t)} \mathbf{W}_x + \hat{\mathbf{Y}}_{(t-1)} \mathbf{W}_y + \mathbf{b}) \\ &= \phi([\mathbf{X}_{(t)} \quad \hat{\mathbf{Y}}_{(t-1)}] \mathbf{W} + \mathbf{b}) \text{ where } \mathbf{W} = [\mathbf{W}_x \quad \mathbf{W}_y]^T \end{aligned}$$

In this equation:

- $\hat{\mathbf{Y}}_{(t)}$ is an $m \times n_{\text{neurons}}$ matrix containing the layer's outputs at time step t for each instance in the mini-batch (m is the number of instances in the mini-batch and n_{neurons} is the number of neurons).
- $\mathbf{X}_{(t)}$ is an $m \times n_{\text{inputs}}$ matrix containing the inputs for all instances (n_{inputs} is the number of input features).
- $\mathbf{W}_x$ is an $n_{\text{inputs}} \times n_{\text{neurons}}$ matrix containing the connection weights for the inputs of the current time step.
- $\mathbf{W}_y$ is an $n_{\text{neurons}} \times n_{\text{neurons}}$ matrix containing the connection weights for the outputs of the previous time step.
- $\mathbf{b}$ is a vector of size n_{neurons} containing each neuron's bias term.
- The weight matrices $\mathbf{W}_x$ and $\mathbf{W}_y$ are often concatenated vertically into a single weight matrix $\mathbf{W}$ of shape $(n_{\text{inputs}} + n_{\text{neurons}}) \times n_{\text{neurons}}$ (see the second line of the above equation).
- The notation $[\mathbf{X}_{(t)} \quad \hat{\mathbf{Y}}_{(t-1)}]$ represents the horizontal concatenation of the matrices $\mathbf{X}_{(t)}$ and $\hat{\mathbf{Y}}_{(t-1)}$.

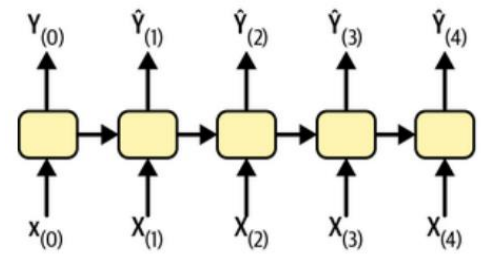
$\hat{\mathbf{Y}}_{(t)}$ is a function of $\mathbf{X}_{(t)}$ and $\hat{\mathbf{Y}}_{(t-1)}$, which is a function of $\mathbf{X}_{(t-1)}$ and $\hat{\mathbf{Y}}_{(t-2)}$, which is a function of $\mathbf{X}_{(t-2)}$ and $\hat{\mathbf{Y}}_{(t-3)}$, and so on. This makes $\hat{\mathbf{Y}}_{(t)}$ a function of all the inputs since time $t = 0$ (that is, $\mathbf{X}_{(0)}$, $\mathbf{X}_{(1)}$, ..., $\mathbf{X}_{(t)}$). At the first time step, $t = 0$, there are no previous outputs, so they are typically assumed to be all zeros.

27. Explain the architecture and the working of *sequence-to-sequence* network, *sequence-to-vector* network, *vector-to-sequence* network and *encoder-decoder* network. Also, develop intuition for strategy “*backpropagation through time*” to train a recurrent network. Provide appropriate diagrams.

Answer:

Sequence-to-Sequence Networks:

An RNN can simultaneously take a sequence of inputs and produce a sequence of outputs (see figure). This type of sequence-to-sequence network is useful to forecast time series, such as home's daily power consumption over the last N days fed in training it to output the power consumption shifted by one day into the future (i.e., from $N - 1$ days ago to tomorrow).

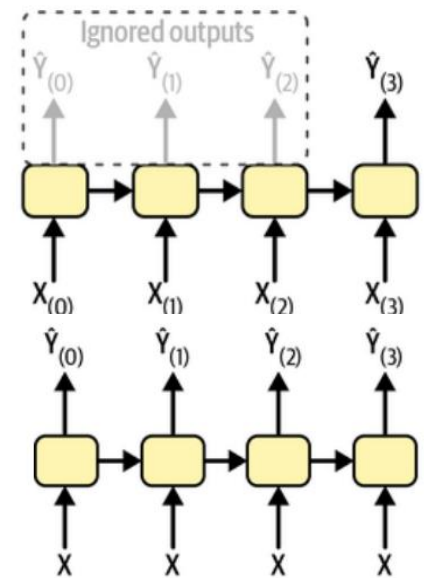


Sequence-to-Vector Networks:

Alternatively, the network can be fed with a sequence of inputs ignoring all outputs except for the last one (see figure). This is a *sequence-to-vector network*. For example, a sequence of words corresponding to a movie review can be fed into the network, and the network would output a sentiment score (e.g., from 0 [hate] to 1 [love]).

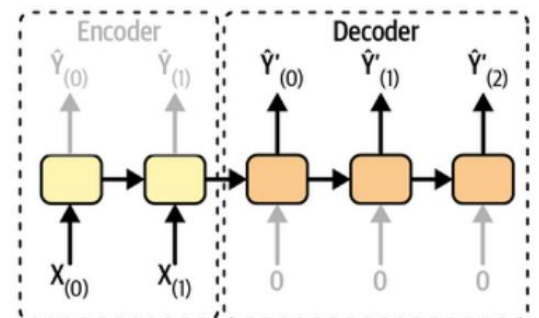
Vector-to-Sequence Networks:

Conversely, the same input vector could also be fed into the network over and over again at each time step and it outputs a sequence (see figure). This is a *vector-to-sequence network*. For example, the input could be an image (or the output of a CNN), and the output could be a caption for that image.



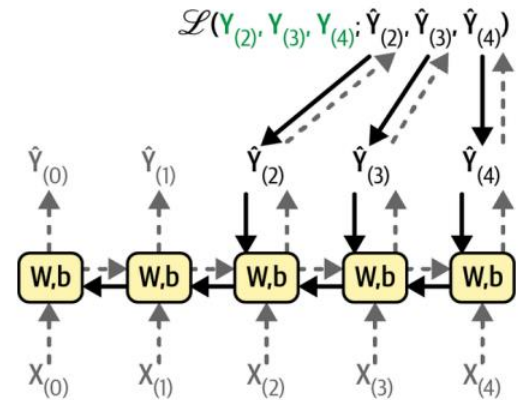
Encoder-Decoder Networks:

An encoder-decoder network (see figure) can be made by having a sequence-to-vector network, called an encoder, followed by a vector-to-sequence network, called a decoder. For example, this could be used for translating a sentence from one language to another. A sentence in one language can be fed into the network, then the encoder would convert this sentence into a single vector representation, and then the decoder would decode this vector into a sentence in another language. This two-step model, called an encoder-decoder.



Backpropagation Through Time:

To train an RNN, the trick is to unroll it through time and then use regular backpropagation (see figure). This strategy is called backpropagation through time (BPTT). Just like in regular backpropagation, there is a first forward pass through the unrolled network (represented by the dashed arrows). Then the output sequence is evaluated using a loss function $\mathcal{L}(\mathbf{Y}_{(0)}, \mathbf{Y}_{(1)}, \dots, \mathbf{Y}_{(T)}; \hat{\mathbf{Y}}_{(0)}, \hat{\mathbf{Y}}_{(1)}, \dots, \hat{\mathbf{Y}}_{(T)})$ (where $\hat{\mathbf{Y}}_{(i)}$ is the i^{th} target, $\hat{\mathbf{Y}}_{(i)}$ is the i^{th} prediction, and T is the max time step).



This loss function may ignore some outputs. For example, in a sequence-to-vector RNN, all outputs are ignored except for the very last one. In the figure, the loss function is computed based on the last three outputs only. The gradients of that loss function are then propagated backward through the unrolled network (represented by the solid arrows). In this example, since the outputs $\hat{\mathbf{Y}}_{(0)}$ and $\hat{\mathbf{Y}}_{(1)}$ are not used to compute the loss, the gradients do not flow backward through them; they only flow through $\hat{\mathbf{Y}}_{(2)}$, $\hat{\mathbf{Y}}_{(3)}$, and $\hat{\mathbf{Y}}_{(4)}$. Moreover, since the same parameters $\mathbf{W}$ and $\mathbf{b}$ are used at each time step, their gradients will be tweaked multiple times during backprop. Once the backward phase is complete and all the gradients have been computed, BPTT can perform a gradient descent step to update the parameters (this is no different from regular backprop).

28. Explain the following with respect to time-series forecasting. Provide relevant examples.

- a) Univariate & Multivariate time-series
- b) Seasonality
- c) Lagged time-series & Differencing
- d) Autocorrelated time-series
- e) Stationary time-series

Answer:

a) Univariate & Multivariate time-series:

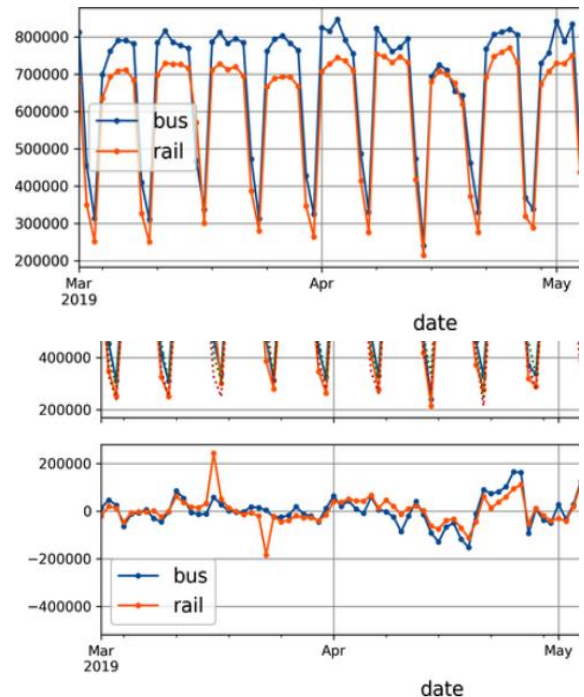
Data with values at different time steps, usually at regular intervals, is called a time series. More specifically, since there are multiple values (often considered features) per time step, this is called a multivariate time series. If we only looked at the bus column in the Chicago's Transit Authority (CTA) dataset, it would be a univariate time series, with a single value per time step. Similarly, if more than columns such as bus, rail and day are considered, then it will be a multivariate time-series. Predicting future values (i.e., forecasting) is the most typical task when dealing with time series.

b) Seasonality:

Looking at the figure, a similar pattern clearly observed to be repeated every week. This is called a *weekly seasonality*. Looking at the time series, there doesn't appear to be any significant monthly seasonality.

c) Lagged time-series & Differencing:

In time series analysis, lag refers to the delay between an observed data point and its preceding values. So, lag is the time difference between two observations in a sequence, or the number of steps back in time a past observation is from the current time. In the CTA dataset, two overlaid time-series (for bus and rail) as well as the same time series lagged by one week (i.e., shifted toward the right) are shown using dotted lines (see figure on top). The difference between the two (i.e., the value at time t minus the value at, for example, time $t - 7$) is called the *differencing* (see figure at bottom).

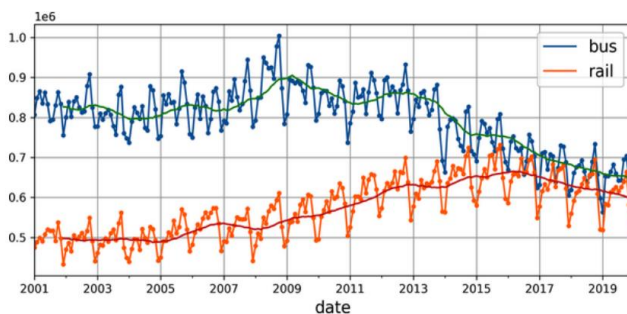


d) Autocorrelated Time-series:

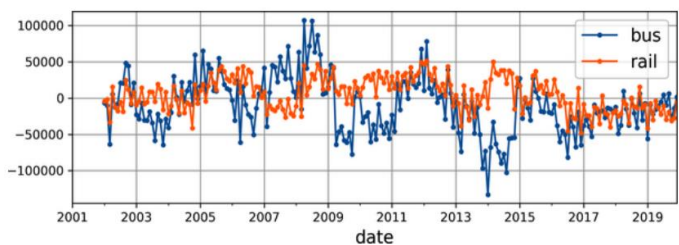
Autocorrelation measures the degree of similarity between a given time series and the lagged version of that time series over successive time periods. It is similar to calculating the correlation between two different variables except in Autocorrelation we calculate the correlation between two different versions of the same time series. The top figure above shows how closely the lagged time series tracks the actual time series. When a time series is correlated with a lagged version of itself, we say that the time series is autocorrelated. In this example, most of the differences are fairly small, except at the end of May and it could be due to various reasons.

e) Stationary Time-series:

A stationary time series is one whose statistical properties, such as mean, variance, and autocorrelation, remain constant over time.



Yearly seasonality and long-term trends



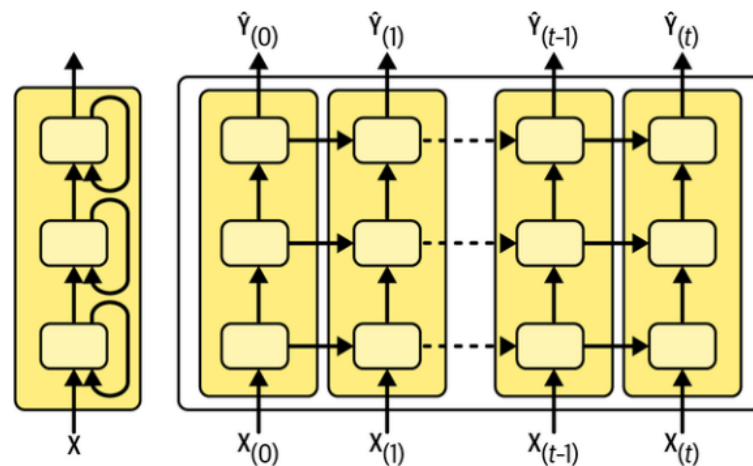
The 12-month difference

The first plot is a 12-month rolling average for each series to visualize long-term trends from CTA dataset. There's definitely some yearly seasonality as well, although it is noisier, and more visible for the rail series than the bus series. Peaks and troughs are visible at roughly the same dates each year. The 12-month difference plot is shown on the right. Now the differencing not only removed the yearly seasonality, but it also removed the long-term trends. For example, the linear downward trend present in the time series from 2016 to 2019 became a roughly constant negative value in the differenced time series. In fact, differencing is a common technique used to remove trend and seasonality from a time series: it's easier to study a stationary time series, meaning one whose statistical properties remain constant over time, without any seasonality or trends. Once accurate forecasts can be made on the differenced time series, it's easy to turn them into forecasts for the actual time series by just adding back the past values that were previously subtracted.

29. Make use of simple recurrent neural network (RNN) layers to develop a deep RNN giving code-snippet and explain its working. Assume appropriate considerations and provide the network architecture diagram along with justification for return sequences in recurrent layers.

Answer:

It is quite common to stack multiple layers of cells, as shown in figure. This builds a deep RNN.



Implementing a deep RNN with Keras is straightforward: it is just about stacking recurrent layers. In the following example, three `SimpleRNN` layers are used. But any other type of recurrent layer such as an `LSTM` layer or a `GRU` layer could also be used instead.

```
deep_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])
```

The first two are sequence-to-sequence layers, and the last one is a sequence-to-vector layer. Finally, the `Dense` layer produces the model's forecast. This can be thought of as a vector-to-vector layer. So this model is just like the model represented in figure, except the outputs $\hat{Y}_{(0)}$ to $\hat{Y}_{(t-1)}$ are ignored, and there's a dense layer on top of $\hat{Y}_{(t)}$, which outputs the actual forecast.

Importance of Return Sequences:

It is to be ensured to set `return_sequences=True` for all recurrent layers except the last one considering the output from the last layer would be the prediction we care about. Omitting this setting for one recurrent layer, will output a 2D array containing only the output of the last time step, instead of a 3D array containing outputs for all time steps. The next recurrent layer will complain about not receiving the sequences in the expected 3D format.

30. Choose appropriate measure to alleviate unstable gradient problem in handling long sequences with a recurrent neural network (RNN) and discuss the solutions.

Answer:

To train an RNN on long sequences, it must be run over many time steps, making the unrolled RNN a very deep network. Just like any deep neural network it may suffer from the unstable gradients problem.

Inappropriateness of Saturating Activation Function: Many of the tricks that are generally used in deep nets to alleviate the unstable gradients problem can also be used for RNNs: good parameter initialization, faster optimizers, dropout, and so on. However, non-saturating activation functions (e.g., ReLU) may not help as much here. In fact, they may actually lead the RNN to be even more unstable during training. Because gradient descent updates the weights in a way that increases the outputs slightly at the first time step. As the same weights are used at every time step, the outputs at the second time step may also be slightly increased, and those at the third, and so on until the outputs explode—and a non-saturating activation function does not prevent that.

Appropriateness of Smaller Learning Rate & tanh Activation Function: This risk can be reduced by using a smaller learning rate or using a saturating activation function like the hyperbolic tangent (tanh).

Gradient Clipping: In much the same way, the gradients themselves can explode. If instability in training is noticed, then the size of the gradients can be monitored, and perhaps gradient clipping can be used.

Batch Normalization Between Recurrent Layers: Moreover, batch normalization (BN) cannot be used as efficiently with RNNs as with deep feedforward nets. In fact, it cannot be used between time steps, only between recurrent layers. To be more precise, it is technically possible to add a BN layer to a memory cell so that it will be applied at each time step (both on the inputs for that time step and on the hidden state from the previous step). However, the same BN layer will be used at each time step, with the same parameters, regardless of the actual scale and offset of the inputs and hidden state. In practice, this does not yield good results. BN would be slightly beneficial only when it is applied to the layer's inputs, not to the hidden states. In other words, it is slightly better than nothing when applied between recurrent layers, but not within recurrent layers. In Keras, the BN can be applied between layers simply by adding a `BatchNormalization` layer before each recurrent layer, but it will slow down training, and it may not help much.

Layer Normalization: Another form of normalization often works better with RNNs is layer normalization. It is very similar to batch normalization, but instead of normalizing across the batch dimension, layer normalization normalizes across the features dimension. One advantage is that it can compute the required statistics on the fly, at each time step, independently for each instance. This also means that it behaves the same way during training and testing (as opposed to BN), and it does not need to use exponential moving averages to estimate the feature statistics across all instances in the training set, like BN does. Like BN, layer normalization learns a scale and an offset parameter for each input. In an RNN, it is typically used right after the linear combination of the inputs and the hidden states.

With these techniques, the unstable gradients problem can be alleviated and an RNN can be trained much more efficiently.

31. Experiment with a *long short-term memory* (LSTM) cell to tackle short-term memory problem. Provide network architecture diagram, relevant code-snippet and equations for LSTM computations.

Answer:

Due to the transformations that the data goes through when traversing an RNN, some information is lost at each time step. After a while, the RNN's state contains virtually no trace of the first inputs. This can be a showstopper. This can be imagined by considering someone trying to translate a long sentence, but by the time she finished reading it, she forgot where it started. To tackle this problem, various types of cells with long-term memory have been introduced. They have proven so successful that the basic cells are not used much anymore. The most popular of these long-term memory cells is the LSTM cell.

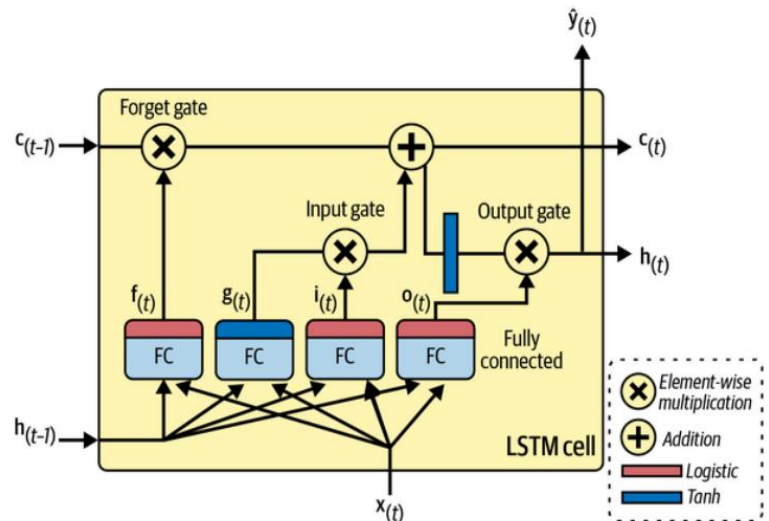
Long Short-term Memory (LSTM) Cell

The LSTM cell can be used very much like a basic cell, except it will perform much better; training will converge faster, and it will detect longer-term patterns in the data. In Keras, the `LSTM` layer can simply be used instead of the `SimpleRNN` layer:

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])
```

The LSTM cell architecture is shown in the figure. Without looking what's inside the box, the LSTM cell looks exactly like a regular cell, except that its state is split into two vectors: $\mathbf{h}_{(t)}$ and $\mathbf{c}_{(t)}$ ("c" stands for "cell"). The $\mathbf{h}_{(t)}$ and $\mathbf{c}_{(t)}$ can be thought of as the short-term state and the long-term state, respectively.

The key idea is that the network can learn what to store in the long-term state, what to throw away, and what to read from it. As the long-term state $\mathbf{c}_{(t-1)}$ traverses the network from left to right, it first goes through a *forget gate*, dropping some memories, and then it adds some new memories via the addition operation (which adds the memories that were selected by an *input gate*). The result $\mathbf{c}_{(t)}$ is sent straight out, without any further transformation. So, at each time step, some memories are dropped and some memories are added. Moreover, after the addition operation, the long-term state is copied and passed through the tanh function, and then the result is filtered by the *output gate*. This produces the short-term state $\mathbf{h}_{(t)}$ (which is equal to the cell's output for this time step, $\mathbf{y}_{(t)}$). Now let's look at where new memories come from and how the gates work.



First, the current input vector $\mathbf{x}_{(t)}$ and the previous short-term state $\mathbf{h}_{(t-1)}$ are fed to four different fully connected layers. They all serve a different purpose:

- The main layer is the one that outputs $\mathbf{g}_{(t)}$. It has the usual role of analyzing the current inputs $\mathbf{x}_{(t)}$ and the previous (short-term) state $\mathbf{h}_{(t-1)}$. In a basic cell, there is nothing other than this layer, and its output goes straight out to $\mathbf{y}_{(t)}$ and $\mathbf{h}_{(t)}$. But in an LSTM cell, this layer's output does not go straight out; instead its most important parts are stored in the long-term state (and the rest is dropped).
- The three other layers are *gate controllers*. Since they use the logistic activation function, the outputs range from 0 to 1. As you can see, the gate controllers' outputs are fed to element-wise

multiplication operations: if they output 0s they close the gate, and if they output 1s they open it. Specifically:

- The *forget gate* (controlled by $\mathbf{f}_{(t)}$) controls which parts of the long-term state should be erased.
- The *input gate* (controlled by $\mathbf{i}_{(t)}$) controls which parts of $\mathbf{g}_{(t)}$ should be added to the long-term state.
- Finally, the *output gate* (controlled by $\mathbf{o}_{(t)}$) controls which parts of the long-term state should be read and output at this time step, both to $\mathbf{h}_{(t)}$ and to $\mathbf{y}_{(t)}$.

In short, an LSTM cell can learn to recognize an important input (that's the role of the input gate), store it in the long-term state, preserve it for as long as it is needed (that's the role of the forget gate), and extract it whenever it is needed. This explains why these cells have been amazingly successful at capturing long-term patterns in time series, long texts, audio recordings, and more.

LSTM Computations

The equations below summarize how to compute the cell's long-term state, its short-term state, and its output at each time step for a single instance (the equations for a whole mini-batch are very similar).

$$\mathbf{i}_{(t)} = \sigma(\mathbf{W}_{xi}^T \mathbf{x}_{(t)} + \mathbf{W}_{hi}^T \mathbf{h}_{(t-1)} + \mathbf{b}_i)$$

$$\mathbf{f}_{(t)} = \sigma(\mathbf{W}_{xf}^T \mathbf{x}_{(t)} + \mathbf{W}_{hf}^T \mathbf{h}_{(t-1)} + \mathbf{b}_f)$$

$$\mathbf{o}_{(t)} = \sigma(\mathbf{W}_{xo}^T \mathbf{x}_{(t)} + \mathbf{W}_{ho}^T \mathbf{h}_{(t-1)} + \mathbf{b}_o)$$

$$\mathbf{g}_{(t)} = \tanh(\mathbf{W}_{xg}^T \mathbf{x}_{(t)} + \mathbf{W}_{hg}^T \mathbf{h}_{(t-1)} + \mathbf{b}_g)$$

$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}$$

$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)})$$

In this equation:

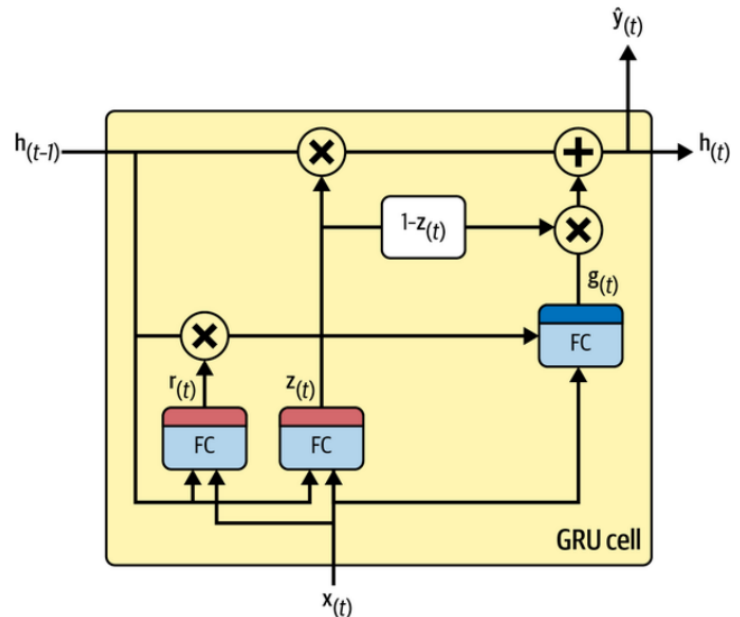
- $\mathbf{W}_{xi}$, $\mathbf{W}_{xf}$, $\mathbf{W}_{xo}$, and $\mathbf{W}_{xg}$ are the weight matrices of each of the four layers for their connection to the input vector $\mathbf{x}_{(t)}$.
- $\mathbf{W}_{hi}$, $\mathbf{W}_{hf}$, $\mathbf{W}_{ho}$, and $\mathbf{W}_{hg}$ are the weight matrices of each of the four layers for their connection to the previous short-term state $\mathbf{h}_{(t-1)}$.
- $\mathbf{b}_i$, $\mathbf{b}_f$, $\mathbf{b}_o$, and $\mathbf{b}_g$ are the bias terms for each of the four layers. TensorFlow initializes $\mathbf{b}_f$ to a vector full of 1s instead of 0s. This prevents forgetting everything at the beginning of training.

32. Identify the simplifications made into *gated recurrent unit* (GRU) over *long short-term memory* (LSTM). Show architecture of a GRU cell and provide equations for GRU computations.

Answer:

The GRU cell is a simplified version of the LSTM cell (refer figure), and it seems to perform just as well (which explains its growing popularity). These are the main simplifications:

- Both state vectors are merged into a single vector $\mathbf{h}_{(t)}$.
- A single gate controller $\mathbf{z}_{(t)}$ controls both the forget gate and the input gate. If the gate controller outputs a 1, the forget gate is open (= 1) and the input gate is closed ($1 - 1 = 0$). If it outputs a 0, the opposite happens. In other words, whenever a memory must be stored, the location where it will be stored is erased first. This is actually a frequent variant to the LSTM cell in and of itself.
- There is no output gate; the full state vector is output at every time step. However, there is a new gate controller $\mathbf{r}_{(t)}$ that controls which part of the previous state will be shown to the main layer ($\mathbf{g}_{(t)}$).



The equations below summarize how to compute the cell's state at each time step for a single instance.

$$\mathbf{z}_{(t)} = \sigma(\mathbf{W}_{xz}^T \mathbf{x}_{(t)} + \mathbf{W}_{hz}^T \mathbf{h}_{(t-1)} + \mathbf{b}_z)$$

$$\mathbf{r}_{(t)} = \sigma(\mathbf{W}_{xr}^T \mathbf{x}_{(t)} + \mathbf{W}_{hr}^T \mathbf{h}_{(t-1)} + \mathbf{b}_r)$$

$$\mathbf{g}_{(t)} = \tanh(\mathbf{W}_{xg}^T \mathbf{x}_{(t)} + \mathbf{W}_{hg}^T (\mathbf{r}_{(t)} \otimes \mathbf{h}_{(t-1)}) + \mathbf{b}_g)$$

$$\mathbf{h}_{(t)} = \mathbf{z}_{(t)} \otimes \mathbf{h}_{(t-1)} + (1 - \mathbf{z}_{(t)}) \otimes \mathbf{g}_{(t)}$$

Keras provides a `tf.keras.layers.GRU` layer: using it is just a matter of replacing `SimpleRNN` or `LSTM` with `GRU`.

33. Identify how learning long-term patterns in sequence can be extended by using 1-dimensional convolutional layer along with GRU layer in a sequential model. Provide appropriate code-snippet.

Answer:

Similar to the way, a 2D convolutional layer slides with several fairly small kernels (or filters) across an image, producing multiple 2D feature maps (one per kernel), a 1D convolutional layer slides several kernels across a sequence, producing a 1D feature map per kernel. Each kernel will learn to detect a single very short sequential pattern (no longer than the kernel size). If 10 kernels are used, then the layer's output will be composed of 10 1D sequences (all of the same length), or equivalently this output can be viewed as a single 10D sequence. This means that a neural network, composed of a mix of recurrent layers and 1D convolutional layers (or even 1D pooling layers), can be built. If a 1D convolutional layer with a stride of 1 and "same" padding is used, then the output sequence will have the same length as the input sequence. But if "valid" padding or a stride greater than 1 is used, then the output sequence will be shorter than the input sequence, so the targets are to be adjusted accordingly.

For example, the following model starts with a 1D convolutional layer that downsamples the input sequence by a factor of 2, using a stride of 2. The kernel size is larger than the stride, so all inputs will be used to compute the layer's output, and therefore the model can learn to preserve the useful information, dropping only the unimportant details. By shortening the sequences the convolutional layer may help the GRU layers detect longer patterns, so the input sequence length can be extended.

```
conv_rnn_model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=4,
                           strides=2, activation="relu", input_shape=[None, 5]),
    tf.keras.layers.GRU(32, return_sequences=True),
    tf.keras.layers.Dense(14)
])
```

It is actually possible to use only 1D convolutional layers and drop the recurrent layers entirely.

CHAPTER: Text Processing

34. Build a language model over a “stateless” character recurrent neural network (Char-RNN) to predict the next character in a sentence considering an appropriate dataset. Provide all required code-snippets and explanations.

Answer:

Data Preparation:

The following code snippets build a simple stateless language model using character CNN to predict the next character in a sequence.

```
import tensorflow as tf

# Loads text data from an input file

file_path = "C:\data\shakespeare.txt"
with open(filepath) as f:
    shakespeare_text = f.read()

# Encodes text data using a text vectorizer

text_vec_layer = tf.keras.layers.TextVectorization(split="character",
                                                    standardize="lower")
text_vec_layer.adapt([shakespeare_text])

n_tokens = text_vec_layer.vocabulary_size()      # number of distinct chars
```

Next, the following helper function prepares dataset by batching the windows of input/target pairs and then enables prefetching. It takes a sequence as input (i.e., the encoded text) and

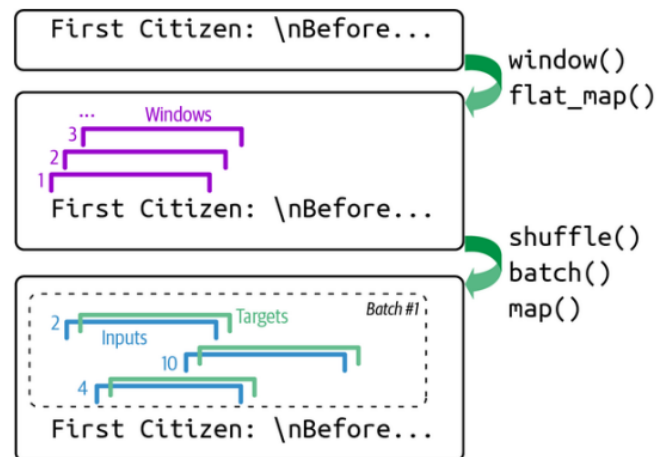
- creates a dataset containing all the windows of the desired length,
- flattens the dataset for individual window of desired length to be available in batches,
- shuffles the windows,
- batches them, and
- splits them into input/output pairs and enables prefetching.

```
def to_dataset(sequence, length, shuffle=False, seed=None, batch_size=32):
    ds = tf.data.Dataset.from_tensor_slices(sequence)
    ds = ds.window(length + 1, shift=1, drop_remainder=True)
    ds = ds.flat_map(lambda window_ds: window_ds.batch(length + 1))
    if shuffle:
        ds = ds.shuffle(buffer_size=100_000, seed=seed)
    ds = ds.batch(batch_size)

    return ds.map(lambda window: (window[:, :-1], window[:, 1:])).prefetch(1)
```

The figure on right summarizes the dataset preparation steps: it shows windows of length 11, and a batch size of 3. The start index of each window is indicated next to it.

Next, it splits encoded dataset into train, validation and test set. The window length is set to 100 as the RNN will not be able to learn any pattern longer than length, so don't make it too small. But the RNNs are easier and faster to train on shorter input sequences.



```
length = 100

tf.random.set_seed(42)
train_set = to_dataset(encoded[:1_000_000], length=length, shuffle=True,
                        seed=42)
valid_set = to_dataset(encoded[1_000_000:1_060_000], length=length)
test_set = to_dataset(encoded[1_060_000:], length=length)
```

Modeling:

Considering the dataset is reasonably large, it builds and trains a model with one GRU layer composed of 128 units instead of a simple RNN with few recurrent units.

```
model = tf.keras.Sequential([
    text_vec_layer, # Handles preprocessing
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16),
    tf.keras.layers.GRU(128, return_sequences=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])

model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])

model_ckpt = tf.keras.callbacks.ModelCheckpoint(
    "my_shakespeare_model", monitor="val_accuracy", save_best_only=True)

history = model.fit(
    train_set, validation_data=valid_set, epochs=10, callbacks=[model_ckpt])
```

Performing Predictions:

The following code snippets predict the next character in a sentence.

```
y_proba = shakespeare_model.predict(["To be or not to b"])[0, -1]
y_pred = tf.argmax(y_proba)      # Chooses the most probable character ID
text_vec_layer.get_vocabulary()[y_pred]    # Prints the predicted character
```

35. Identify the design considerations for choosing a stateful RNN over a stateless RNN for text generation task. Also, build a language model over a “stateful” character RNN to predict the next character in a sentence considering an appropriate dataset and providing all required explanations.

Answer:

Stateless RNNs:

In a stateless RNN, at each training iteration, the model starts with a hidden state full of zeros, then it updates this state at each time step, and after the last time step, it throws it away as it is not needed anymore. With this approach, the model does not learn patterns beyond 100 timesteps.

Stateful RNNs:

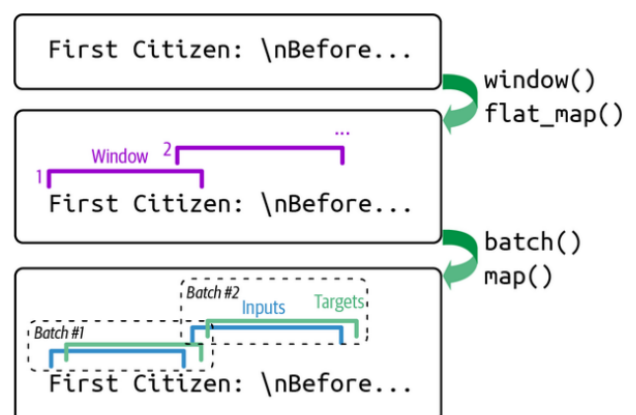
An RNN could preserve this final state after processing a training batch and use it as the initial state for the next training batch for it to learn long-term patterns despite only backpropagating through short sequences. This is called a stateful RNN.

Implication of Preparing Batches for Stateful RNN:

A stateful RNN only makes sense if each input sequence in a batch starts exactly where the corresponding sequence in the previous batch is left off. So, the first thing would be to do to build a stateful RNN is to use sequential and nonoverlapping input sequences rather than the shuffled and overlapping sequences that is generally used to train stateless RNNs. When creating the `tf.data.Dataset`, shifting must therefore be set as `shift=length` (instead of `shift=1`) when calling the `window()` method. Moreover, the `shuffle()` method must not be called.

Batch Preparation for Stateful RNN:

If 32, for example, is set as batch size by calling `batch(32)`, then 32 consecutive windows would be put in the same batch, and the following batch would not continue each of these windows where it left off. The first batch would contain windows 1 to 32 and the second batch would contain windows 33 to 64, windows in each of these batches would not be consecutive. The simplest solution to this problem is to just use a batch size of 1. After preparing the



dataset, the consecutive fragments may look like on the ones shown in the figure on right.

Building a Stateful RNN:

Stateful argument is set to True when creating each recurrent layer. Because the stateful RNN needs to know the batch size (since it will preserve a state for each input sequence in the batch), the `batch_input_shape` argument is also set in the first layer. The second dimension is left unspecified, since the input sequences could have any length.

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=n_tokens, output_dim=16,
                              batch_input_shape=[1, None]),
    tf.keras.layers.GRU(128, return_sequences=True, stateful=True),
    tf.keras.layers.Dense(n_tokens, activation="softmax")
])
```

At the end of each epoch, states are reset before it goes back to the beginning of the text. This can be implemented using a small custom Keras callback.

```
class ResetStatesCallback(tf.keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs):
        self.model.reset_states()
```

And the model can be compiled and trained using custom callback.

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])
history = model.fit(stateful_train_set, validation_data=stateful_valid_set,
                    epochs=10, callbacks=[ResetStatesCallback(), model_ckpt])
```

36. Develop code-snippets to build a recurrent neural network for sentiment classification considering an appropriate dataset and explain all the modeling steps. How can model performance be improved by applying masking on padding tokens?

Answer:

Sentiment analysis is one of the most common applications of NLP as a task. In this example, the IMDb dataset consists of 50,000 movie reviews in English (25,000 for training, 25,000 for testing) is being considered. It comes with a simple binary target for each review indicating whether it is negative (0) or positive (1).

Building RNN for Sentiment Analysis:

Data Preparation: The following code-snippet loads the IMDb dataset using the TensorFlow Datasets library. The first 90% of the training set is to be used for training, and the remaining 10% for validation.

```
import tensorflow_datasets as tfds

raw_train_set, raw_valid_set, raw_test_set = tfds.load(name="imdb_reviews",
    split=["train[:90%]", "train[90%:]", "test"], as_supervised=True )

tf.random.set_seed(42)

train_set = raw_train_set.shuffle(5000, seed=42).batch(32).prefetch(1)
valid_set = raw_valid_set.batch(32).prefetch(1)
test_set = raw_test_set.batch(32).prefetch(1)
```

Next, the input sequences are preprocessed by chopping each sequence into words (instead of characters) using `tf.keras.layers.TextVectorization` layer. Spaces are used to identify word boundaries.

```
# Limits the vocabulary to 1,000 tokens including the most frequent 998
words plus a padding token and a token for unknown words

vocab_size = 1000

text_vec_layer = tf.keras.layers.TextVectorization(max_tokens=vocab_size)
text_vec_layer.adapt(train_set.map(lambda reviews, labels: reviews))
```

Modeling: The following code-snippet creates the model and trains it.

```
embed_size = 128

tf.random.set_seed(42)

model = tf.keras.Sequential([

    text_vec_layer,

    tf.keras.layers.Embedding(vocab_size, embed_size),

    tf.keras.layers.GRU(128),

    tf.keras.layers.Dense(1, activation="sigmoid")

])

model.compile(loss="binary_crossentropy", optimizer="nadam",

    metrics=["accuracy"])

history = model.fit(train_set, validation_data=valid_set, epochs=2)
```

The first layer is the `TextVectorization` layer that will convert words into word IDs (numerical), followed by an `Embedding` layer that will convert word IDs into embeddings. The embedding matrix needs to have one row per token in the vocabulary (`vocab_size`) and one column per embedding dimension (128 in this example).

Next, a GRU layer and a Dense layer with a single neuron are used. Sigmoid activation function is used since this is a binary classification task: the model's output will be the estimated probability that the review expresses a positive sentiment regarding the movie. Then the model is compiled and fitted on the dataset prepared earlier for a couple of epochs (training for longer will yield better results).

Model Performance Analysis: But this model fails to learn anything at all: the accuracy remains close to 50%. As the reviews have different lengths, the `TextVectorization` layer converts them to sequences of token IDs and pads the shorter sequences using the padding token (with ID 0) to make them as long as the longest sequence in the batch. As a result, most sequences end with many padding tokens. So, when the GRU layer goes through many padding tokens, it ends up forgetting what the review was about. Even though a GRU layer, which is much better than a `SimpleRNN` layer, its short-term memory is still not great. One solution is to feed the model with batches of equal-length sentences (which also speeds up training). Another solution is to make the RNN ignore the padding tokens. This can be done using masking.

Applying Masking

Making the model to ignore padding tokens can be done simply by adding `mask_zero=True` when creating the `Embedding` layer. This means that padding tokens (whose ID is 0) will be ignored by all downstream layers. Retraining the previous model for a few epochs, the validation accuracy will reach over 80%.

How the Masking Works in Embedding Layer: The `Embedding` layer creates a mask tensor equal to `tf.math.not_equal(inputs, 0)`: it is a Boolean tensor with the same shape as the inputs, and it is equal to `False` anywhere the token IDs are 0, or `True` otherwise. This mask tensor is then automatically propagated by the model to the next layer. If that layer's `call()` method has a `mask` argument, then it automatically receives the mask. This allows the layer to ignore the masked time steps. For example, when a recurrent layer encounters a masked time step, it simply copies the output from the previous time step.

Propagating Mask to Subsequent Layers: Next, if the layer's `supports_masking` attribute is `True`, then the mask is automatically propagated to the next layer. It keeps propagating this way for as long as the layers have `supports_masking=True`. As an example, a recurrent layer's `supports_masking` attribute is `True` when `return_sequences=True`, but it's `False` when `return_sequences=False` since there's no need for a mask anymore in this case. So for a model with several recurrent layers with `return_sequences=True`, followed by a recurrent layer with `return_sequences=False`, then the mask will automatically propagate up to the last

recurrent layer: that layer will use the mask to ignore masked steps, but it will not propagate the mask any further. Similarly, if the Embedding layer in the sentiment analysis model is set with `mask_zero=True`, then the GRU layer will receive and use the mask automatically, but it will not propagate it any further, since `return_sequences` is not set to `True`.

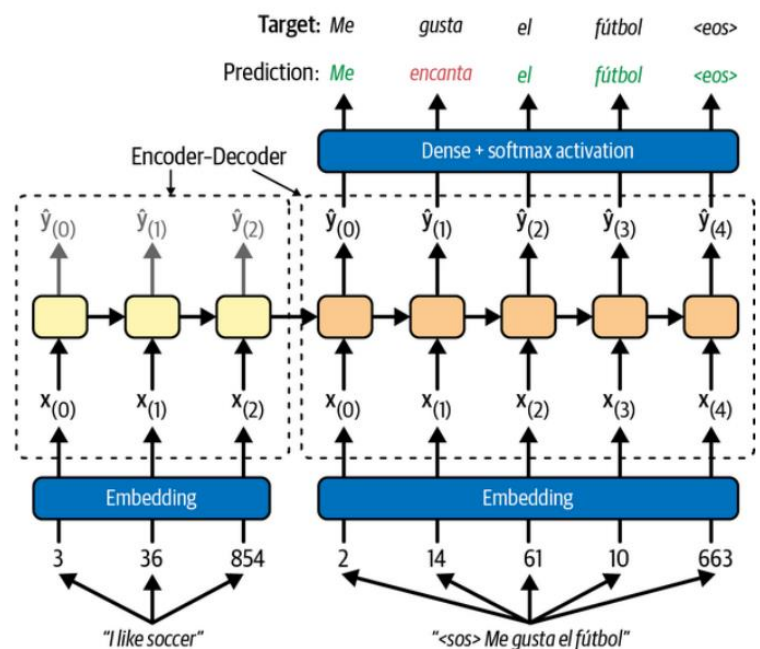
37. Build a simple machine translation model using an encoder-decoder recurrent neural network considering appropriate assumptions. Provide all design considerations, model architecture, and code-snippets.

Answer:

The following implementation builds a simple neural machine translation (NMT) model that will translate English sentences to Spanish.

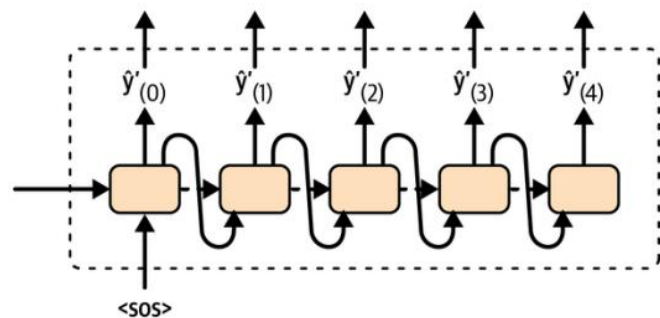
Design Considerations & The Model Architecture:

English sentences are fed as inputs to the encoder, and the decoder outputs the Spanish translations (refer figure). The Spanish translations are also used as inputs to the decoder during training but shifted back by one step. In other words, during training the decoder is given as input the word that it should have output at the previous step, regardless of what it actually outputs. This is called *teacher forcing*—a technique that significantly speeds up training and improves the model's performance. For the very first word, the decoder is given the start-of-sequence (SOS) token, and the decoder is expected to end the sentence with an end-of-sequence (EOS) token.



Each word is initially represented by its ID (e.g., 854 for the word "soccer"). Next, an Embedding layer returns the word embedding. These word embeddings are then fed to the encoder and the decoder. At each step, the decoder outputs a score for each word in the output vocabulary (i.e., Spanish), then the softmax activation function turns these scores into probabilities. The model can be trained using the "sparse_categorical_crossentropy" loss.

Due to the non-availability of target sequence during inference time, the decoder is fed with the word that it has just output at the previous step, as shown in figure on right (this will require an embedding lookup that is not shown in the diagram).



Data Preprocessing:

This example considers a text file where each line contains an English sentence and the corresponding Spanish translation, separated by a tab. First sentence pairs are parsed and shuffled and split the pairs into two separate lists - `sentences_en`, `sentences_es`, one per language.

Data Preparation:

Next, two `TextVectorization` layers—one per language—are created and adapted to the text.

```
import numpy as np

vocab_size = 1000      # Set to short value due to small dataset

# input sequence to be cropped to 50 tokens; shorter ones will be padded
# with 0s to make sequence 50 tokens long
max_length = 50

text_vec_layer_en = tf.keras.layers.TextVectorization(vocab_size,
    output_sequence_length=max_length)

text_vec_layer_es = tf.keras.layers.TextVectorization(vocab_size,
    output_sequence_length=max_length)

# Prepares a vocabulary of string terms from tokens in English
text_vec_layer_en.adapt(sentences_en)

# Prepares a vocabulary of string terms from tokens in Spanish
text_vec_layer_es.adapt([f"startofseq {s} endofseq" for s in sentences_es])
```

For the Spanish text, word “startofseq” (SOS) and “endofseq” (EOS) are added to each sentence when adapting the `TextVectorization` layer.

Next, the training set and the validation set are created. The decoder’s inputs are the Spanish sentences plus an SOS token prefix. The targets are the Spanish sentences plus an EOS suffix.

```
# For encoder

X_train = tf.constant(sentences_en[:100_000])
X_valid = tf.constant(sentences_en[100_000:])
```

```

# For decoder

X_train_dec = tf.constant([f"startofseq {s}" for s in sentences_es[:100_000]])
X_valid_dec = tf.constant([f"startofseq {s}" for s in sentences_es[100_000:]])

# Targets for the decoder

y_train = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[:100_000]])
y_valid = text_vec_layer_es([f"{s} endofseq" for s in sentences_es[100_000:]])

```

Modeling:

The model requires two text inputs—one for the encoder and one for the decoder.

```

encoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)
decoder_inputs = tf.keras.layers.Input(shape=[], dtype=tf.string)

```

Next, these sentences are encoded using the `TextVectorization` layers prepared earlier, followed by an `Embedding` layer for each language, with `mask_zero=True` to ensure masking is handled automatically.

```

embed_size = 128      # Embedding size is a hyperparameter that can be tuned

encoder_input_ids = text_vec_layer_en(encoder_inputs)
decoder_input_ids = text_vec_layer_es(decoder_inputs)

encoder_embedding_layer = tf.keras.layers.Embedding(
    vocab_size, embed_size, mask_zero=True)
decoder_embedding_layer = tf.keras.layers.Embedding(
    vocab_size, embed_size, mask_zero=True)

encoder_embeddings = encoder_embedding_layer(encoder_input_ids)
decoder_embeddings = decoder_embedding_layer(decoder_input_ids)

```

Now, the encoder is created using a single `LSTM` layer, and the embedded inputs are passed into it.

```

encoder = tf.keras.layers.LSTM(512, return_state=True)

encoder_outputs, *encoder_state = encoder(encoder_embeddings)

```

Also set `return_state=True` to get a reference to the layer's final state. `*encoder_state` groups both the states - the short-term state and the long-term state, returned separately by `LSTM` layer. This (double) state is used as the initial state of the decoder.

```

decoder = tf.keras.layers.LSTM(512, return_sequences=True)

decoder_outputs = decoder(decoder_embeddings, initial_state=encoder_state)

```

Next, the decoder's outputs are passed through a Dense layer with the softmax activation function to get the word probabilities for each step.

```
output_layer = tf.keras.layers.Dense(vocab_size, activation="softmax")

y_proba = output_layer(decoder_outputs)
```

Next, the Keras model just needs to be created, compiled and trained.

```
model = tf.keras.Model(inputs=[encoder_inputs, decoder_inputs],
                       outputs=[y_proba])

model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam",
              metrics=["accuracy"])

model.fit((X_train, X_train_dec), y_train, epochs=10,
        validation_data=((X_valid, X_valid_dec), y_valid))
```

Performing machine translation:

The following custom memory cell keeps track of the previous output and feeds it to the encoder at the next time step. The model is just called multiple times, predicting one extra word at each round.

```
def translate(sentence_en):
    translation = ""
    for word_idx in range(max_length):
        X = np.array([sentence_en])           # encoder input
        X_dec = np.array(["startofseq " + translation]) # decoder input
        y_proba = model.predict((X, X_dec))[0, word_idx] # last token's probas
        predicted_word_id = np.argmax(y_proba)
        predicted_word = text_vec_layer_es.get_vocabulary()[predicted_word_id]
        if predicted_word == "endofseq":
            break
        translation += " " + predicted_word

    return translation.strip()
```

The function simply keeps predicting one word at a time, gradually completing the translation, and it stops once it reaches the EOS token.

```
translate("I like soccer")

'me gusta el fútbol'
```

38. Make use of *Bidirectional RNNs* and *Beam Search* to improve natural language translation performance by identifying all their design considerations. Provide required figures and code-snippets.

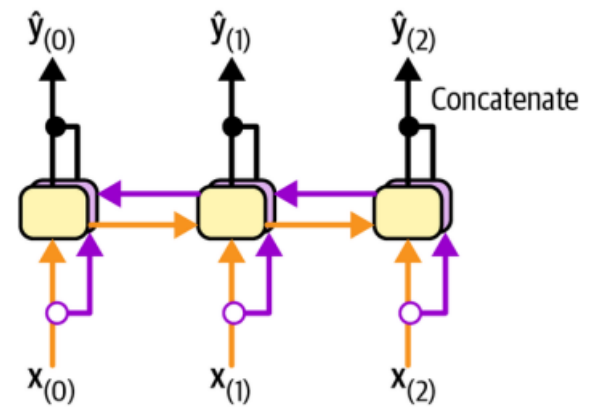
Answer:

The model's language translation performance can be improved to an extent by increasing the training set size and adding more LSTM layers in both the encoder and the decoder. But the capability

of the model can further be improved by applying sophisticated techniques such as *bidirectional recurrent layers* and *beam search*.

Bidirectional RNNs:

The Need for Bidirectional RNN: At each time step, a regular recurrent layer only looks at past and present inputs before generating its output. In other words, it is causal, meaning it cannot look into the future. This type of RNN makes sense when forecasting time series, or in the decoder of a sequence-to-sequence (seq2seq) model. But for tasks like text classification, or in the encoder of a seq2seq model, it is often preferable to look ahead at the next words before encoding a given word. For example, to properly encode the word “right” for phrases such as “the right arm”, “the right person”, and “the right to criticize”, they should be looked ahead. One solution is to run two recurrent layers on the same inputs, one reading the words from left to right and the other reading them from right to left, then combine their outputs at each time step, typically by concatenating them. This is what a bidirectional recurrent layer does (see figure).



Implementation: The following `Bidirectional` layer could be used as the encoder in our translation model. The `Bidirectional` layer will create a clone of the `GRU` layer (but in the reverse direction), and it will run both and concatenate their outputs. So, although the `GRU` layer has 10 units, the `Bidirectional` layer will output 20 values per time step.

```
encoder = tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(256,
                                                                return_state=True))
```

Concatinating LSTM Layers' Long and Short-term States: This layer will now return four states instead of two: the final short-term and long-term states of the forward LSTM layer, and the final short-term and long-term states of the backward LSTM layer. These quadruple states cannot be used directly as the initial state of the decoder's LSTM layer since it expects just two states (short-term and long-term). The decoder cannot also be made bidirectional, since it must remain causal. Instead, the two short-term and long-term states are concatenated separately.

```
encoder_outputs, *encoder_state = encoder(encoder_embeddings)

encoder_state = [tf.concat(encoder_state[:2], axis=-1), # short-term (0 & 2)
                 tf.concat(encoder_state[1:2], axis=-1)] # long-term (1 & 3)
```

Beam Search:

Another popular technique that can greatly improve the performance of a translation model at inference time is beam search. This technique gives an encoder–decoder model a chance to go back and fix mistakes it made earlier.

Beam Width: It keeps track of a short list of the k most promising sentences, and at each decoder step it tries to extend them by one word, keeping only the k most likely sentences. The parameter k is called the beam width.

Conditional

Probabilities: For example, suppose the model needs to translate the sentence “I like soccer” using beam search with a beam width of 3 (see figure). At the first decoder step, the



model will output an estimated probability for each possible first word in the translated sentence. Suppose the top three shortlisted words are “me” (75% estimated probability), “a” (3%), and “como” (1%). Next, the model finds conditional probabilities for the next word for each sentence. For the first sentence (“me”), perhaps the model outputs a probability of 36% for the word “gustan”, 32% for the word “gusta”, 16% for the word “encanta”, and so on. For the second sentence (“a”), the model might output a conditional probability of 50% for the word “mi”, and so on. Assuming the vocabulary has 1,000 words, we will end up with 1,000 probabilities per sentence.

Next, the probabilities of each of the 3,000 two-word sentences we considered ($3 \times 1,000$) are computed by multiplying the estimated conditional probability of each word by the estimated probability of the sentence it completes. After that only the top 3 probabilities are kept. In this example they all start with the word “me”: “me gustan” (27%), “me gusta” (24%), and “me encanta” (12%). The sentences with the highest conditional probabilities are kept dropping the other ones with relatively lower probabilities.

Then the same process is repeated: the model predicts the next word in each of these three sentences, and the probabilities of all 3,000 three-word sentences we considered are computed. Perhaps the top three are now “me gustan los” (10%), “me gusta el” (8%), and “me gusta mucho” (2%). At the next step we may get “me gusta el fútbol” (6%), “me gusta mucho el” (1%), and “me gusta el deporte” (0.2%). Notice that “me gustan” was eliminated, and the correct translation is now ahead. In this way, the encoder–decoder model’s performance can be boosted without any extra training, simply by using it more wisely.

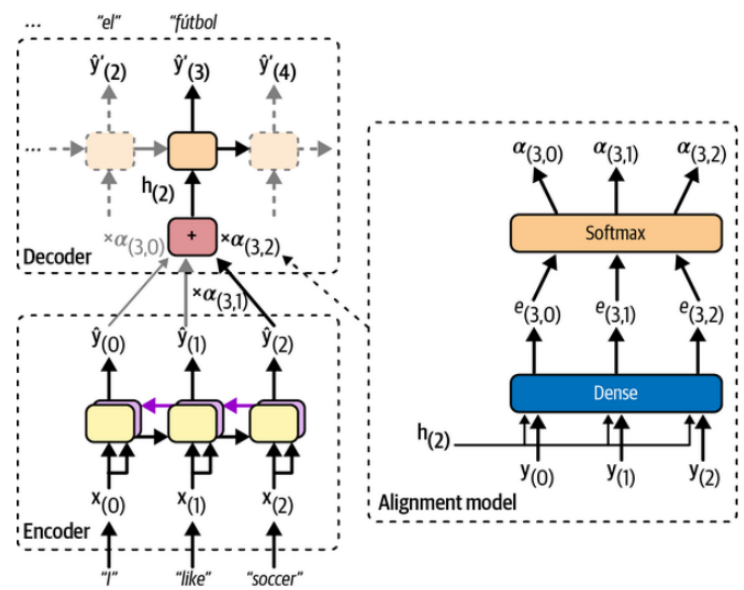
39. Identify how *Bahdanau attention mechanism* alleviates the short-term memory limitation of RNNs. Provide a figure and equation for this attention mechanism.

Answer:

Bahdanau attention mechanism is a technique that allows the decoder to focus on the appropriate words (as encoded by the encoder) at each time step. For example, at the time step where the decoder needs to output the word “fútbol” in a neural machine translation scenario where, for example, a English sentence “I like soccer” gets translated into a French sentence “me gusta el fútbol”, it will focus its attention on the word “soccer” shortening the path from an input word to its translation in beam search resulting less short-term memory impact for RNNs. Attention mechanisms revolutionized neural machine translation (and deep learning in general), allowing a significant improvement in the state of the art, especially for long sentences (e.g., over 30 words).

The figure on the right shows the encoder–decoder model with an added attention mechanism. The encoder and the decoder are on the left. Instead of just sending the encoder’s final hidden state to the decoder, as well as the previous target word at each step (which is still done, although it is not shown in the figure), now all of the encoder’s outputs are sent to the decoder as well. Since the decoder cannot deal with all these encoder outputs at once, they need to be aggregated: at each time step, the decoder’s memory cell computes a weighted sum of all the encoder outputs.

This determines which words it will focus on at this step. The weight $\alpha_{(t, i)}$ is the weight of the i^{th} encoder output at the t^{th} decoder time step. For example, if the weight $\alpha_{(3, 2)}$ is much larger than the weights $\alpha_{(3, 0)}$ and $\alpha_{(3, 1)}$, then the decoder will pay much more attention to the encoder’s output for word #2 (“soccer”) than to the other two outputs, at least at this time step. The rest of the decoder



works just like earlier: at each time step the memory cell receives the inputs, plus the hidden state from the previous time step, and finally (although it is not represented in the diagram) it receives the target word from the previous time step (or at inference time, the output from the previous time step).

The $\alpha_{(t,i)}$ weights are generated by a small neural network called an *alignment model* (or an attention layer), which is trained jointly with the rest of the encoder–decoder model. This alignment model is illustrated on the right-hand side of the figure. It starts with a `Dense` layer composed of a single neuron that processes each of the encoder’s outputs, along with the decoder’s previous hidden state (e.g., $\mathbf{h}_{(2)}$). This layer outputs a score (or energy) for each encoder output (e.g., $e_{(3,2)}$): this score measures how well each output is aligned with the decoder’s previous hidden state. For example, in figure, the model has already output “me gusta el” (meaning “I like”), so it’s now expecting a noun: the word “soccer” is the one that best aligns with the current state, so it gets a high score. Finally, all the scores go through a softmax layer to get a final weight for each encoder output (e.g., $\alpha_{(3,2)}$). All the weights for a given decoder time step add up to 1. This particular attention mechanism is called *Bahdanau attention*. Since it concatenates the encoder output with the decoder’s previous hidden state, it is sometimes called *concatenative attention* (or *additive attention*). The equations for concatenative mechanism along with two other popular attention mechanisms, namely *dot product mechanism* and “general” *dot product mechanism*, are summarized in equation below.

$$\tilde{\mathbf{h}}_{(t)} = \sum_i \alpha_{(t,i)} \mathbf{y}_{(i)} \text{ with } \alpha_{(t,i)} = \frac{\exp(e_{(t,i)})}{\sum_{i'} \exp(e_{(t,i')})} \text{ and } e_{(t,i)} = \begin{cases} \mathbf{h}_{(t)}^\top \mathbf{y}_{(i)} & \text{dot} \\ \mathbf{h}_{(t)}^\top \mathbf{W} \mathbf{y}_{(i)} & \text{general} \\ \mathbf{v}^\top \tanh(\mathbf{W} [\mathbf{h}_{(t)}; \mathbf{y}_{(i)}]) & \text{concat} \end{cases}$$

So, the attention layer provides a way to focus the attention of the model on part of the inputs.

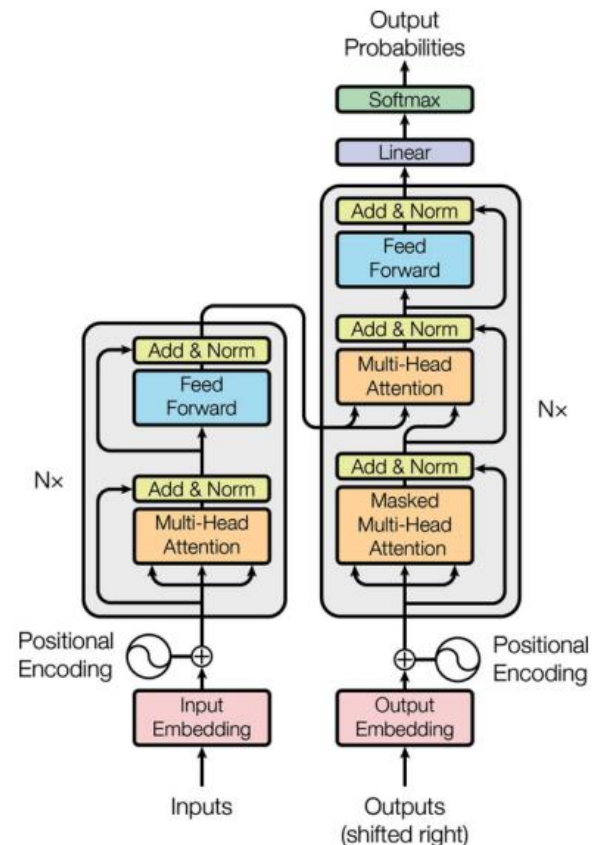
40. Construct a neat diagram of an original *transformer* architecture [2017] and identify all its design aspects and working principles in context of neural machine translation.

Answer:

The *transformer* architecture, which significantly improved the state-of-the-art in neural machine translation, does not use any recurrent or convolutional layers, rather it uses just attention mechanisms (plus embedding layers, dense layers, normalization layers, and a few other bits and pieces). Because the model is not recurrent, it doesn't suffer as much from the vanishing or exploding gradients problems as RNNs, it can be trained in fewer steps, it's easier to parallelize across multiple GPUs, and it can better capture long-range patterns than RNNs. The original 2017 transformer architecture is represented in the figure.

The left part of the figure is the encoder, and the right part is the decoder. Each embedding layer outputs a 3D tensor of shape [batch size, sequence length, embedding size]. After that, the tensors are gradually transformed as they flow through the transformer, but their shape remains the same.

For neural machine translation using the transformer, the encoder is to be fed with English sentences, and the decoder is to be fed with the corresponding Spanish translations during the training, with an extra SOS token inserted at the start of each sentence. At inference time, the transformer must be called multiple times, producing the translations one word at a time and feeding the partial translations to the decoder at each round.



Capturing Context Specific Meaning of Words in a Sentence: The encoder's role is to gradually transform the inputs—word representations of the English sentence—until each word's representation perfectly captures the meaning of the word, in the context of the sentence. For example, if the encoder is fed with the sentence "I like soccer", then the word "like" will start off with a rather vague representation, since this word could mean different things in different contexts. But after going through the encoder, the word's representation should capture the correct meaning of "like" in the given sentence (i.e., to be fond of), as well as any other information that may be required for translation (e.g., it's a verb).

The decoder's role is to gradually transform each word representation in the translated sentence into a word representation of the next word in the translation. For example, if the sentence to translate is "I like soccer", and the decoder's input sentence is "<SOS> me gusta el fútbol", then after going through the decoder, the word representation of the word "el" will end up transformed into a

representation of the word “fútbol”. Similarly, the representation of the word “fútbol” will be transformed into a representation of the EOS token.

After going through the decoder, each word representation goes through a final `Dense` layer with a softmax activation function, which will hopefully output a high probability for the correct next word and a low probability for all other words. The predicted sentence should be “me gusta el fútbol <EOS>”.

The Architecture:

- Both the encoder and the decoder contain modules that are stacked N times ($N = 6$ in original architecture). The final outputs of the whole encoder stack are fed to the decoder at each of these N levels.
- There are two embedding layers; several skip connections, each of them followed by a layer normalization layer; several feedforward modules that are composed of two dense layers each (the first one using the ReLU activation function, the second with no activation function); and finally the output layer is a dense layer using the softmax activation function. A bit of dropouts can also be added after the attention layers and the feedforward modules, if needed. Since all of these layers are time-distributed, each word is treated independently from all the others. But to translate a sentence by looking at the words sequentially, new components work as follows:
 - The encoder’s *multi-head attention* layer updates each word representation by attending to (i.e., paying attention to) all other words in the same sentence. That’s where the vague representation of the word “like” becomes a richer and more accurate representation, capturing its precise meaning in the given sentence.
 - The decoder’s *masked multi-head attention* layer does the same thing, but when it processes a word, it doesn’t attend to words located after it: it’s a causal layer. For example, when it processes the word “gusta”, it only attends to the words “<SOS> me gusta”, and it ignores the words “el fútbol” (or else that would be cheating).
 - The decoder’s upper *multi-head attention* layer is where the decoder pays attention to the words in the English sentence. This is called *cross-attention*, not self-attention in this case. For example, the decoder will probably pay close attention to the word “soccer” when it processes the word “el” and transforms its representation into a representation of the word “fútbol”.
 - The *positional encodings* are dense vectors (much like word embeddings) that represent the position of each word in the sentence. The n^{th} positional encoding is added to the word embedding of the n^{th} word in each sentence. This is needed because all layers in the transformer architecture ignore word positions: without positional encodings, the input sequences could be shuffled, and it would just shuffle the output sequences in the same way. Obviously, the order of words matters, which is why we need to give positional information to

the transformer somehow: adding positional encodings to the word representations is a good way to achieve this.

41. Identify how positional encodings inject word order information into *Transformer* models. Provide its equation and illustrative visual representation to support the answer.

Answer:

A positional encoding is a dense vector that encodes the position of a word within a sentence: the i^{th} positional encoding is added to the word embedding of the i word in the sentence. The easiest way to implement this is to use an `Embedding` layer and make it encode all the positions from 0 to the maximum sequence length in the batch, then add the result to the word embeddings. The rules of broadcasting will ensure that the positional encodings get applied to every input sequence. For example, here is how to add positional encodings to the encoder and decoder inputs:

```
max_length = 50 # max length in the whole training set
embed_size = 128

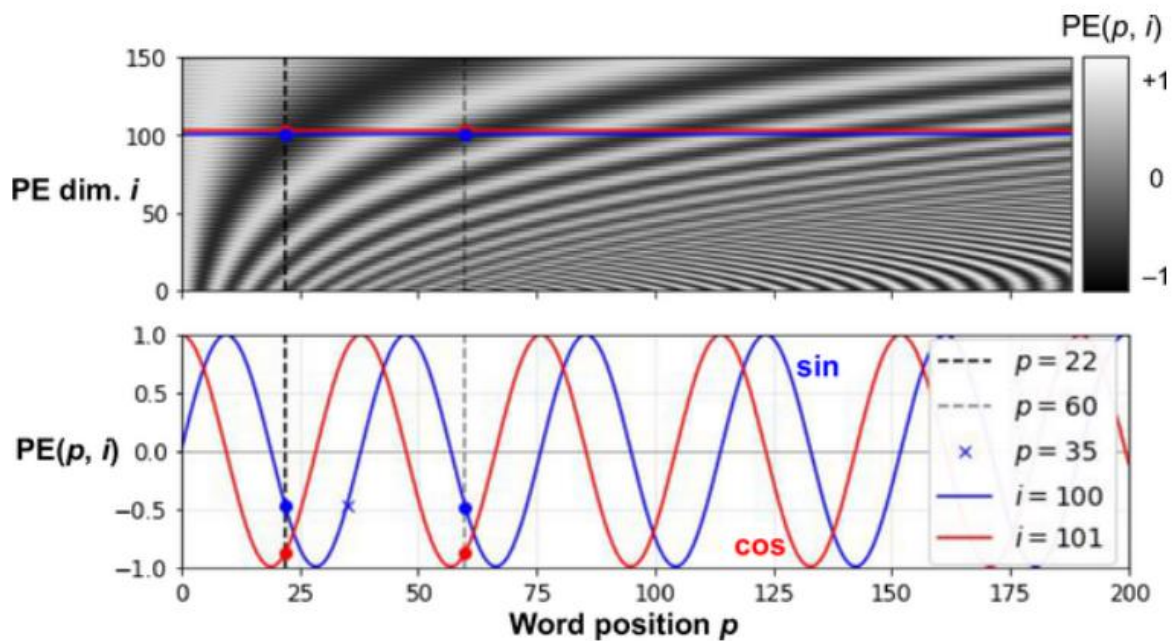
pos_embed_layer = tf.keras.layers.Embedding(max_length, embed_size)
batch_max_len_enc = tf.shape(encoder_embeddings)[1]
encoder_in = encoder_embeddings + pos_embed_layer(tf.range(batch_max_len_enc))

batch_max_len_dec = tf.shape(decoder_embeddings)[1]
decoder_in = decoder_embeddings + pos_embed_layer(tf.range(batch_max_len_dec))
```

Note that this implementation assumes that the embeddings are represented as regular tensors, not ragged tensors. The encoder and the decoder share the same `Embedding` layer for the positional encodings, since they have the same embedding size (this is often the case).

Trainable vs. fixed positional encodings: Instead of using trainable positional encodings, the authors of the transformer paper chose to use fixed positional encodings, based on the sine and cosine functions at different frequencies. The positional encoding matrix $\mathbf{P}$ is defined in equation below and represented at the top of provided figure (transposed), where $P_{p,i}$ is the i^{th} component of the encoding for the word located at the p position in the sentence. This solution can give the same performance as trainable positional encodings, and it can extend to arbitrarily long sentences without adding any parameters to the model (however, when there is a large amount of pretraining data, trainable positional encodings are usually favored).

$$P_{p,i} = \begin{cases} \sin\left(p/10000^{i/d}\right) & \text{if } i \text{ is even} \\ \cos\left(p/10000^{(i-1)/d}\right) & \text{if } i \text{ is odd} \end{cases}$$



Reason for using sine and cosine function: After these positional encodings are added to the word embeddings, the rest of the model has access to the absolute position of each word in the sentence because there is a unique positional encoding for each position (e.g., the positional encoding for the word located at the 22nd position in a sentence is represented by the vertical dashed line at the top left of the above figure, and that it is unique to that position). Moreover, the choice of oscillating functions (sine and cosine) makes it possible for the model to learn relative positions as well. For example, words located 38 words apart (e.g., at positions $p = 22$ and $p = 60$) always have the same positional encoding values in the encoding dimensions $i = 100$ and $i = 101$, as can be seen in the figure. This explains why we need both the sine and the cosine for each frequency: if we only used the sine (the blue wave at $i = 100$), the model would not be able to distinguish positions $p = 22$ and $p = 35$ (marked by a cross).

42. Make use of *scaled dot-product attention layer* (provide equation) and develop intuition how *multi-head attention* works also showing its architecture diagram.

Answer:

Scaled dot-product attention layer: Its equation is shown below, in a vectorized form. It's the same as Luong attention, except for a scaling factor.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{keys}}}} \right) \mathbf{V}$$

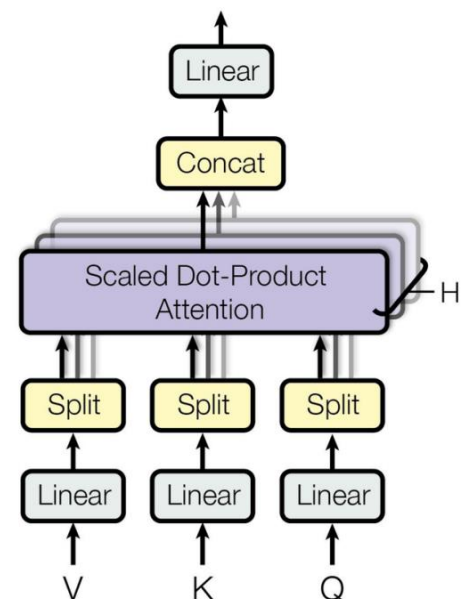
In this equation:

- **Q** is a matrix containing one row per query. Its shape is $[n_{\text{queries}}, d_{\text{keys}}]$, where n_{queries} is the number of queries and d_{keys} is the number of dimensions of each query and each key.
- **K** is a matrix containing one row per key. Its shape is $[n_{\text{keys}}, d_{\text{keys}}]$, where n_{keys} is the number of keys and values.
- **V** is a matrix containing one row per value. Its shape is $[n_{\text{keys}}, d_{\text{values}}]$, where d_{values} is the number of dimensions of each value.
- The shape of **Q K^T** is $[n_{\text{queries}}, n_{\text{keys}}]$: it contains one similarity score for each query/key pair. To prevent this matrix from being huge, the input sequences must not be too long. The output of the softmax function has the same shape, but all rows sum up to 1. The final output has a shape of $[n_{\text{queries}}, d_{\text{values}}]$: there is one row per query, where each row represents the query result (a weighted sum of the values).
- The scaling factor $1 / (\sqrt{d_{\text{keys}}})$ scales down the similarity scores to avoid saturating the softmax function, which would lead to tiny gradients. It is possible to mask out some key/value pairs by adding a very large negative value to the corresponding similarity scores, just before computing the softmax. This is useful in the masked multi-head attention layer.

Multi-head attention: Its architecture is shown in the figure below. It is just a bunch of scaled dot-product attention layers, each preceded by a linear transformation of the values, keys, and queries (i.e., a time-distributed dense layer with no activation function). All the outputs are simply concatenated, and they go through a final linear transformation (again, time-distributed).

Consider once again the word “like” in the sentence “I like soccer”. The encoder was smart enough to encode the fact that it is a verb. But the word representation also includes its position in the text, thanks to the positional encodings, and it probably includes many other features that are useful for its translation, such as the fact that it is in the present tense. In short, the word representation encodes many different characteristics of the word. If we just used a single scaled dot-product attention layer, we would only be able to query all of these characteristics in one shot.

This is why the multi-head attention layer applies multiple different linear transformations of the values, keys, and queries. This allows the model to apply many different projections of the word representation into different subspaces, each focusing on a subset of the word’s characteristics. Perhaps one of the linear layers will project the word representation into a subspace where all that remains is the information that the word is a verb, another linear layer will extract just the fact that it is present tense, and so on. Then the scaled dot-product attention layers implement



the lookup phase, and finally we concatenate all the results and project them back to the original space.

43. Develop code-snippet to show the working of the following using Hugging Face's Transformers library over an appropriate task.

- Default inference pipeline with a specific model
- Inference using Auto-classes for both tokenization and model for more control
- Model fine-tuning

Answer:

Hugging Face builds and maintains a whole ecosystem of easy-to-use open source tools for NLP, computer vision, and beyond. The central component of their ecosystem is the *Transformers* library, which allows easy download of a pretrained model, including its corresponding tokenizer, and then fine-tuning it on a custom dataset, if needed.

Default Inference Pipeline with a Specific Model

The simplest way to use the Transformers library is to use the `transformers.pipeline()` function: the task such as sentiment analysis, must be specified, and it then downloads a default pretrained model, ready to be used.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")      # many other tasks are available

result = classifier("The actors were very convincing".)

>>> result      # The result is a Python list containing one dictionary per input text
[{'label': 'POSITIVE', 'score': 0.9998071789741516}]
```

Inference using Auto-Classes for Both Tokenization and Model for More Control

Though the pipeline API is very simple and convenient, sometimes more control is required. For such cases, the Transformers library provides many classes, including all sorts of tokenizers, models, configurations, callbacks, and much more. In the following example, let loads the `DistilBERT` model, along with its corresponding tokenizer, using the `TFAutoModelForSequenceClassification` and `AutoTokenizer` classes:

```
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

Tokenizing text input: Next, a couple of pairs of sentences are tokenized. In this code, padding is activated and it is specified that TensorFlow tensors, instead of Python lists, are expected:

```
token_ids = tokenizer(["I like soccer. [SEP] We all love soccer!",
                      "Joe lived for a very long time. [SEP] Joe is old."],
                      padding=True, return_tensors="tf")

# The output is a dictionary-like instance of the BatchEncoding class, which contains the sequences of token IDs, as
# well as a mask containing 0s for the padding tokens.

>>> token_ids
{'input_ids': <tf.Tensor: shape=(2, 15), dtype=int32, numpy= array(
  [[101, 1045, 2066, 4715, 1012, 102, 2057, 2035, 2293, 4715, 999, 102, 0, 0, 0],
  [101, 3533, 2973, 2005, 1037, 2200, 2146, 2051, 1012, 102, 3533, 2003, 2214, 1012, 102]]],
 dtype=int32)>,
 'attention_mask': <tf.Tensor: shape=(2, 15), dtype=int32, numpy= array(
  [[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0],
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]], dtype=int32)>}
```

Prediction over tokenized inputs: Next, this BatchEncoding object is passed directly to the model; it returns a TFSequenceClassifierOutput object containing its predicted class logits:

```
>>> outputs = model(token_ids)
>>> outputs
TFSequenceClassifierOutput(loss=None, logits=[<tf.Tensor: [...] numpy=
array([[ -2.1123817 ,  1.1786783 ,  1.4101017 ],
       [-0.01478387,  1.0962474 , -0.9919954 ]], dtype=float32)>], [...])
```

Lastly, the softmax activation function can be applied to convert these logits to class probabilities, and then using the `argmax()` function to predict the class with the highest probability for each input sentence pair:

```
Y_probas = tf.keras.activations.softmax(outputs.logits)
>>> Y_probas
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.01619702, 0.43523544, 0.5485676 ],
       [0.08672056, 0.85204804, 0.06123142]], dtype=float32)>

Y_pred = tf.argmax(Y_probas, axis=1)
>>> Y_pred
# 0 = contradiction, 1 = entailment, 2 = neutral
<tf.Tensor: shape=(2,), dtype=int64, numpy=array([2, 1])>
```

In this example, the model correctly classifies the first sentence pair as neutral and the second pair as an entailment.

Model Fine-Tuning

If this model needs to be fine-tuned on additional dataset, it can be trained as usual with Keras since it's just a regular Keras model with a few extra methods. However, because the model outputs logits instead of probabilities, function

`tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)` loss must be used instead of the usual `sparse_categorical_crossentropy` loss. Moreover, the model does not support `BatchEncoding` inputs during training, so its `data` attribute must be used to get a regular dictionary instead:

```
sentences = [("Sky is blue", "Sky is red"), ("I love her", "She loves me")]
X_train = tokenizer(sentences, padding=True, return_tensors="tf").data
y_train = tf.constant([0, 2]) # contradiction, neutral
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(loss=loss, optimizer="nadam", metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=2)
```

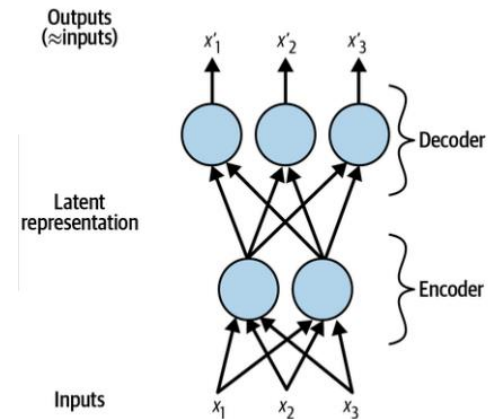
CHAPTER: Representation Learning using Autoencoders

44. Explain the general architecture of an autoencoder outlining the design and working principles. Also build a simple undercomplete linear autoencoder over appropriate assumptions.

Answer:

General Architecture of an Autoencoder

Parts of a simple autoencoder: An autoencoder looks at the inputs, converts them to an efficient *latent representation*, and then spits out something that looks very close to the inputs. An autoencoder is always composed of two parts: an *encoder* (or recognition network) that converts the inputs to a latent representation, followed by a *decoder* (or generative network) that converts the internal representation to the outputs (see figure).



An autoencoder typically has the same architecture as a multilayer perceptron (MLP), except that the number of neurons in the output layer must be equal to the number of inputs. In this example, there is just one hidden layer composed of two neurons (the encoder), and one output layer composed of three neurons (the decoder). The outputs are often called the reconstructions because the autoencoder tries to reconstruct the inputs. The cost function contains a reconstruction loss that penalizes the model when the reconstructions are different from the inputs.

Undercomplete Autoencoders: Because the internal representation has a lower dimensionality than the input data (it is 2D instead of 3D), the autoencoder is said to be *undercomplete*. An undercomplete autoencoder cannot trivially copy its inputs to the codings, yet it must find a way to output a copy of its inputs. It is forced to learn the most important features in the input data (and drop the unimportant ones).

Performing PCA with an Undercomplete Linear Autoencoder

If the autoencoder uses only linear activations and the cost function is the mean squared error (MSE), then it ends up performing principal component analysis. The following code builds a simple linear autoencoder to perform PCA on a 3D dataset, projecting it to 2D:

```
import tensorflow as tf
encoder = tf.keras.Sequential([tf.keras.layers.Dense(2)]) # With no activation
decoder = tf.keras.Sequential([tf.keras.layers.Dense(3)]) # With no activation
```



```

autoencoder = tf.keras.Sequential([encoder, decoder])
optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)
autoencoder.compile(loss="mse", optimizer=optimizer)

X_train = [...]      # generated a 3D dataset

# Trains the model on the simple generated 3D dataset
history = autoencoder.fit(X_train, X_train, epochs=500) # X_train is used as both the
inputs and the targets
codings = encoder.predict(X_train) # performs dimensionality reduction [3D -> 2D]

```

45. With a neat block diagram, explain the general architecture of a stacked autoencoder outlining the design considerations and working principles. Also, develop code-snippet to implement it with appropriate assumptions and explanations.

Answer:

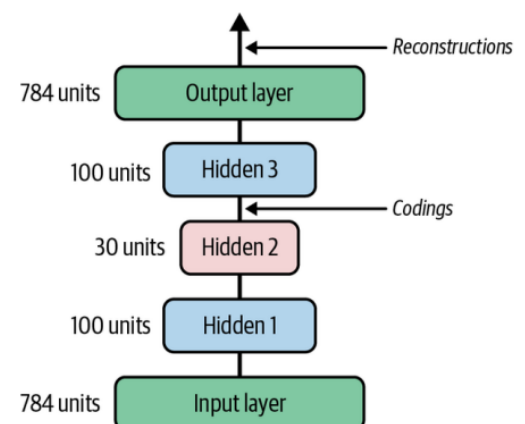
Design Considerations and Working Principles

Autoencoders with Multiple Hidden Layers: Autoencoders can have multiple hidden layers. In this case, they are called *stacked autoencoders* (or deep autoencoders). Adding more layers helps the autoencoder learn more complex codings. That said, one must be careful not to make the autoencoder too powerful. Imagine an encoder so powerful that it just learns to map each input to a single arbitrary number (and the decoder learns the reverse mapping). Obviously such an autoencoder will reconstruct the training data perfectly, but it will not have learned any useful data representation in the process, and it is unlikely to generalize well to new instances.

Architecture of Stacked Autoencoder: The architecture of a stacked autoencoder is typically symmetrical with regard to the central hidden layer (the coding layer). To put it simply, it looks like a sandwich. For example, an autoencoder for Fashion MNIST may have 784 inputs, followed by a hidden layer with 100 neurons, then a central hidden layer of 30 neurons, then another hidden layer with 100 neurons, and an output layer with 784 neurons. This stacked autoencoder is represented in the figure.

Implementing a Stacked Autoencoder Using Keras

A stacked autoencoder can be implemented very much like a regular deep MLP:



```

stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),

    tf.keras.layers.Dense(30, activation="relu"),
])

stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28]) # Outputs 2D to match encoder input
])

stacked_ae = tf.keras.Sequential([stacked_encoder, stacked_decoder])
stacked_ae.compile(loss="mse", optimizer="nadam") history =

stacked_ae.fit(X_train, X_train, epochs=20,
               validation_data=(X_valid, X_valid))

```

Explanation:

- The autoencoder model is split into two sub-models: the encoder and the decoder.
- The encoder takes 2D input with 28×28 , flattens them so that each 2D input is represented as a vector of size 784, then processes these vectors through two Dense layers of diminishing sizes (100 units then 30 units), both using the ReLU activation function. For each input, the encoder outputs a vector of size 30.
- The decoder takes codings of size 30 (output by the encoder) and processes them through two Dense layers of increasing sizes (100 units then 784 units), and it reshapes the final vectors into 28×28 arrays so the decoder's outputs have the same shape as the encoder's inputs.
- MSE loss and Nadam optimization were used when compiling the stacked autoencoder.
- Finally, the model is trained using `X_train` as both the inputs and the targets. Similarly, the model is validated using `X_valid` as both the validation inputs and targets.

46. Make use of a stacked autoencoder to train the model in an unsupervised way over a large dataset that is mostly unlabeled. Also, discuss autoencoder training techniques “weight tying” and “greedy layer-wise training”. Provide appropriate visual representations wherever required.

Answer:

Reusing stacked autoencoder's lower layers in unsupervised way over large dataset that is mostly unlabeled

For a large dataset that is mostly unlabeled, a stacked autoencoder can first be trained using all the data, then its lower layers can be reused to create a neural network for the actual task and training it using the labeled data. For example, the figure shows how to use a stacked autoencoder to perform unsupervised pretraining for a classification neural network. When training the classifier, the pretrained lower layers can be freezed when there is not much labeled training data.

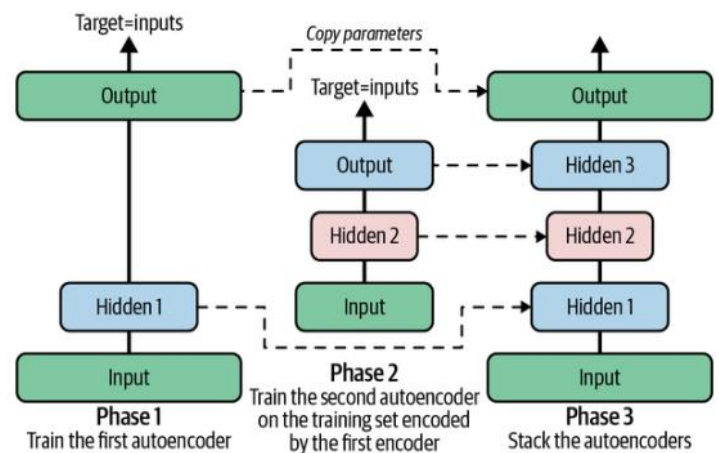
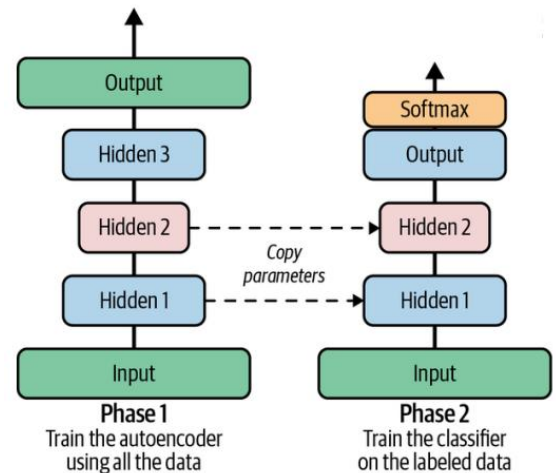
Weight Tying

When an autoencoder is neatly symmetrical, a common technique is to tie the weights of the decoder layers to the weights of the encoder layers. This halves the number of weights in the model, speeding up training and limiting the risk of overfitting. Specifically, if the autoencoder has a total of N layers (not counting the input layer), and $\mathbf{W}_L$ represents the connection weights of the L^{th} layer (e.g., layer 1 is the first hidden layer, layer $N/2$ is the coding layer, and layer N is the output layer), then the decoder layer weights can be defined as $\mathbf{W}_L = \mathbf{W}_{(N-L+1)}^T$ (with $L = N/2 + 1, \dots, N$).

Greedy Layer-wise Training

Rather than training the whole stacked autoencoder in one go, it is possible to train one shallow autoencoder at a time, then stack all of them into a single stacked autoencoder (hence the name), as shown in figure. This technique is called “greedy layer-wise training” and is not used so much these days.

Phase-wise Training: During the first phase of training, the first autoencoder learns to reconstruct the inputs. Then we encode the whole training set using this first autoencoder, and this gives us a new (compressed) training set. We then train a second autoencoder on this new dataset. This is the second phase of training. Finally, we build a big sandwich using all these autoencoders, as shown in the figure (i.e., we first stack the hidden layers of each autoencoder, then the output layers in reverse order). This gives us the final stacked autoencoder. We could easily train more autoencoders this way, building a very deep stacked autoencoder.



47. Explain the working of *convolutional autoencoder* and *denoising autoencoders*. Develop code-snippets to implement each of these.

Answer:

Convolutional Autoencoders

Convolutional autoencoders are autoencoders for images (e.g., for unsupervised pretraining or dimensionality reduction). The encoder is a regular CNN composed of convolutional layers and pooling layers. It typically reduces the spatial dimensionality of the inputs (i.e., height and width) while increasing the depth (i.e., the number of feature maps). The decoder must do the reverse (upscale the image and reduce its depth back to the original dimensions), and for this transpose convolutional layers can be used.

The following code-snippets builds a basic convolutional autoencoder for Fashion MNIST:

```
conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),           # output: 14 x 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),           # output: 7 x 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),           # output: 3 x 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAvgPool2D()                 # output: 30
])

conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same", activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"),
    tf.keras.layers.Reshape([28, 28])
])

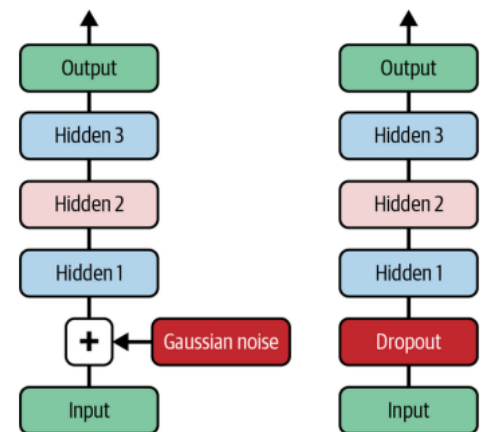
conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder]) # combines encoder and decoder
```

Denoising Autoencoders

An autoencoder can also be used by forcing it to learn useful features from the input added with noise during the training to recover the original, noise-free inputs.

Option for Noise: The noise can be pure Gaussian noise added to the inputs, or it can be randomly switched-off inputs, just like in dropout. The figure shows both the options.

Implementation: The implementation is straightforward. it is a regular stacked autoencoder with an additional Dropout layer applied to the encoder's inputs (or a GaussianNoise layer could also be used instead). It is to be noted that the Dropout layer is only active during training (and so is the GaussianNoise layer):



```
dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu")
])

dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])

# combines encoder and decoder
dropout_ae = tf.keras.Sequential([dropout_encoder, dropout_decoder])
```

The autoencoder guesses details that are actually not in the input. Not only can denoising autoencoders be used for data visualization or unsupervised pretraining, like the other autoencoders, but they can also be used quite simply and efficiently to remove noise from images.

48. Develop code-snippets to build a *sparse autoencoder* using both standard ℓ_1/ℓ_2 regularization and *Kullback-Leibler divergence* and explain the working of each of these approaches. Provide sparsity loss plot for ℓ_1 , ℓ_2 and Kullback-Leibler divergence.

Answer:

Sparsity: Another kind of constraint that often leads to good feature extraction is *sparsity*: by adding an appropriate term to the cost function, the autoencoder is pushed to reduce the number of active neurons in the coding layer. For example, it may be pushed to have, on average, only 5% of significantly active neurons in the coding layer. This forces the autoencoder to represent each input as a combination of a small number of activations. As a result, each neuron in the coding layer typically ends up representing a useful feature.

The Approach #1:

A simple approach is to use the sigmoid activation function in the coding layer (to constrain the codings to values between 0 and 1), use a large coding layer (e.g., with 300 units), and add some ℓ_1 regularization to the coding layer's activations. The decoder is just a regular decoder:

```
sparse_l1_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid"),
    tf.keras.layers.ActivityRegularization(l1=1e-4)
])

sparse_l1_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])

sparse_l1_ae = tf.keras.Sequential([sparse_l1_encoder, sparse_l1_decoder])
```

This `ActivityRegularization` layer just returns its inputs, but as a side effect it adds a training loss equal to the sum of the absolute values of its inputs. This only affects training. This penalty will encourage the neural network to produce codings close to 0, but since it will also be penalized if it does not reconstruct the inputs correctly, it will have to output at least a few nonzero values. Using the ℓ_1 norm rather than the ℓ_2 norm will push the neural network to preserve the most important codings while eliminating the ones that are not needed for the input image (rather than just reducing all codings).

The Approach #2:

Sparsity: Another approach, which often yields better results, is to measure the actual sparsity of the coding layer at each training iteration and penalize the model when the measured sparsity differs from a target sparsity. This is done by computing the average activation of each neuron in the coding layer, over the whole training batch. The batch size must not be too small, or else the mean will not be accurate.

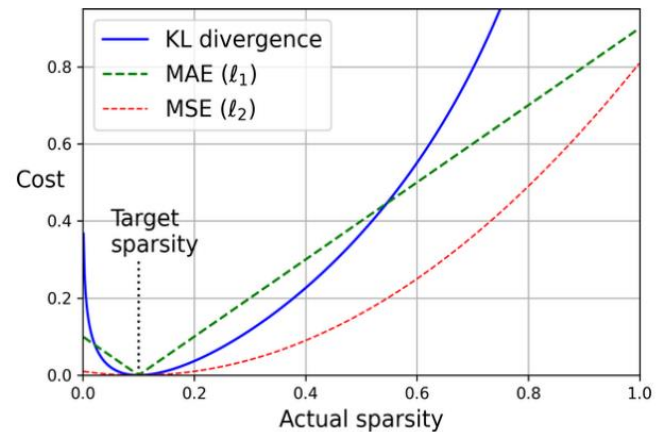
Sparsity Loss: Once we have the mean activation per neuron, we want to penalize the neurons that are too active, or not active enough, by adding a *sparsity loss* to the cost function. For example, if we measure that a neuron has an average activation of 0.3, but the target sparsity is 0.1, it must be

penalized to activate less. One approach could be simply adding the squared error $(0.3 - 0.1)^2$ to the cost function.

Kullback- Leibler Divergence: in practice a better approach is to use the Kullback– Leibler (KL) divergence, which has much stronger gradients than the mean squared error (see figure).

Given two discrete probability distributions P and Q , the KL divergence between these distributions, noted $D_{KL}(P \parallel Q)$, can be computed using equation below.

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$



In our case, we want to measure the divergence between the target probability p that a neuron in the coding layer will activate and the actual probability q , estimated by measuring the mean activation over the training batch. So, the KL divergence simplifies to equation below.

$$D_{KL}(p \parallel q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}$$

Once we have computed the sparsity loss for each neuron in the coding layer, we sum up these losses and add the result to the cost function. In order to control the relative importance of the sparsity loss and the reconstruction loss, we can multiply the sparsity loss by a sparsity weight hyperparameter. If this weight is too high, the model will stick closely to the target sparsity, but it may not reconstruct the inputs properly, making the model useless. Conversely, if it is too low, the model will mostly ignore the sparsity objective and will not learn any interesting features.

49. Identify the characteristics and working principles of *variational autoencoders*. Construct a neat diagram showing its architecture and develop relevant code-snippets for its implementation.

Answer:

One of the most popular autoencoder variants is *variational autoencoders* (VAEs). VAEs are quite different from all the autoencoders, in these particular ways:

- They are probabilistic autoencoders, meaning that their outputs are partly determined by chance, even after training (as opposed to denoising autoencoders, which use randomness only during training).

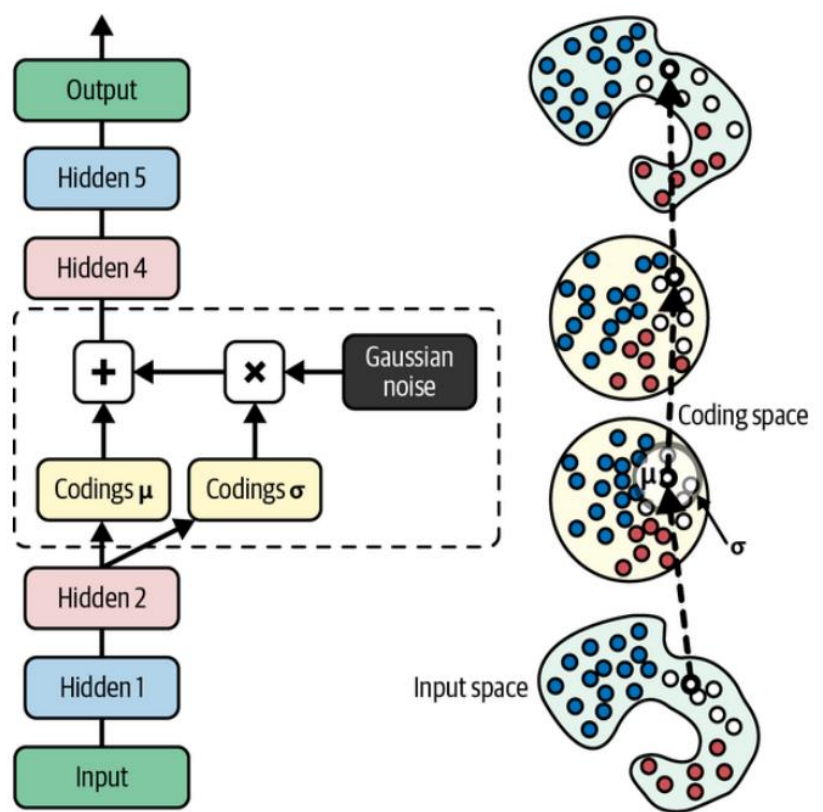
- Most importantly, they are generative autoencoders, meaning that they can generate new instances that look like they were sampled from the training set.

Variational autoencoders perform variational Bayesian inference, which is an efficient way of carrying out approximate Bayesian inference. Bayesian inference means updating a probability distribution based on new data, using equations derived from Bayes' theorem. The original distribution is called the *prior*, while the updated distribution is called the *posterior*. In our case, we want to find a good approximation of the data distribution. Once we have that, we can sample from it.

How It Works

The figure (left) shows a variational autoencoder. You can recognize the basic structure of all autoencoders, with an encoder followed by a decoder (in this example, they both have two hidden layers), but there is a twist: instead of directly producing a coding for a given input, the encoder produces a *mean coding* μ and a standard deviation σ . The actual coding is then sampled randomly from a Gaussian distribution with mean μ and standard deviation σ . After that, the decoder decodes the sampled coding normally. The right part of the diagram shows a training instance going through this autoencoder. First, the encoder produces μ and σ , then a coding is sampled randomly (it is not exactly located at μ), and finally this coding is decoded; the final output resembles the training instance.

As you can see in the diagram, although the inputs may have a very convoluted distribution, a variational autoencoder tends to produce codings that look as though they were sampled from a simple Gaussian distribution: during training, the cost function (discussed next) pushes the codings to gradually migrate within the coding space (also called the latent space) to end up looking like a cloud of Gaussian points. One great consequence is that after training a variational autoencoder, you can very easily generate a new instance: just sample a random coding from the Gaussian distribution and decode it.



Cost Function: The cost function is composed of two parts. The first is the usual reconstruction loss that pushes the autoencoder to reproduce its inputs. We can use the MSE for this. The second is the *latent loss* that pushes the autoencoder to have codings that look as though they were sampled from a simple Gaussian distribution: it is the KL divergence between the target distribution (i.e., the Gaussian distribution) and the actual distribution of the codings. The Gaussian noise limits the amount of information that can be transmitted to the coding layer. This pushes the autoencoder to learn useful features. The latent loss can be computed using equation below.

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2]$$

In this equation, $\mathcal{L}$ is the latent loss, n is the codings' dimensionality, and μ_i and σ_i are the mean and standard deviation of the i^{th} component of the codings. The vectors $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (which contain all the μ_i and σ_i) are output by the encoder, as shown in the above figure (left).

A common tweak to the variational autoencoder's architecture is to make the encoder output $\boldsymbol{\gamma} = \log(\boldsymbol{\sigma}^2)$ rather than $\boldsymbol{\sigma}$. The latent loss can then be computed as shown in equation below. This approach is more numerically stable and speeds up training.

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \gamma_i - \exp(\gamma_i) - \mu_i^2]$$

Implementation

The following code-snippets build a variational autoencoder for Fashion MNIST. First, we will need a custom layer to sample the codings, given $\boldsymbol{\mu}$ and $\boldsymbol{\gamma}$: This Sampling layer takes two inputs: mean ($\boldsymbol{\mu}$) and log_var ($\boldsymbol{\gamma}$). It samples a random vector (of the same shape as $\boldsymbol{\gamma}$) from the Gaussian distribution, with mean 0 and standard deviation 1. Then it multiplies it by $\exp(\boldsymbol{\gamma} / 2)$ (which is equal to $\boldsymbol{\sigma}$), and finally it adds $\boldsymbol{\mu}$ and returns the result. This samples a codings vector from the Gaussian distribution with mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$.

```
class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs
        return tf.random.normal(tf.shape(log_var)) * tf.exp(log_var / 2) + mean
```

Next, we can create the encoder, using the functional API because the model is not entirely sequential:

```
codings_size = 10

inputs = tf.keras.layers.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)

codings_mean = tf.keras.layers.Dense(codings_size)(Z)
codings_log_var = tf.keras.layers.Dense(codings_size)(Z)
```

$\boldsymbol{\mu}$

$\boldsymbol{\gamma}$

```
codings = Sampling()([codings_mean, codings_log_var])

variational_encoder = tf.keras.Model(
    inputs=[inputs],
    outputs=[codings_mean, codings_log_var, codings]
)
```

Now let's build the decoder:

```
decoder_inputs = tf.keras.layers.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="relu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="relu")(x)
x = tf.keras.layers.Dense(28 * 28)(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)

variational_decoder = tf.keras.Model(inputs=[decoder_inputs], outputs=[outputs])
```

Finally, let's build the variational autoencoder model:

```
_, _, codings = variational_encoder(inputs)
reconstructions = variational_decoder(codings)

variational_ae = tf.keras.Model(inputs=[inputs], outputs=[reconstructions])
```

We ignore the first two outputs of the encoder (we only want to feed the codings to the decoder).

Lastly, we must add the latent loss and the reconstruction loss:

```
latent_loss = -0.5 * tf.reduce_sum( 1 + codings_log_var -
tf.exp(codings_log_var) - tf.square(codings_mean), axis=-1)
variational_ae.add_loss(tf.reduce_mean(latent_loss) / 784.)
```

And finally, we can compile and fit the autoencoder.

[illegible]